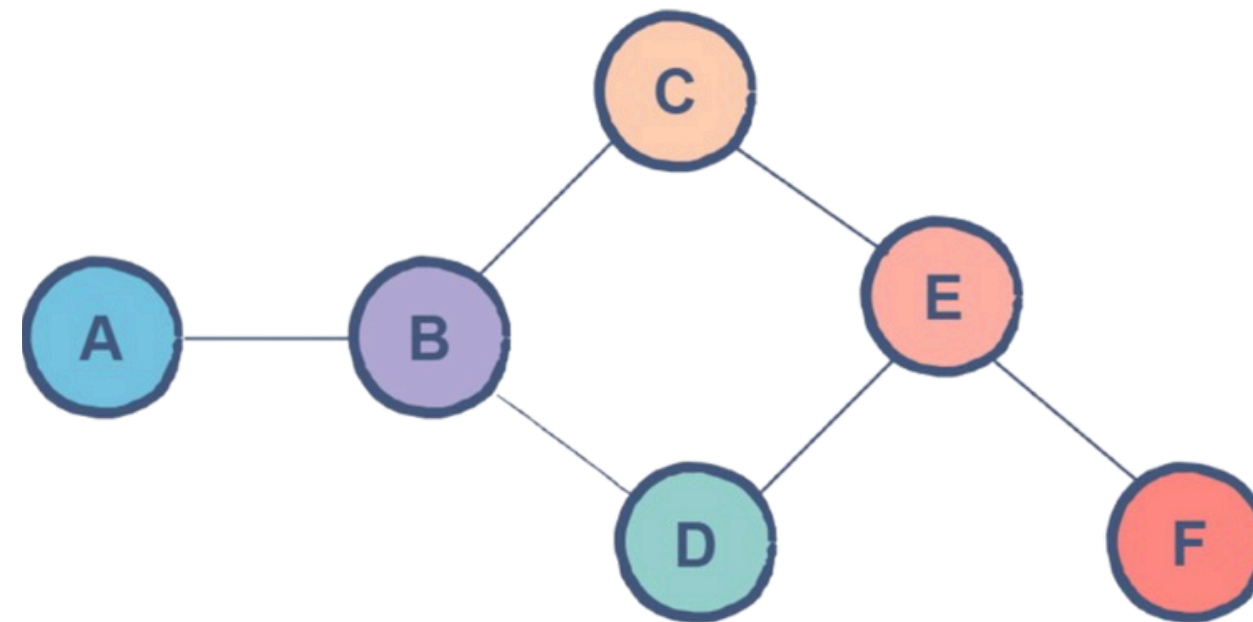
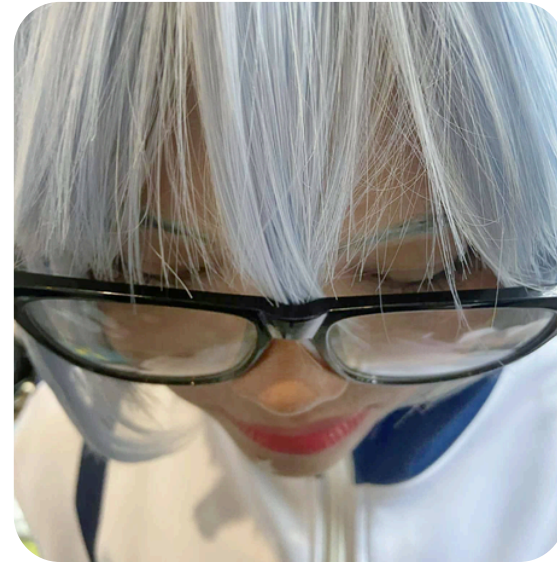


ELECTRICITY SIMULATION





PHEA



TY



DARA



CHAY

PRESENTED BY:

- Kong Sophea
- Chey Naryvety
- Bouy Dara
- Lim Vinchay





CONTENTS

1

Overview

2

Graph/Data Design

3

Algorithms & Workflow

4

Example Input/Output

5

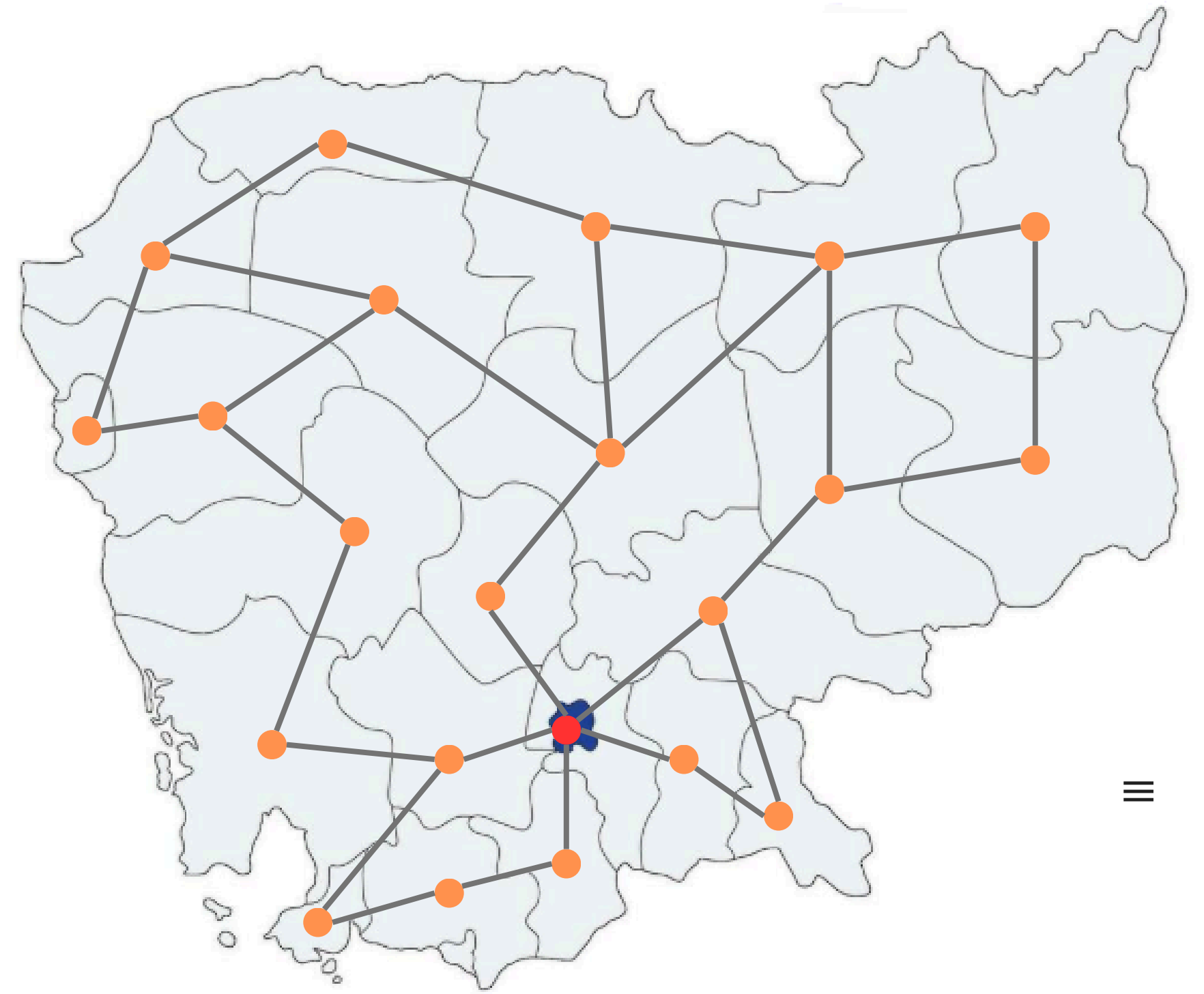
Conclusions

PROBLEM BACKGROUND

We build the electricity simulator because it is interesting and we believe it has real-world applications.

Modern power networks are growing in size and complexity, which creates challenges such as:

- Maintaining reliability when parts of the network fail.
- Avoiding outages caused by disconnected sections.
- Improving efficiency by reducing energy loss.



OVERVIEW



WHY GRAPH?

Grid mapping

As node can represent each provinces, each edge represent the wire line and weights to simulate the distance from each node to another

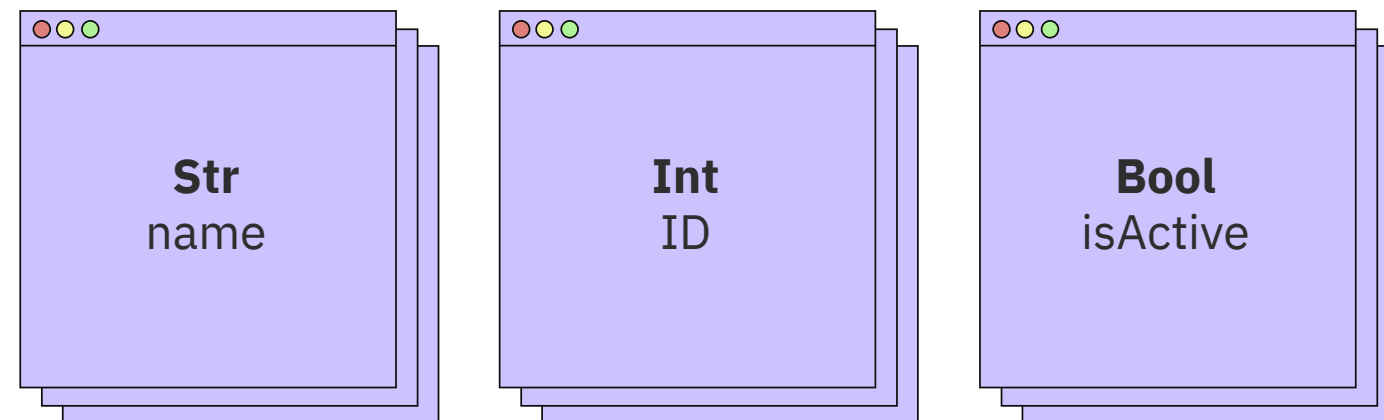
Grid analysis and simulation

Using graph help find the shortest path and also help identify which area or edge is effected in the most efficient way



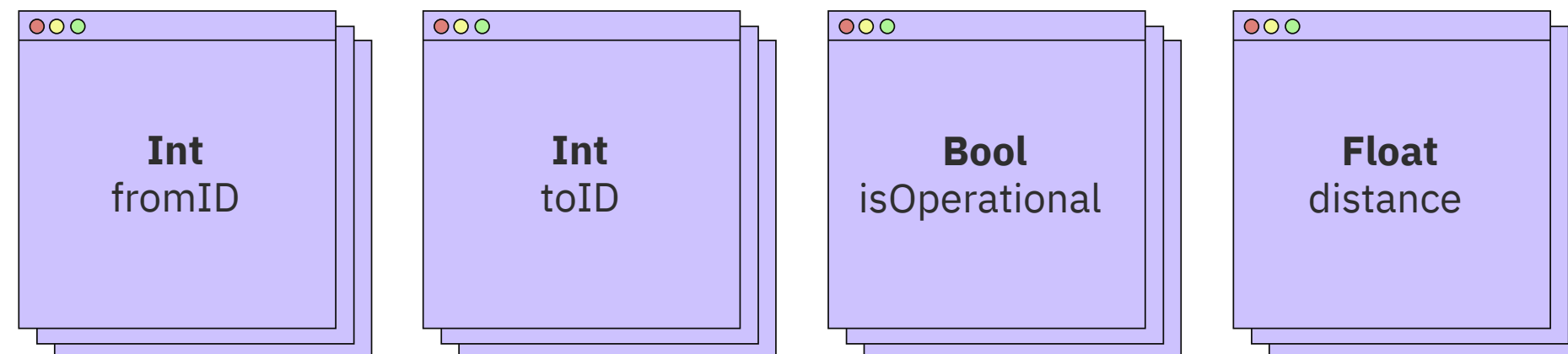
NODE REPRESENTATIONS

Nodes represent the electricity pole in a specific area.
Each node contains:



EDGE REPRESENTATIONS

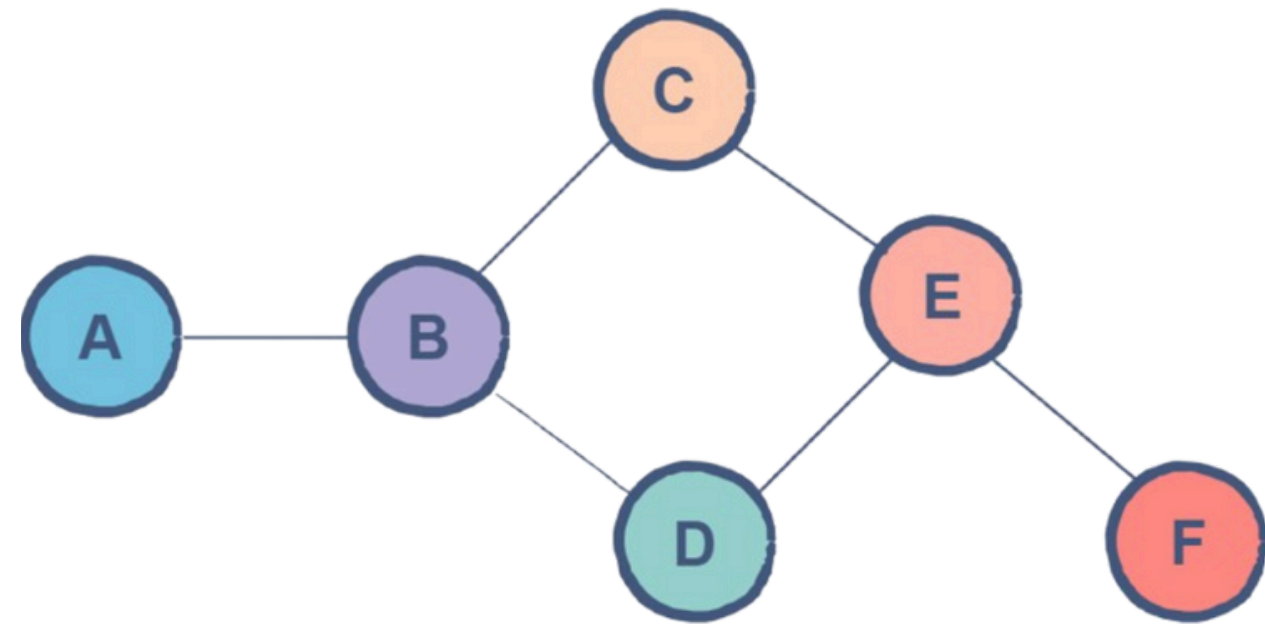
Edges represent the wires connecting each area. Each node contains:



```
class Graph {  
public:  
    struct Node {  
        string name;  
        int id;  
        bool is_active{true};  
    };  
  
    struct Edge {  
        int from;  
        int to;  
        double distance{0.0};  
        bool is_active{true};  
    };  
};
```

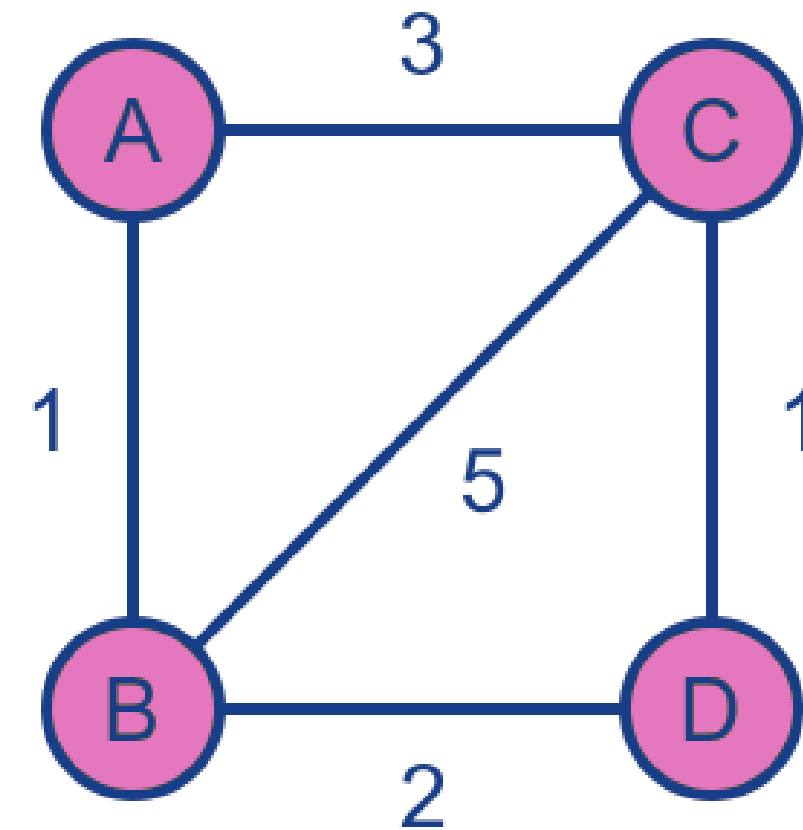
GRAPH DESIGN

GRAPH TYPE



Undirected Graph

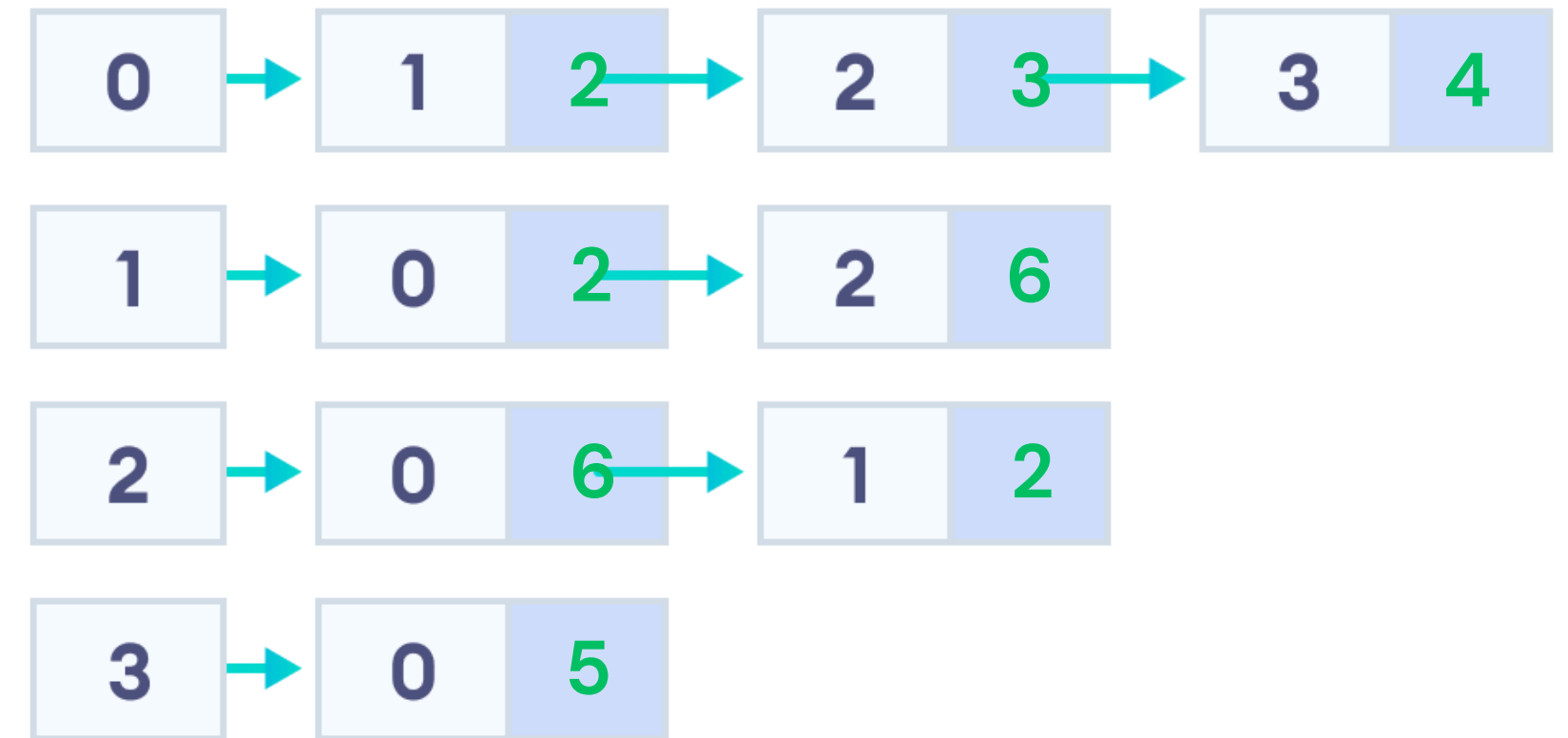
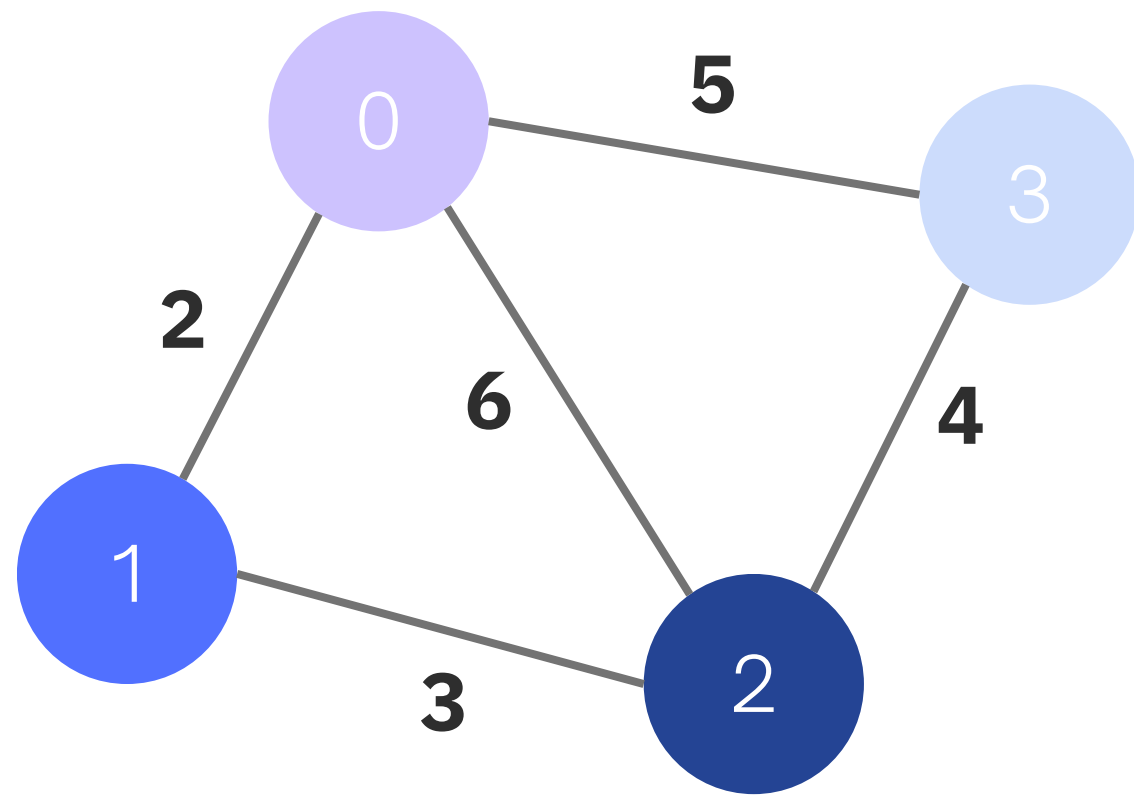
Electricity can flow in both directions between connected poles



Weighted Graph

Each wire has a distance value. It is used as the edge weight when calculating the shortest path.

GRAPH STRUCTURE



Undirected Weighted Graph and its Adjacency List

For each pole, stores all the nearby poles it connects to and the distance of each connection.

This makes graph traversal fast and efficient.





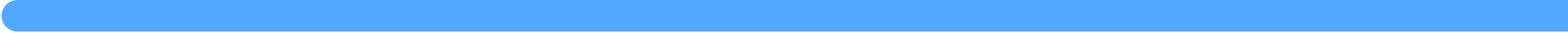
ALGORITHM USED



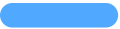
Connectivity

Loop Detection

Shortest Path



CONNECTIVITY



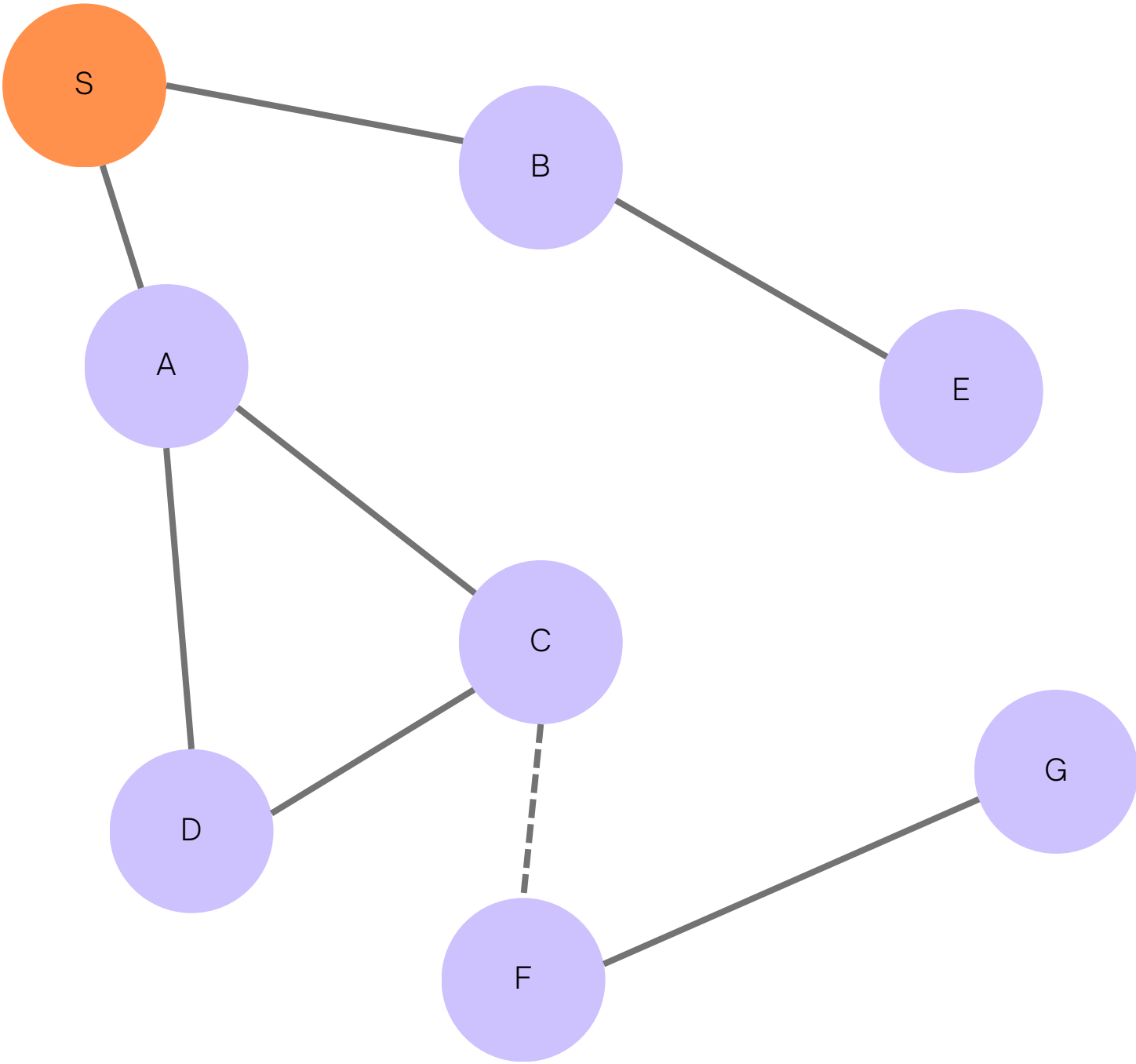
Q



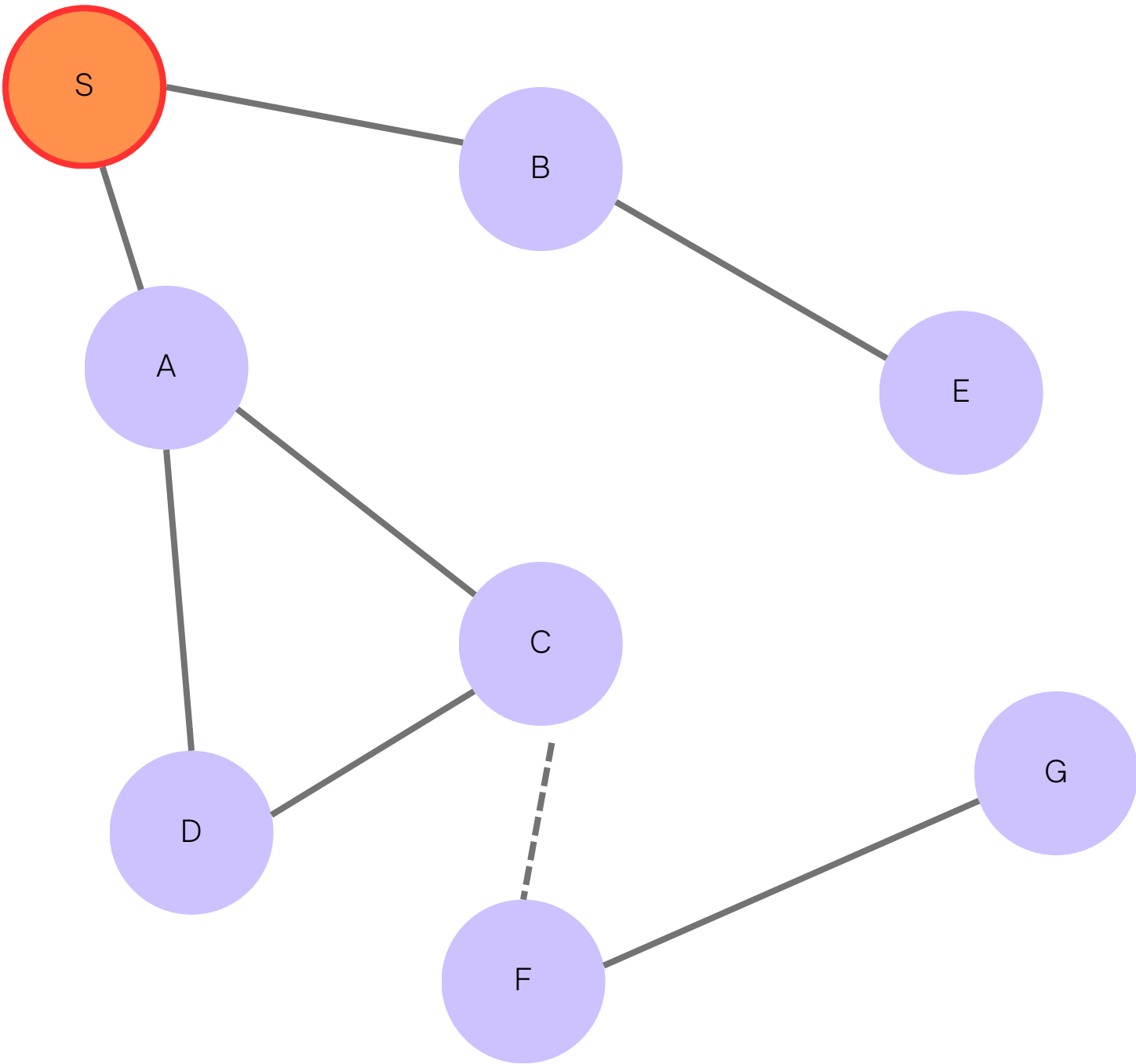
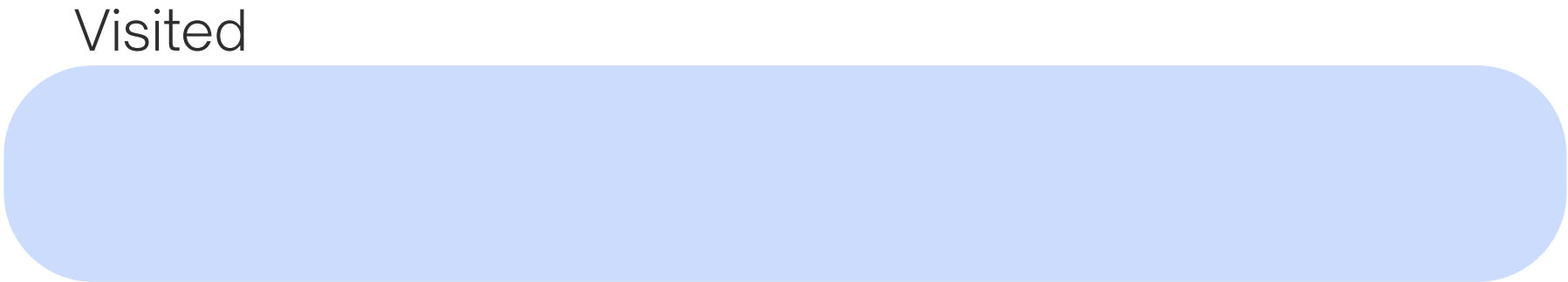
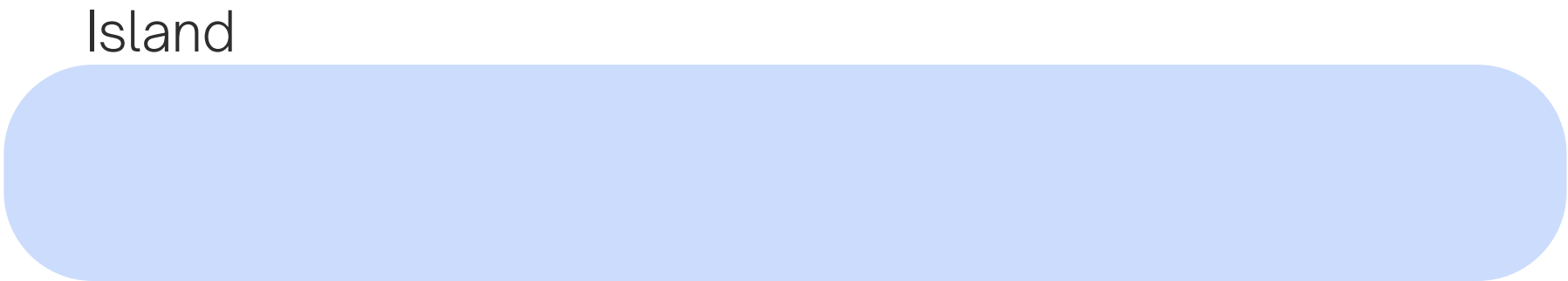
Island



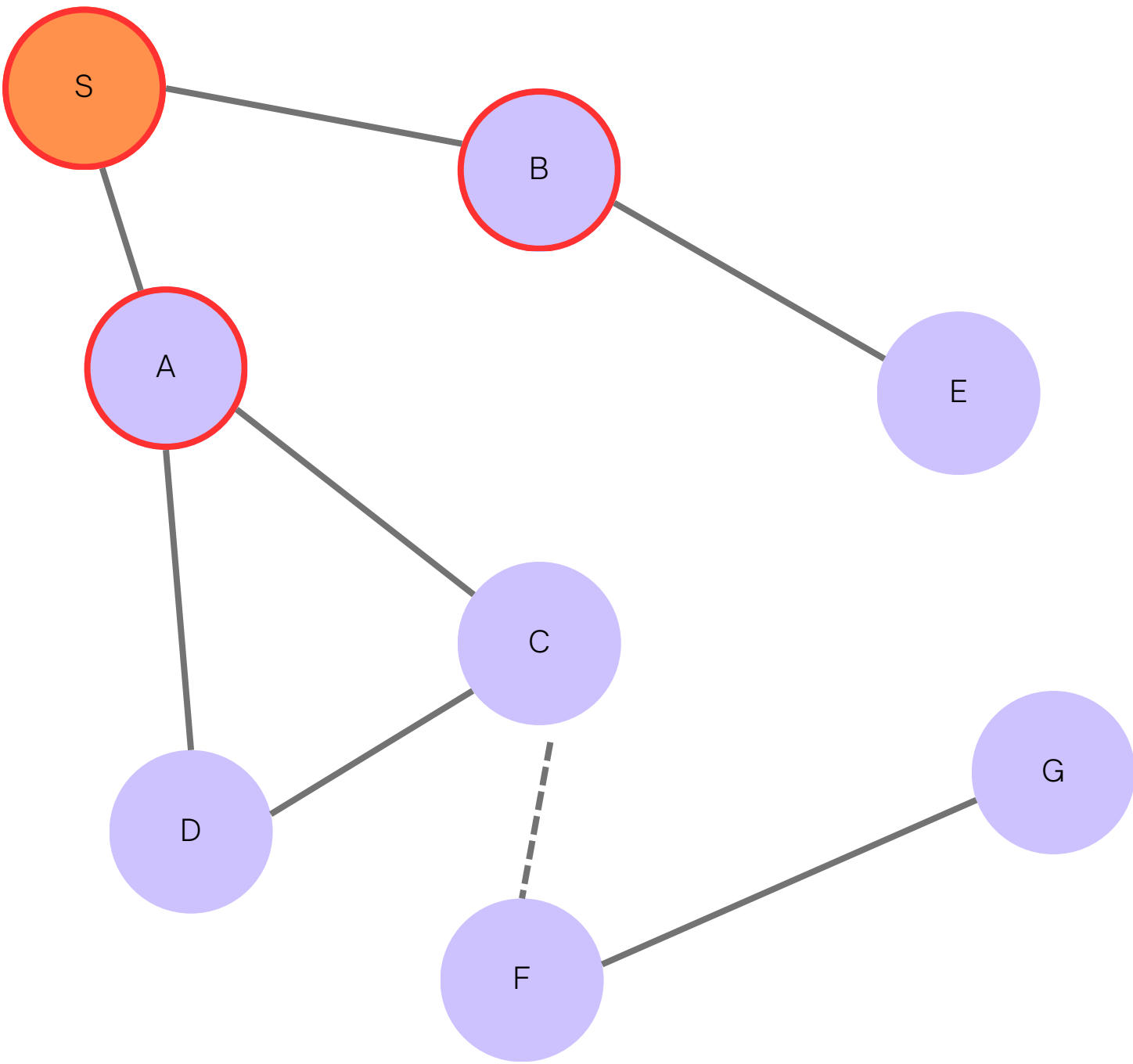
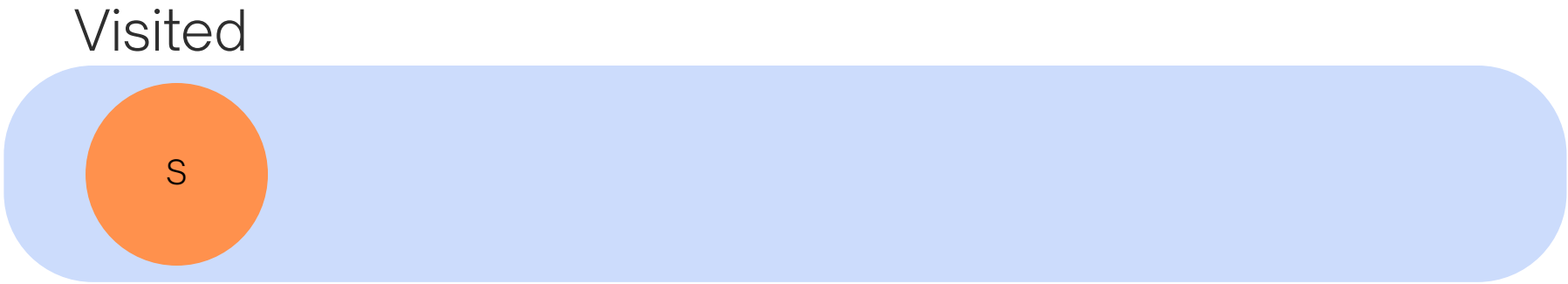
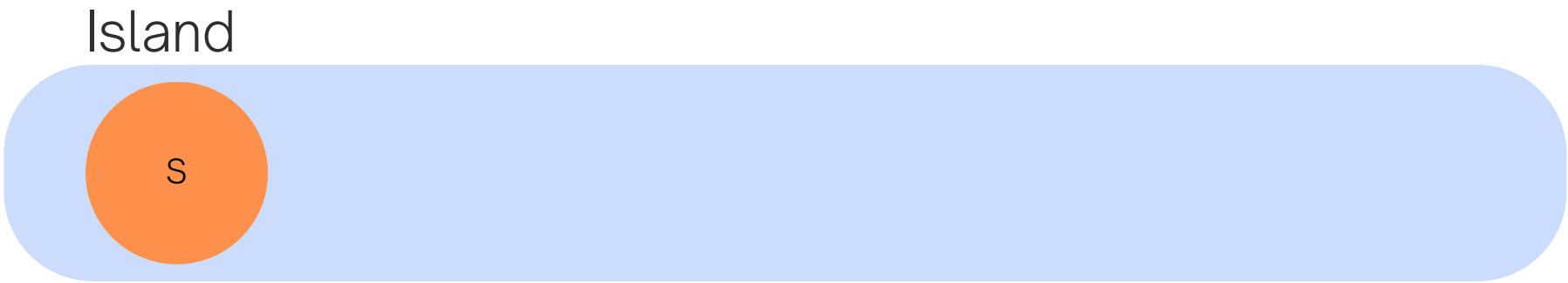
Visited



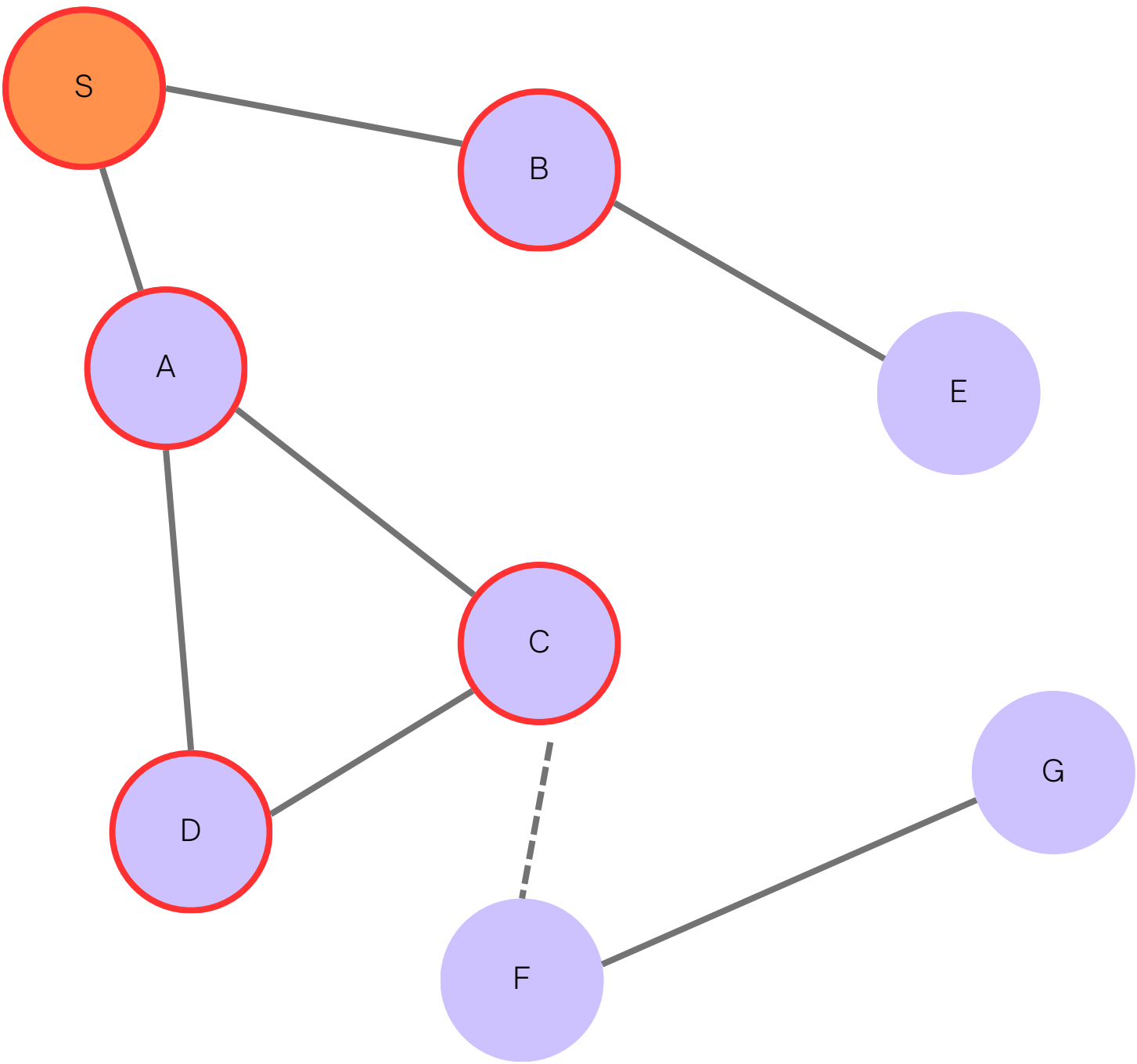
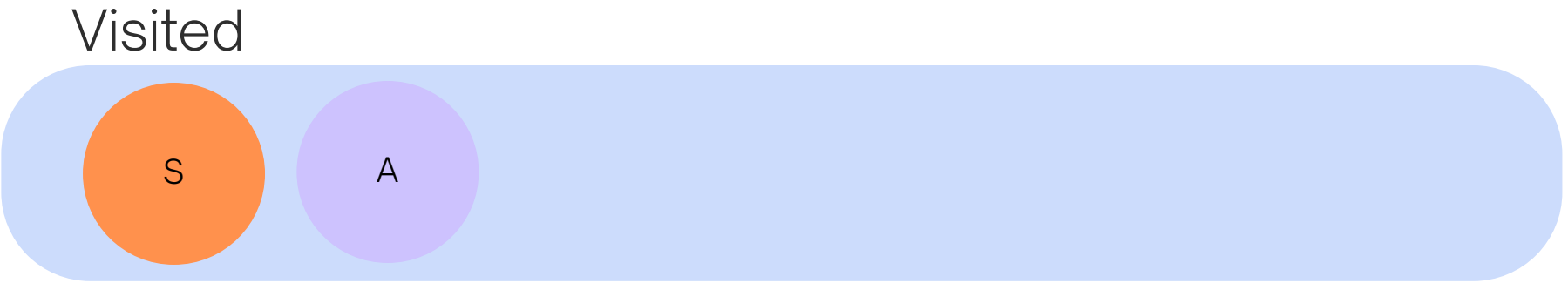
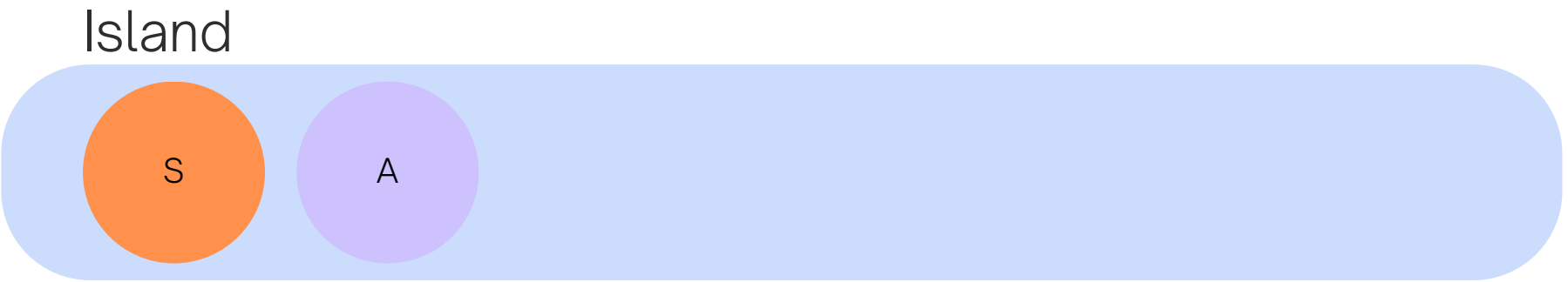
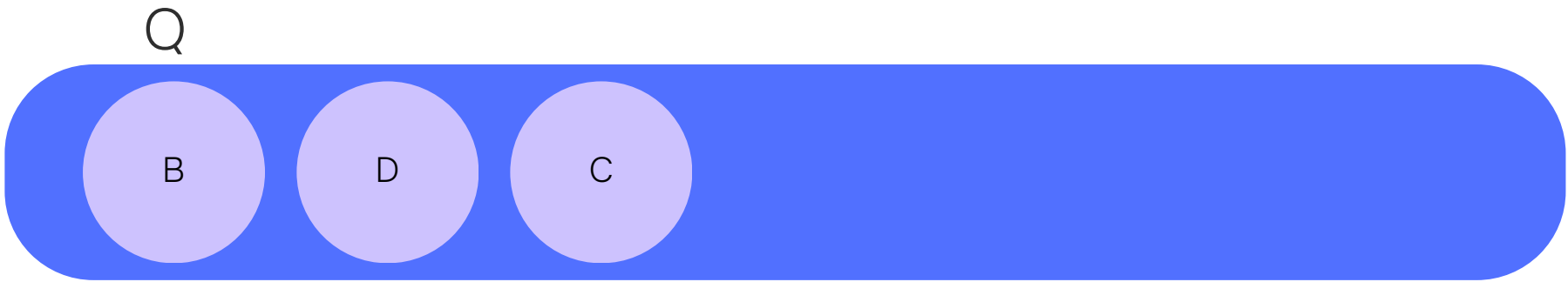
CONNECTIVITY



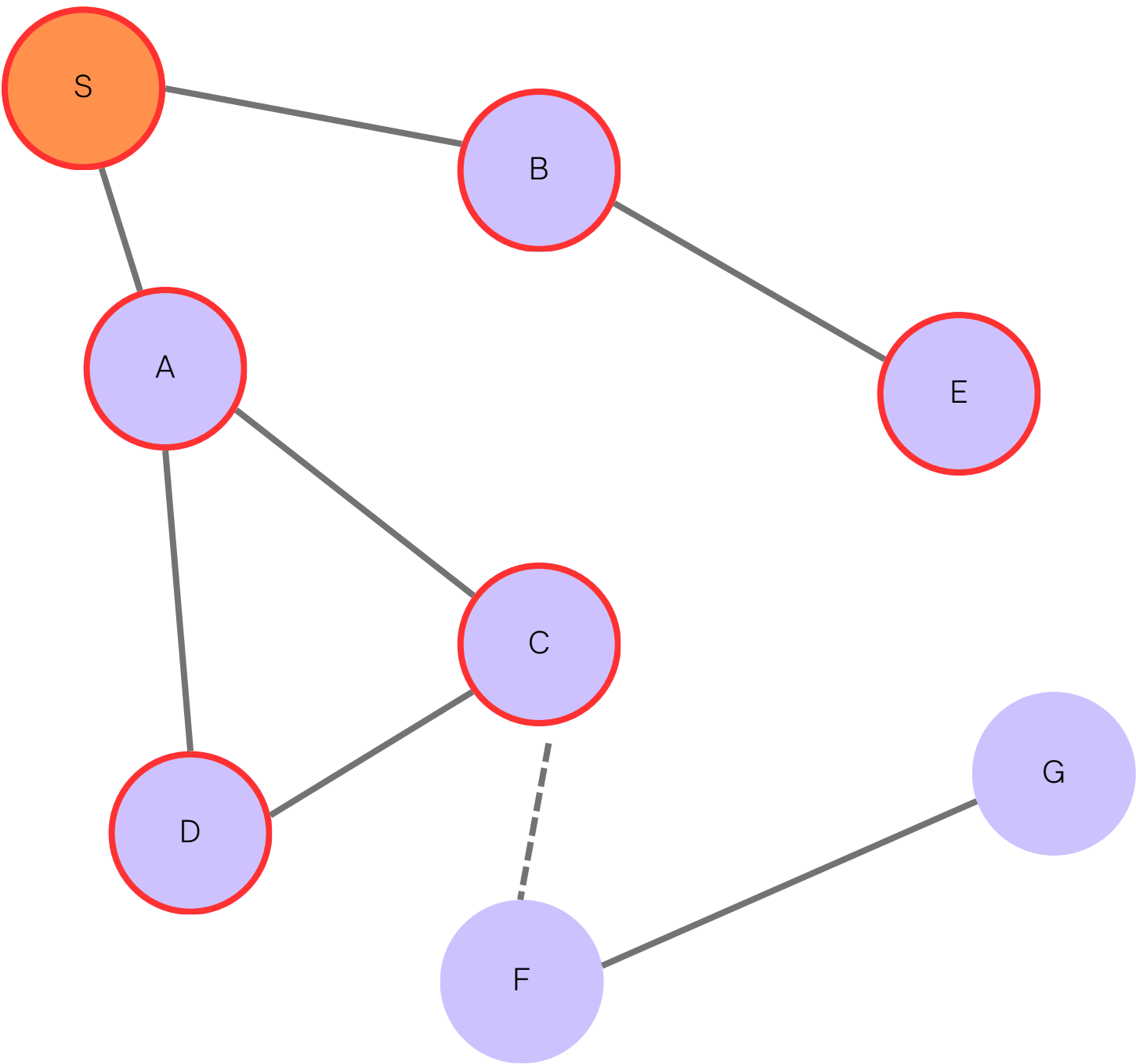
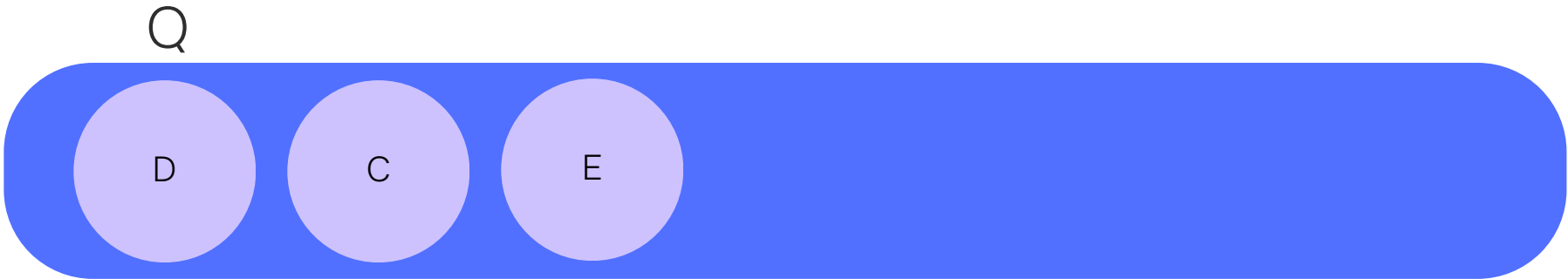
CONNECTIVITY



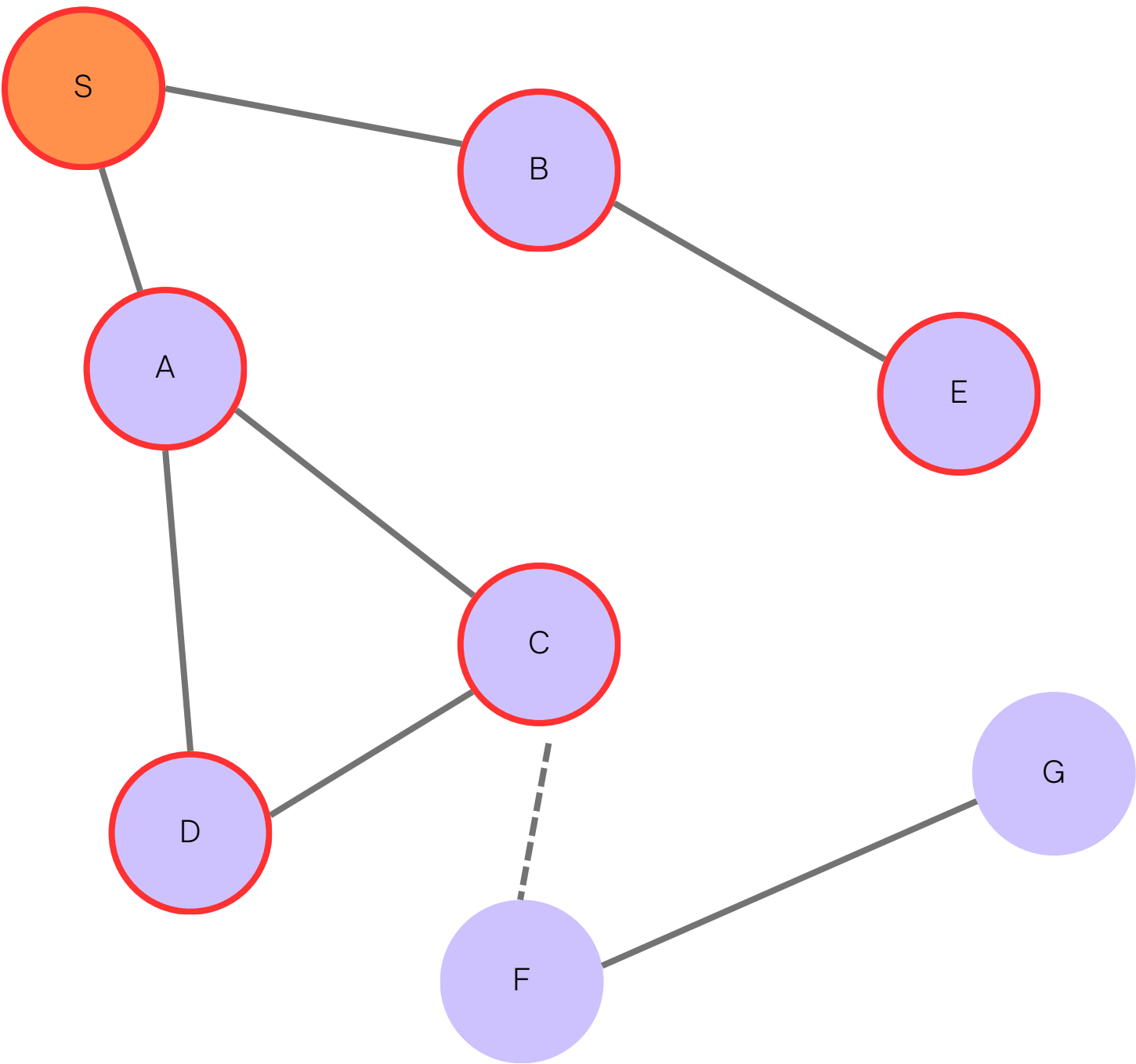
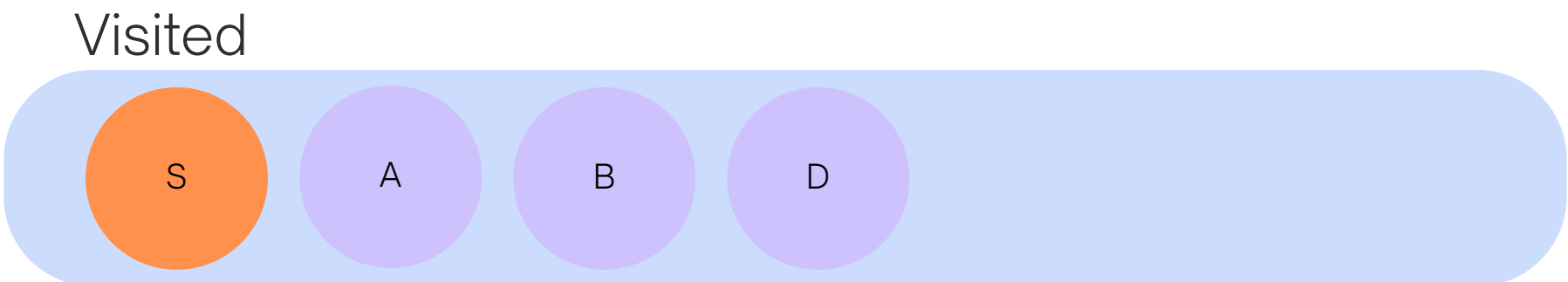
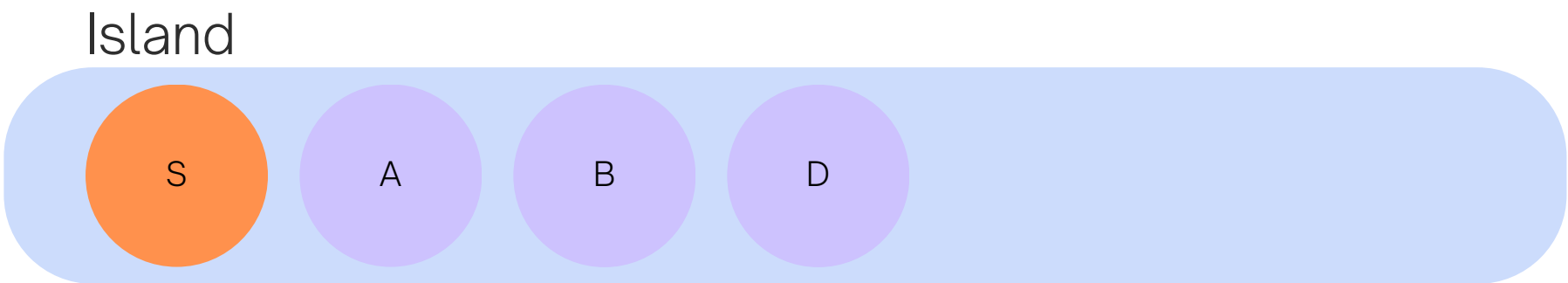
CONNECTIVITY



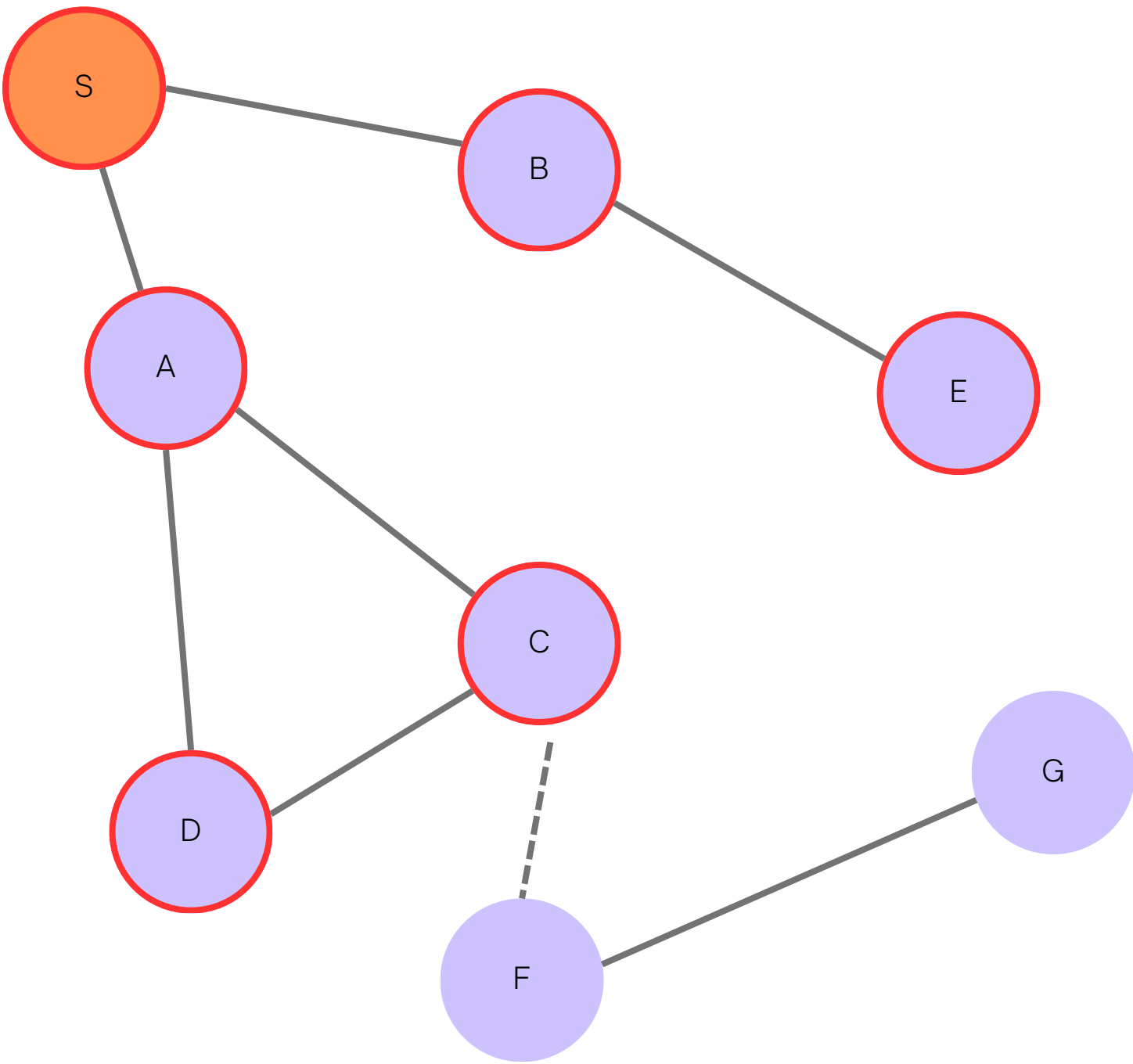
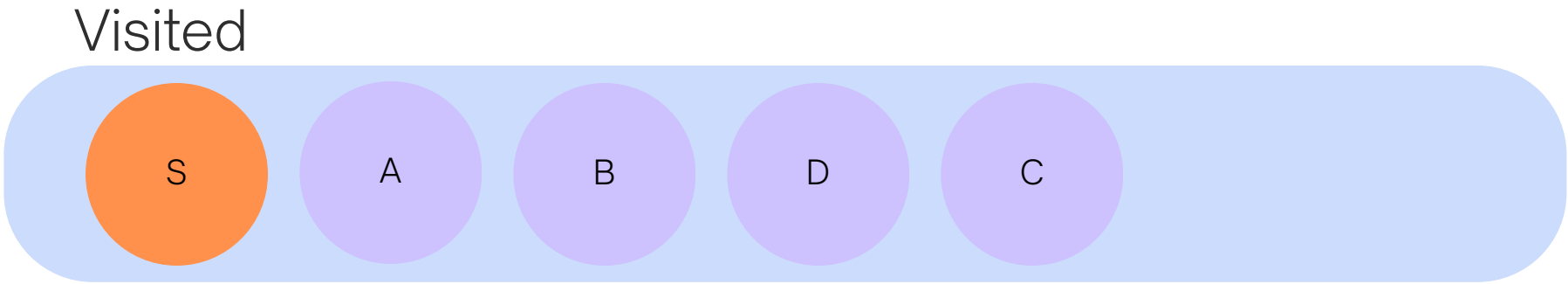
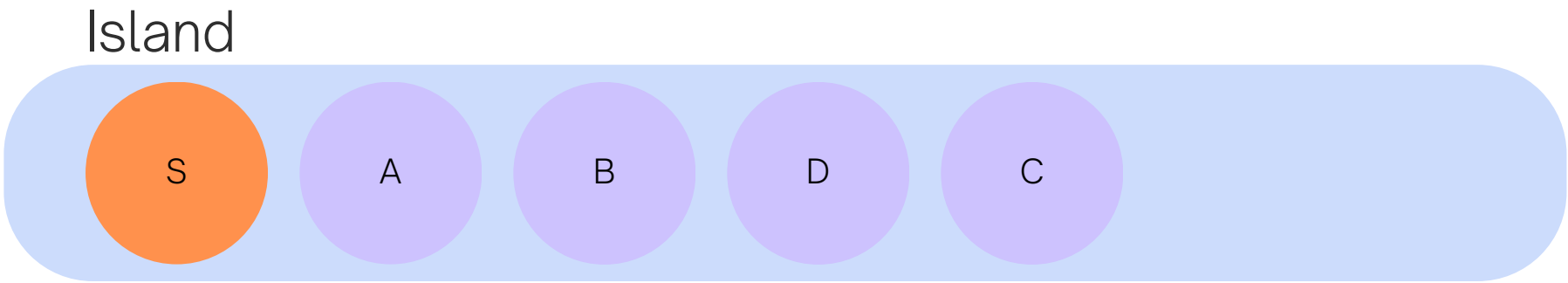
CONNECTIVITY



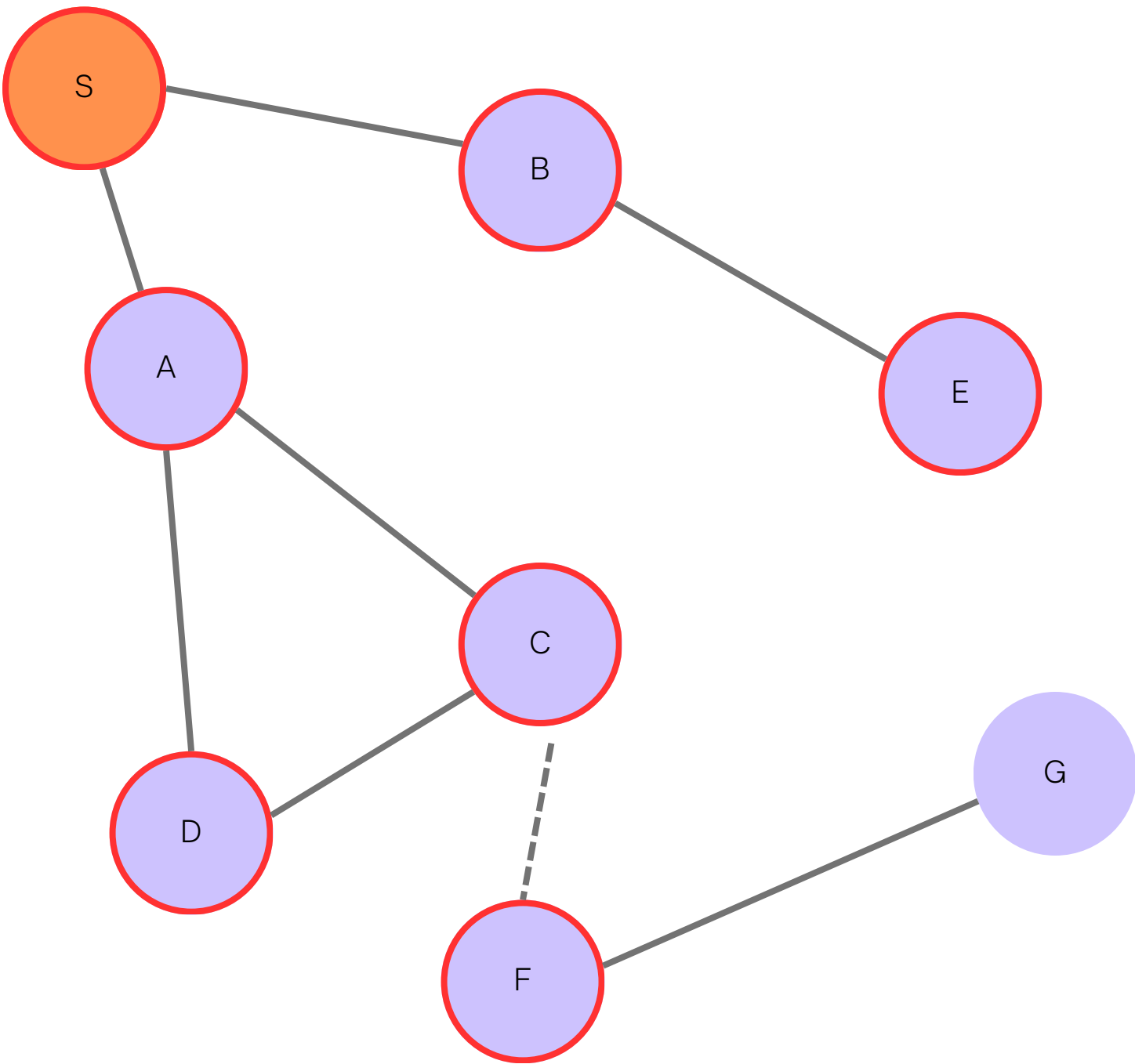
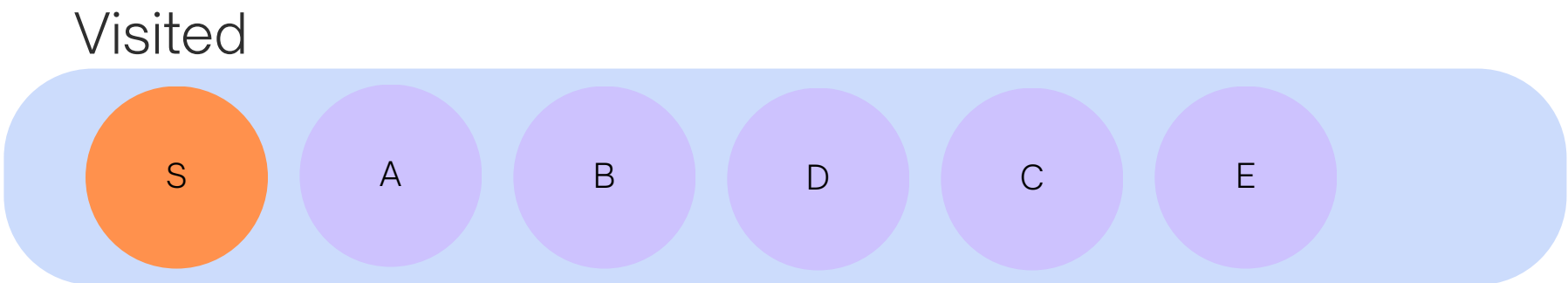
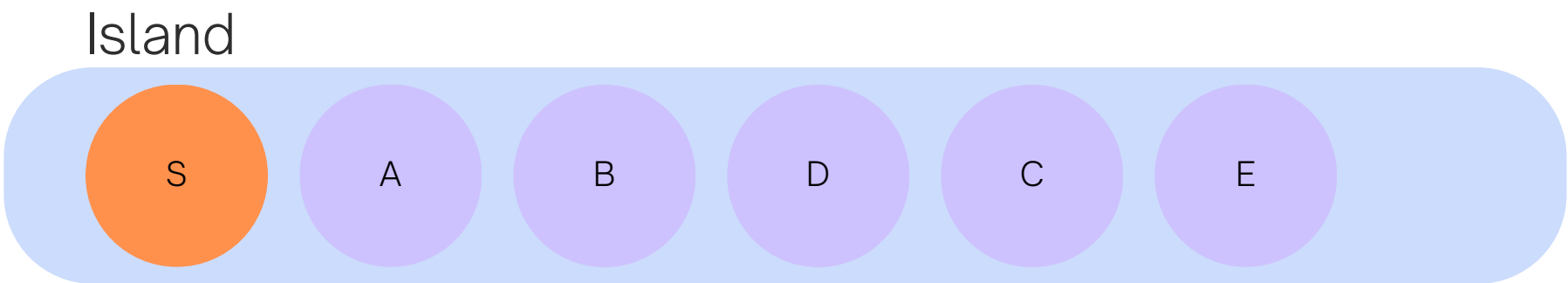
CONNECTIVITY



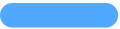
CONNECTIVITY



CONNECTIVITY



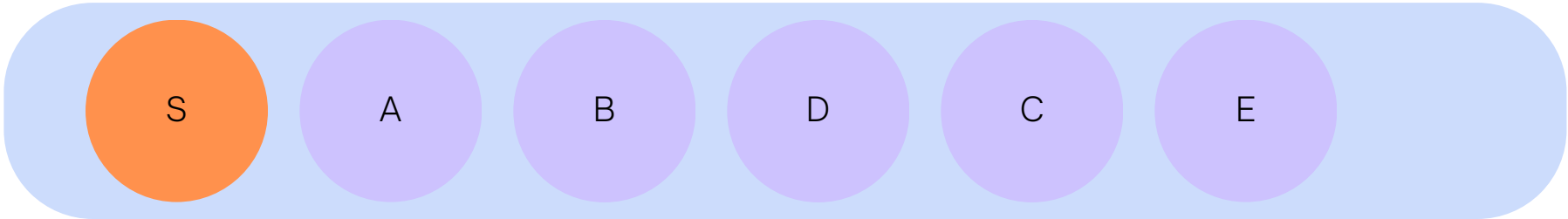
CONNECTIVITY



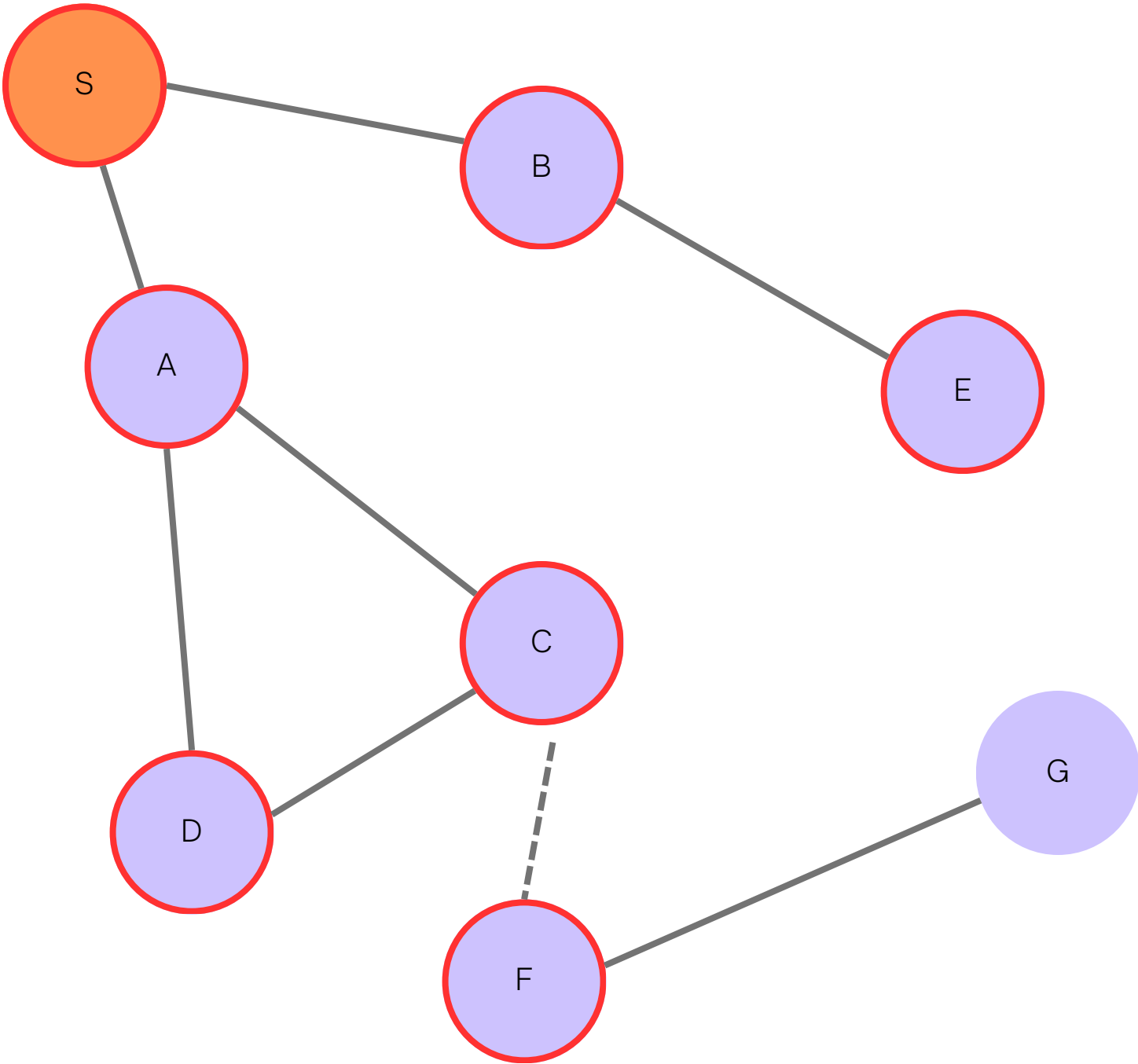
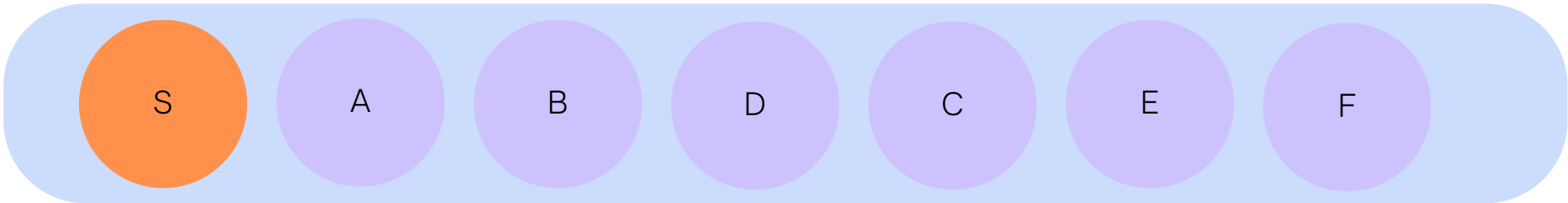
Q



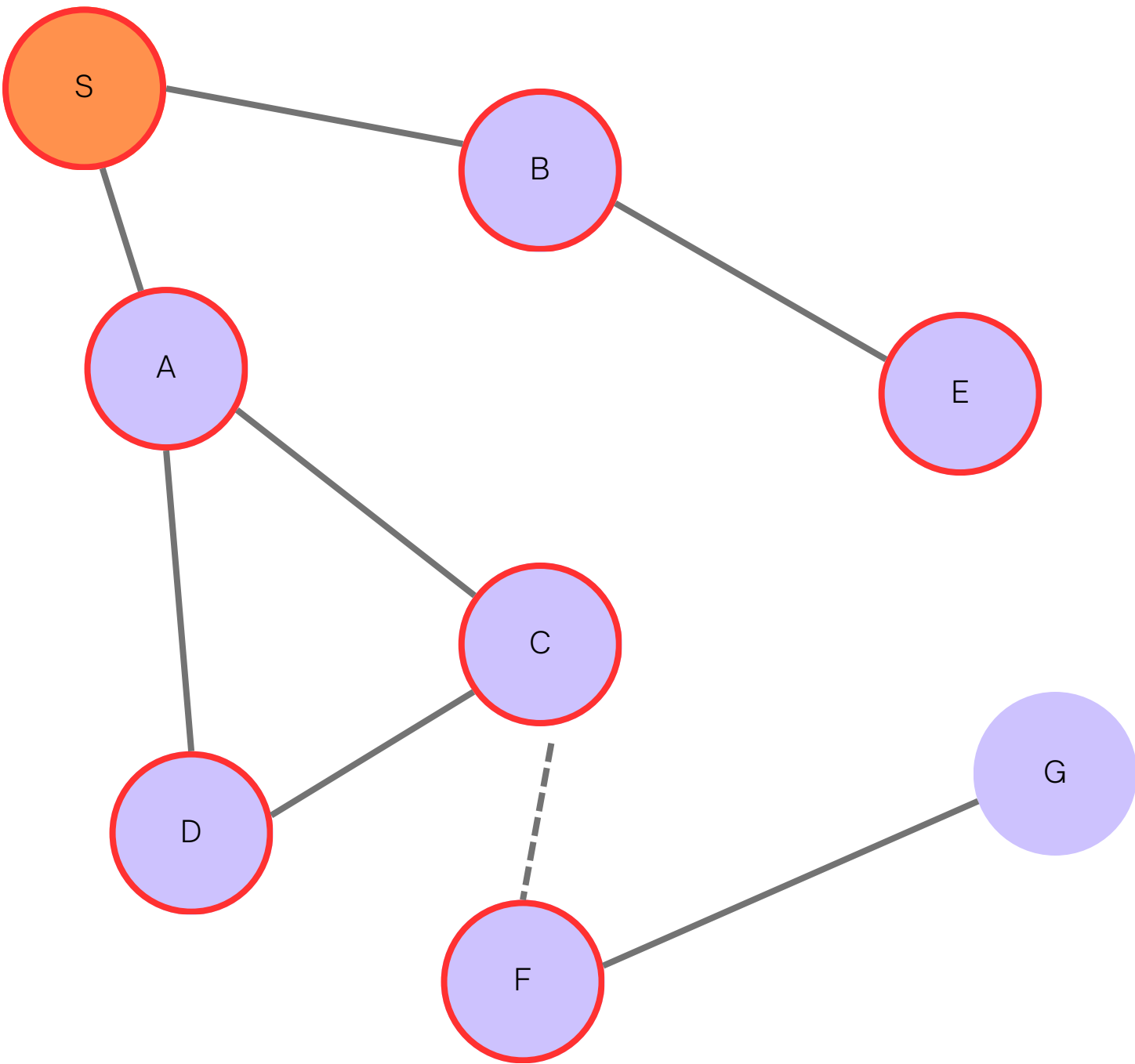
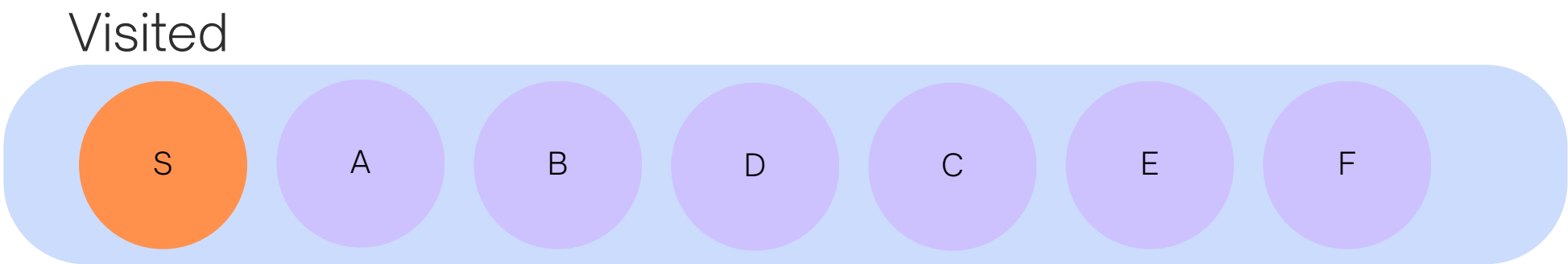
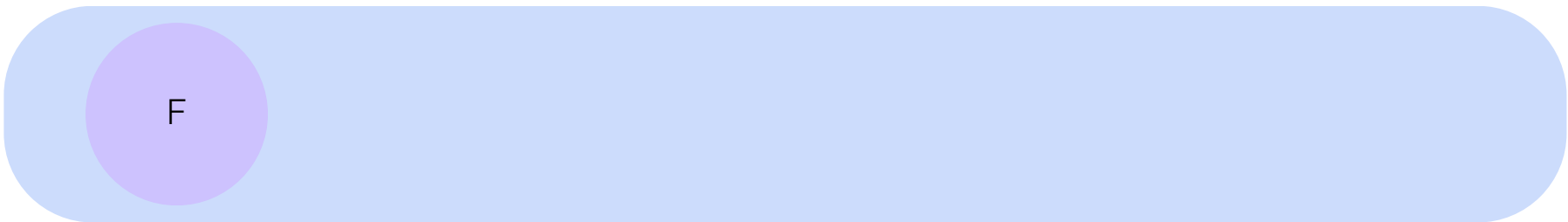
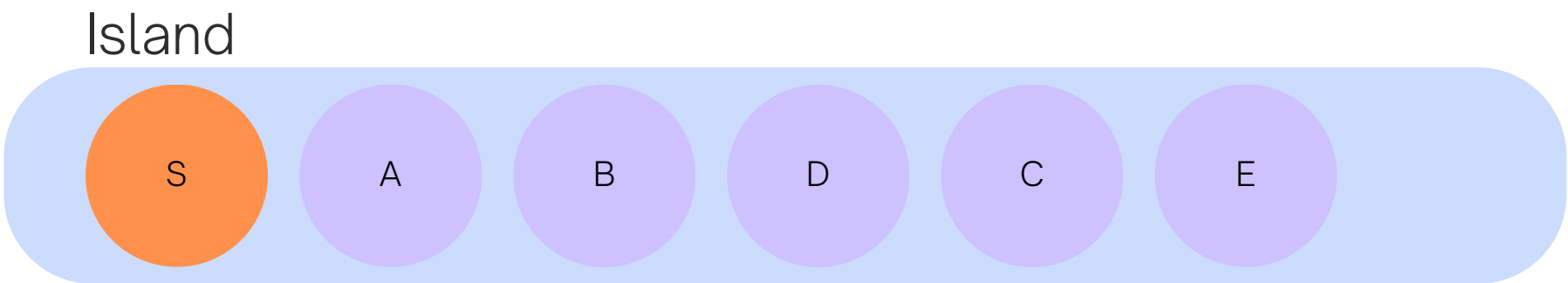
Island



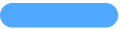
Visited



CONNECTIVITY



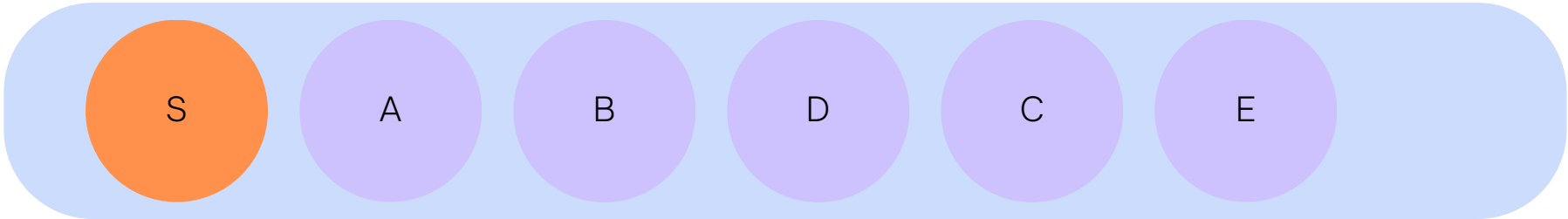
CONNECTIVITY



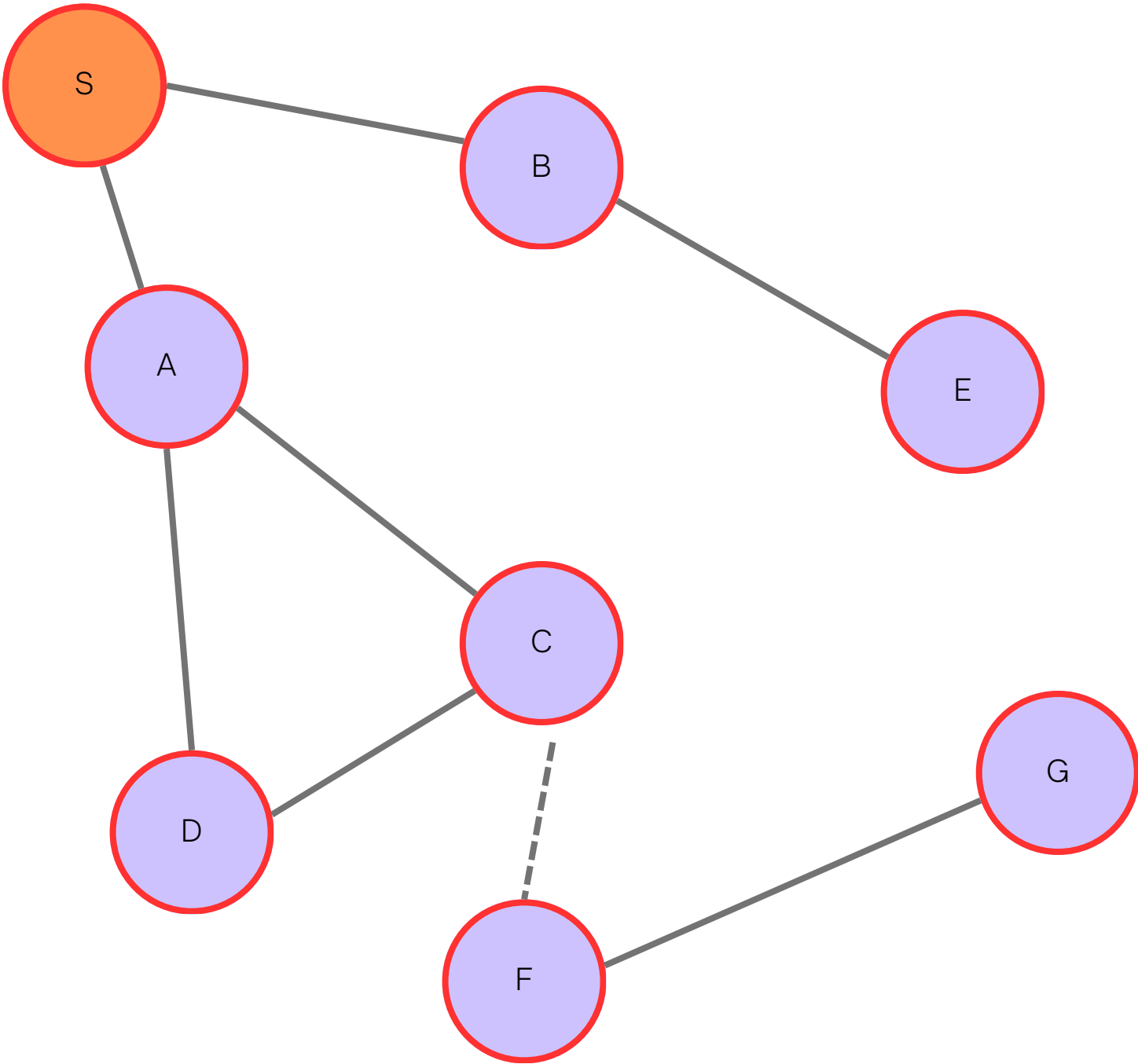
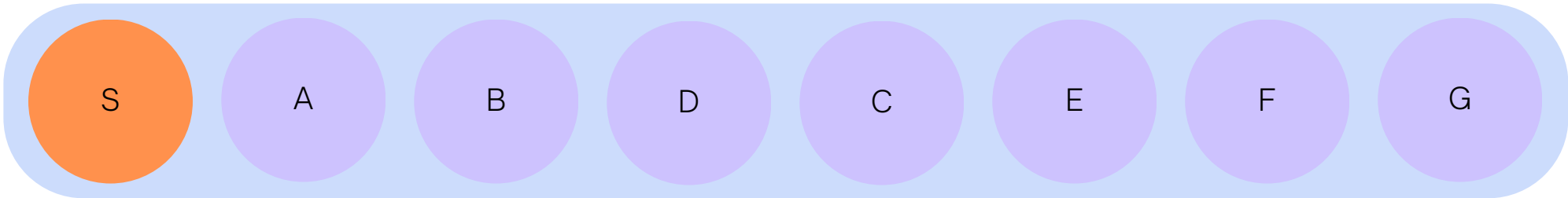
Q



Island



Visited

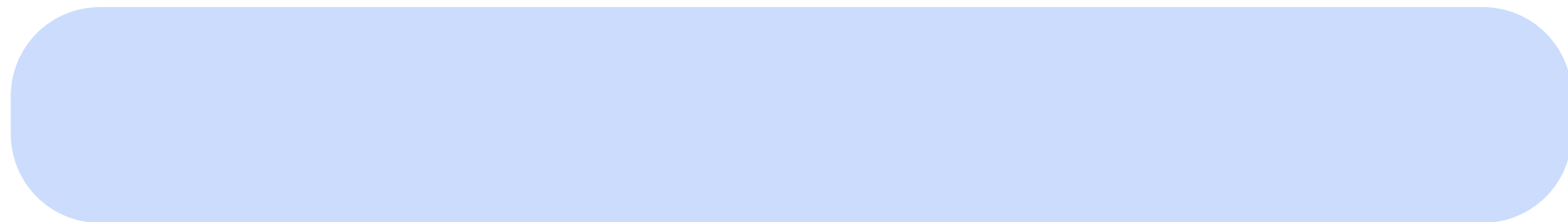


LOOP DETECTION

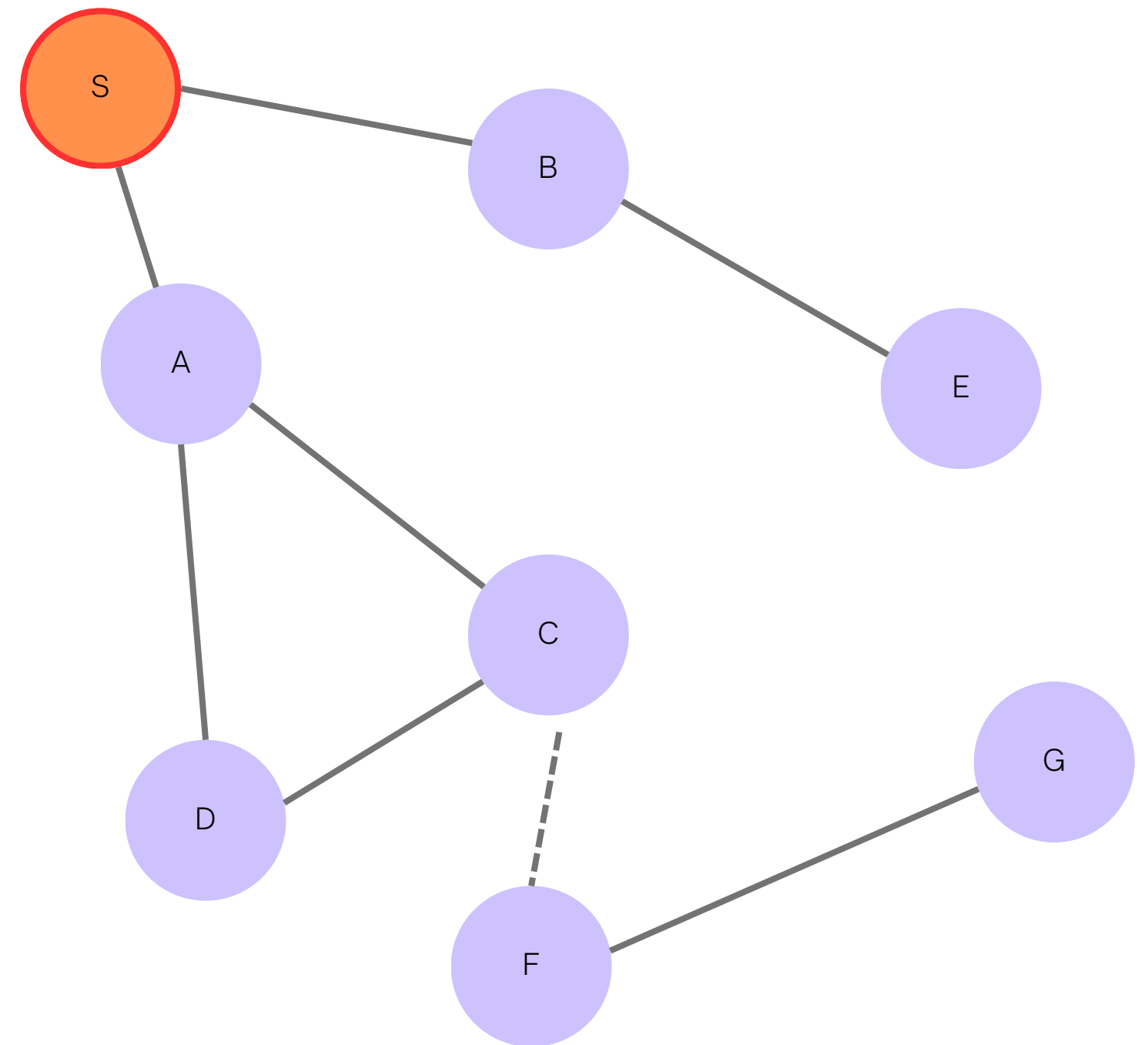
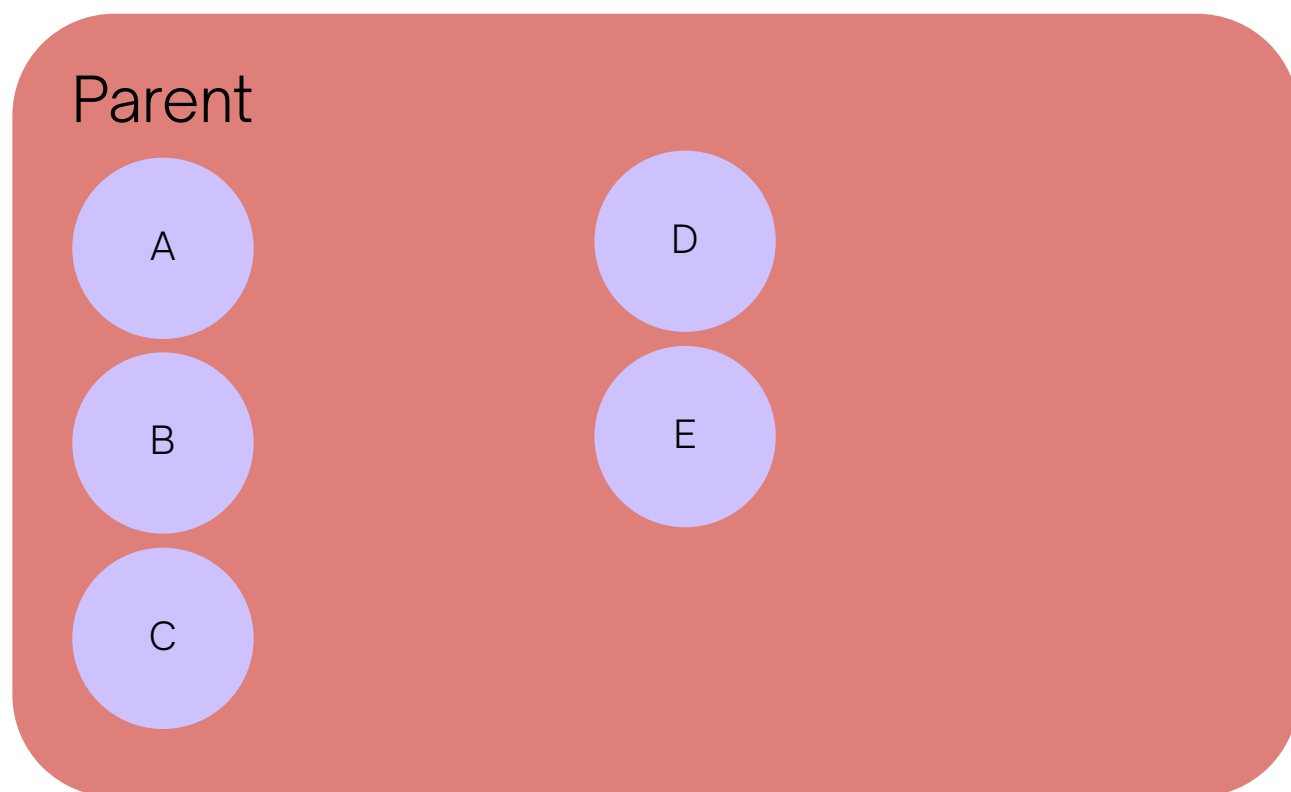
Stack



Visited



Parent



LOOP DETECTION

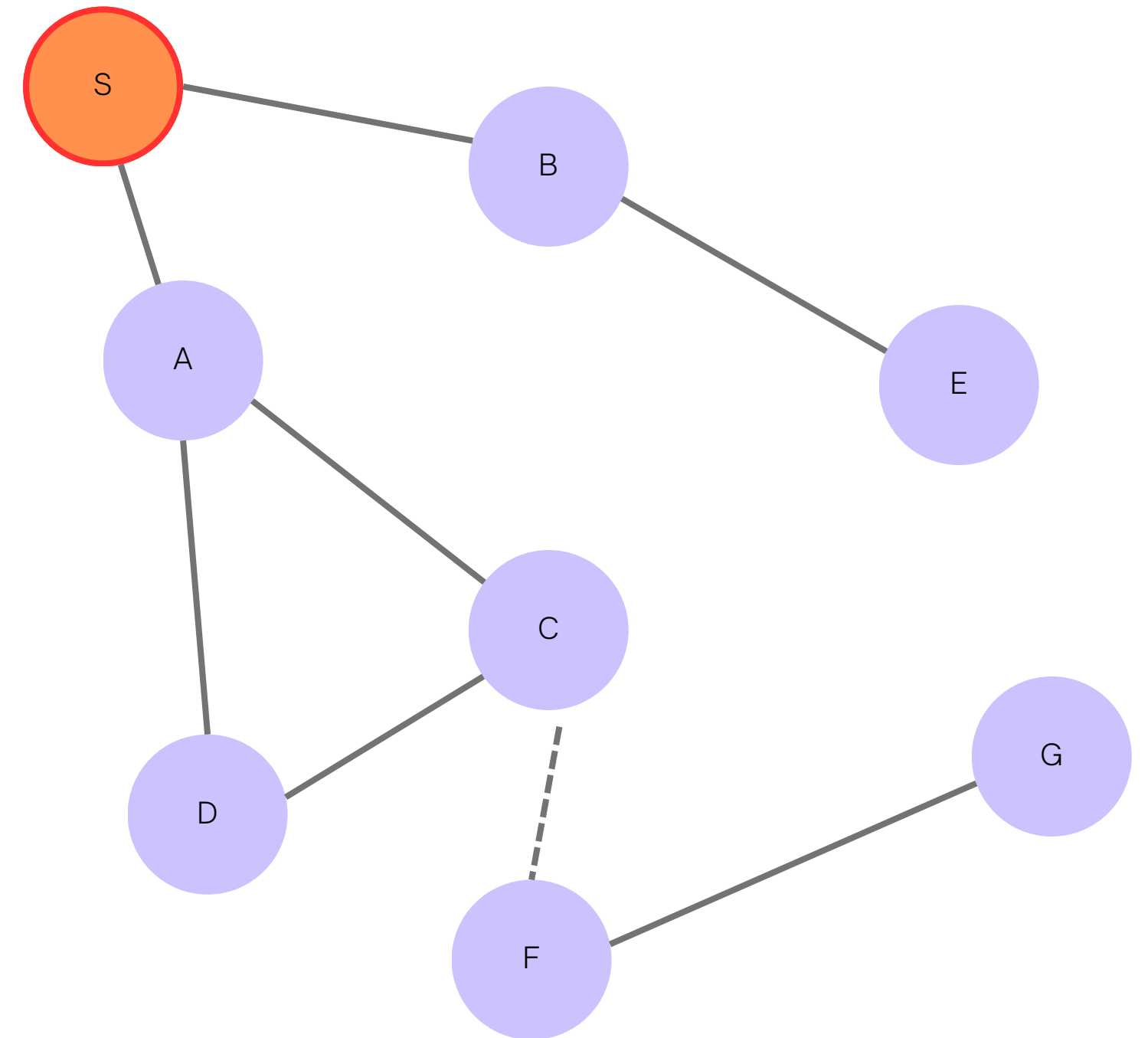
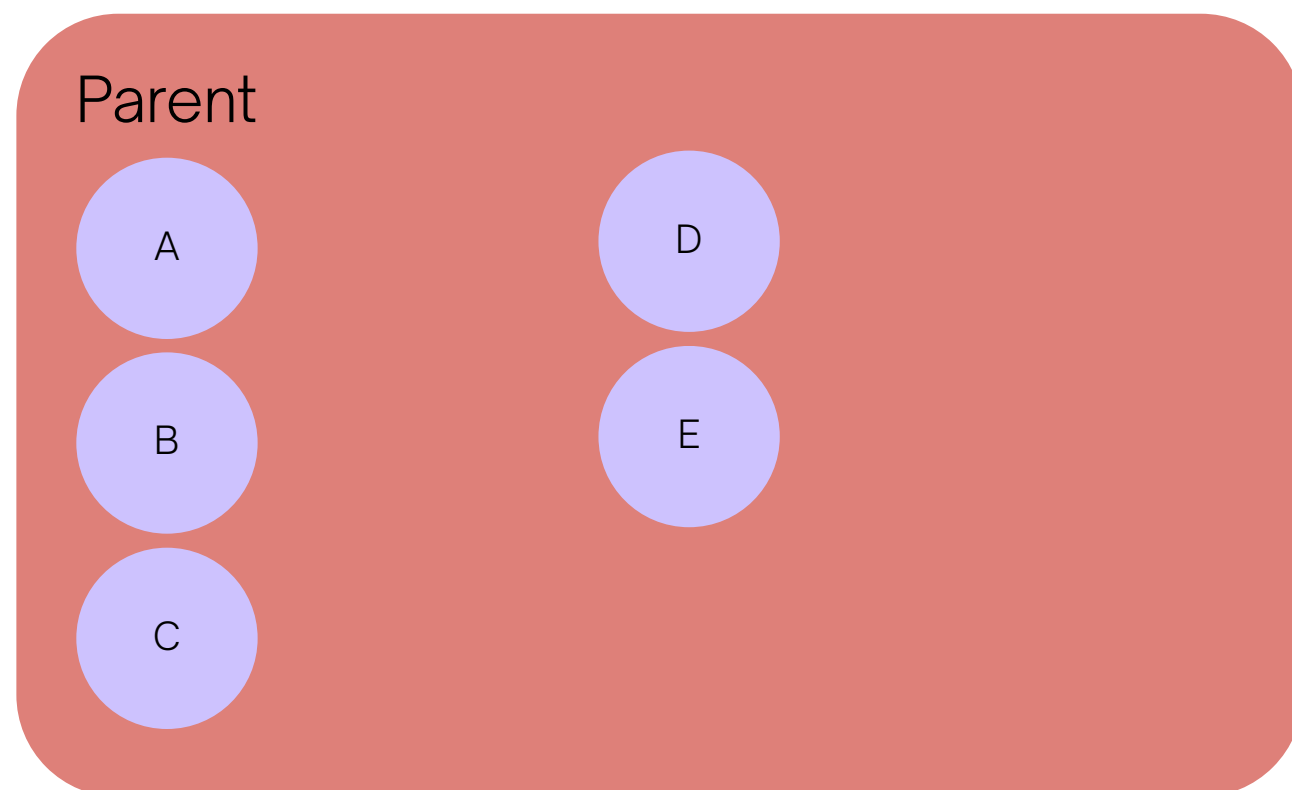
Stack



Visited



Parent

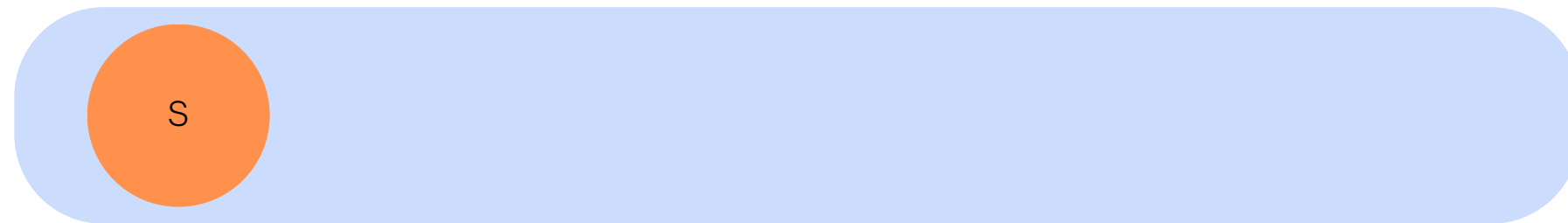


LOOP DETECTION

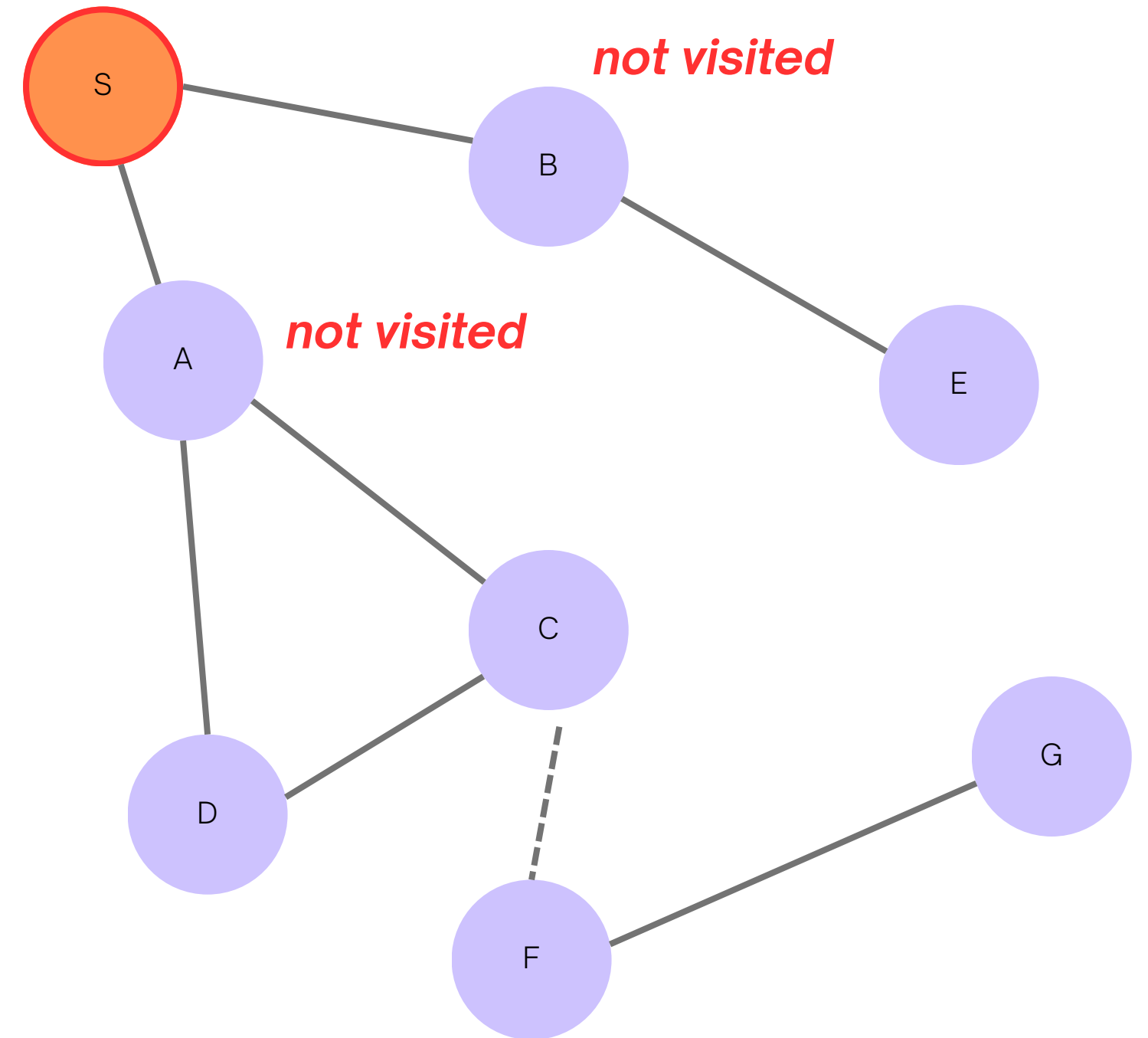
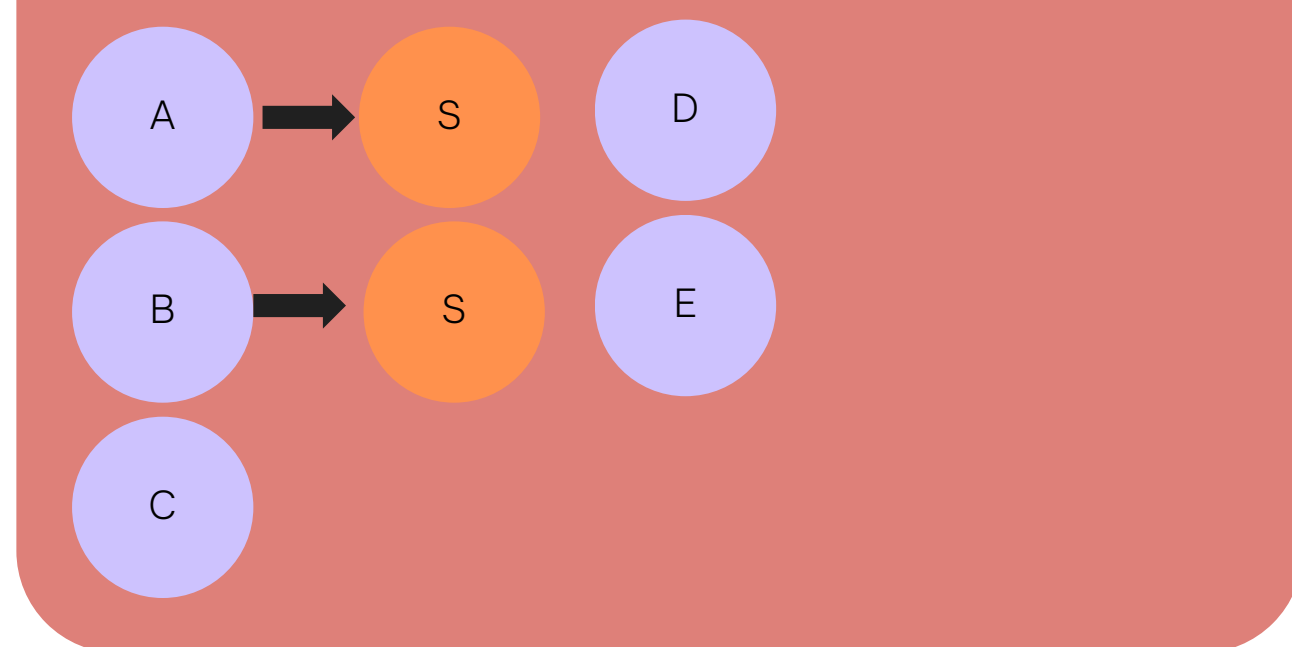
Stack



Visited



Parent

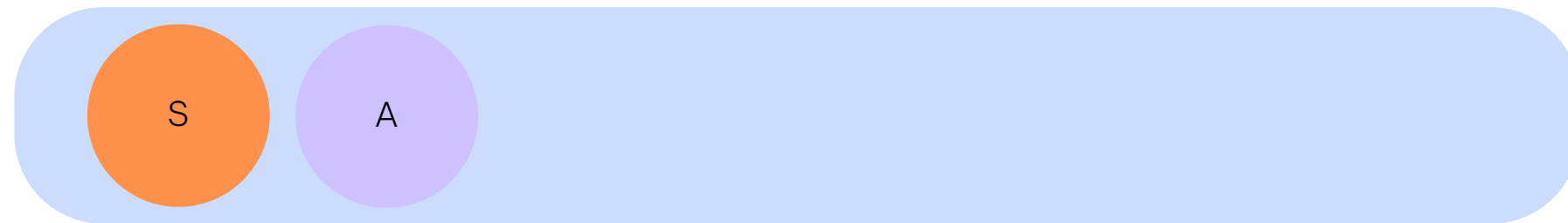


LOOP DETECTION

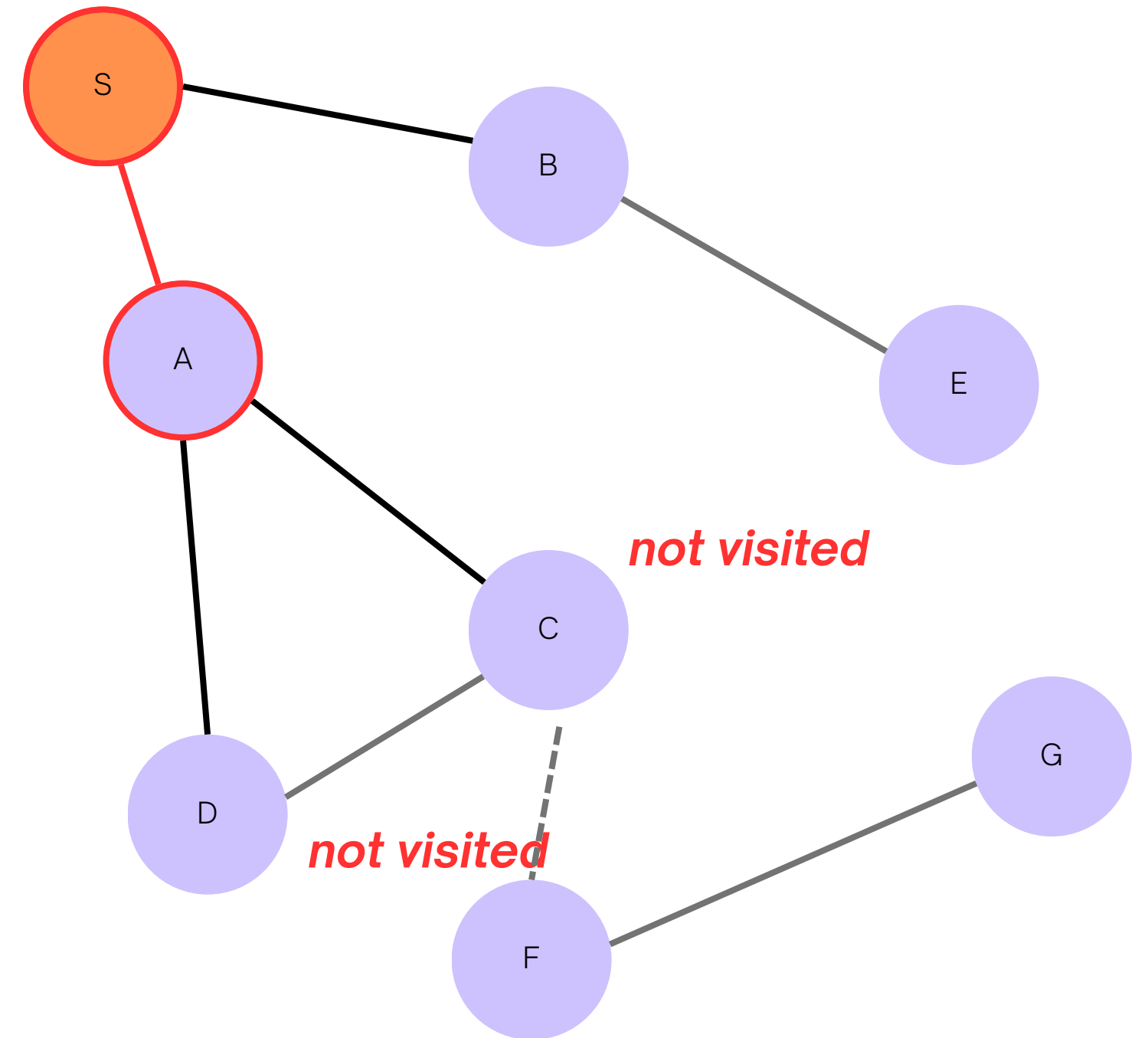
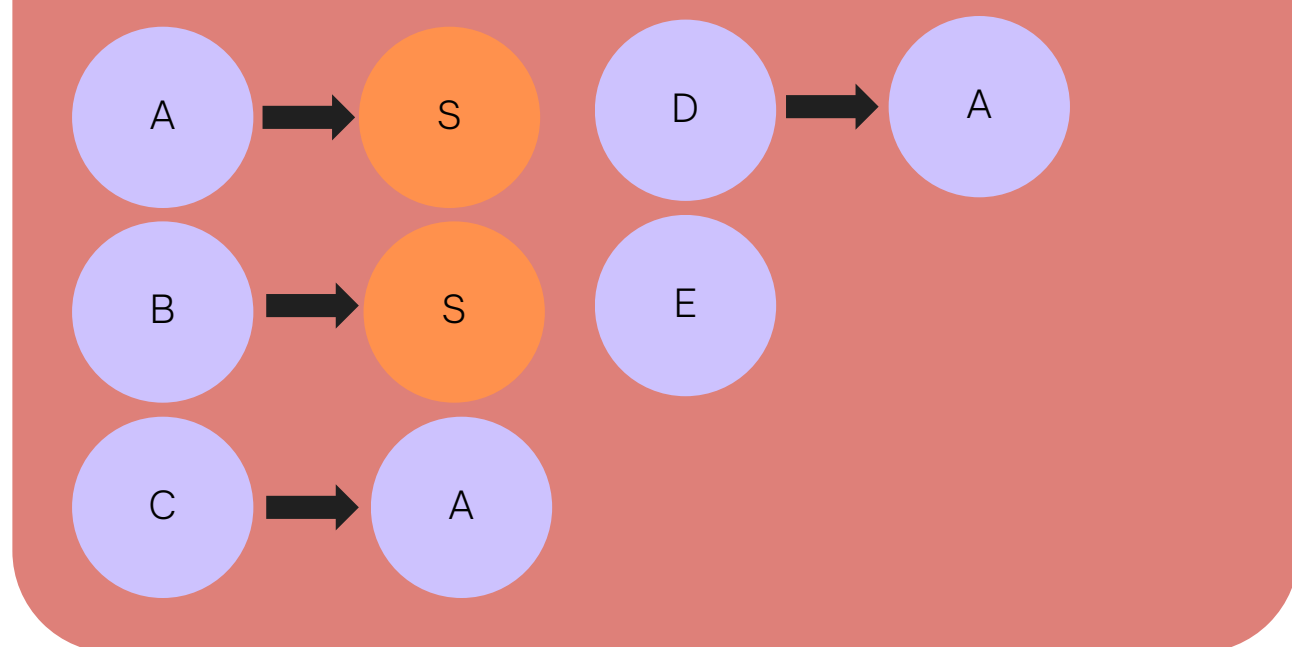
Stack



Visited

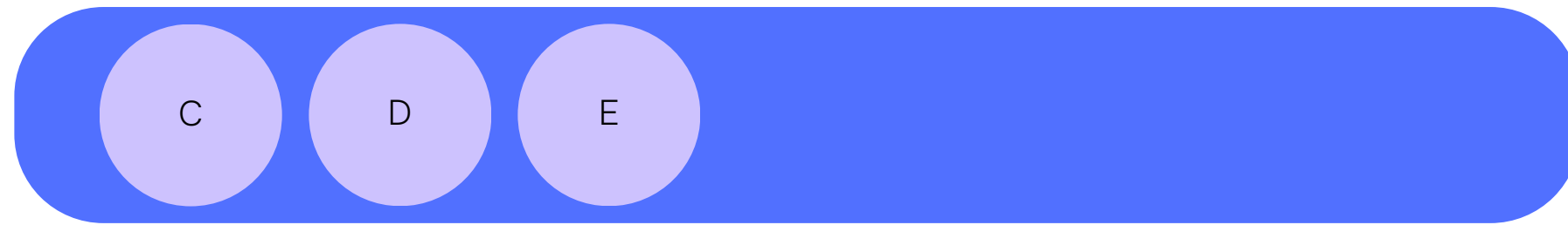


Parent



LOOP DETECTION

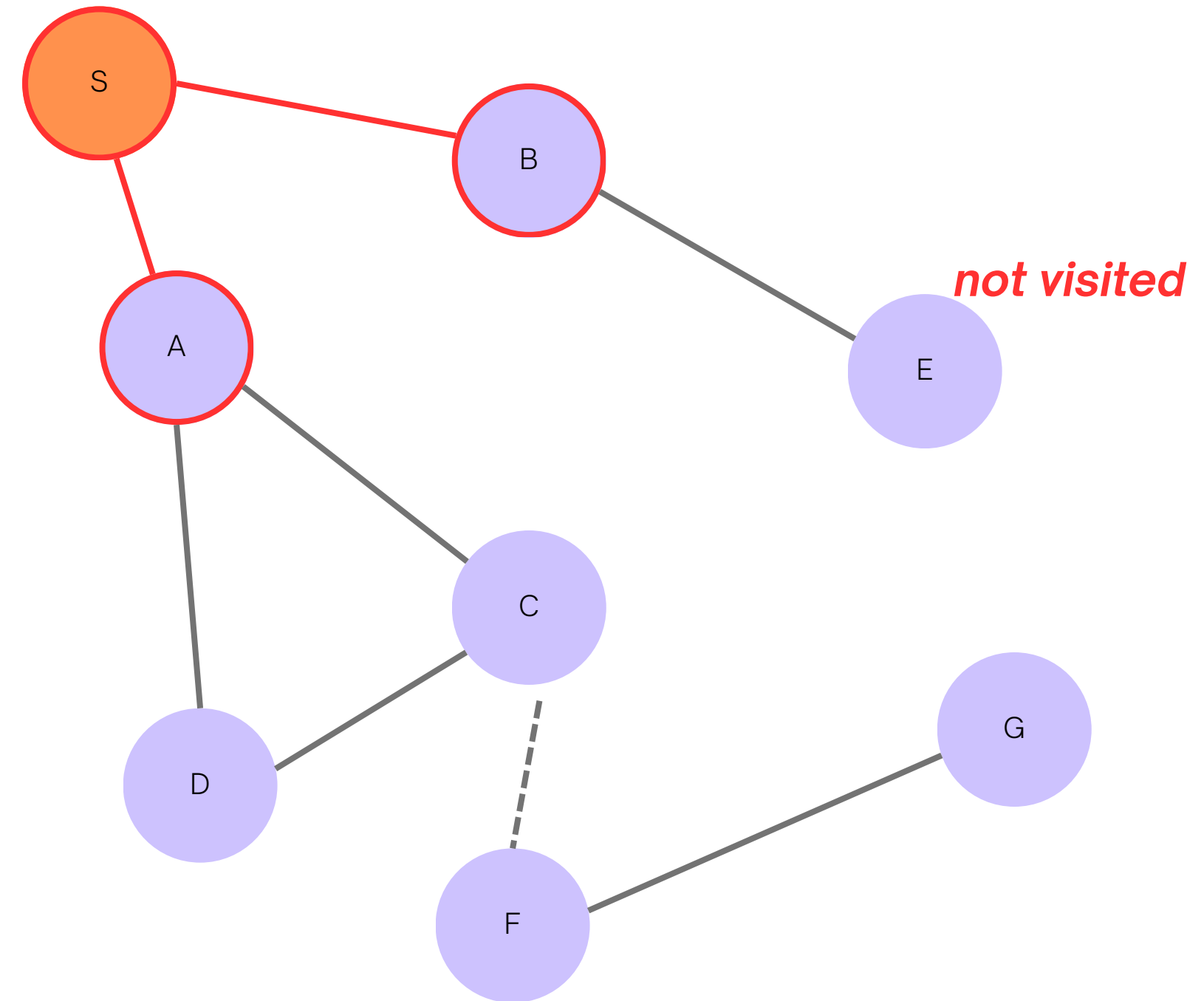
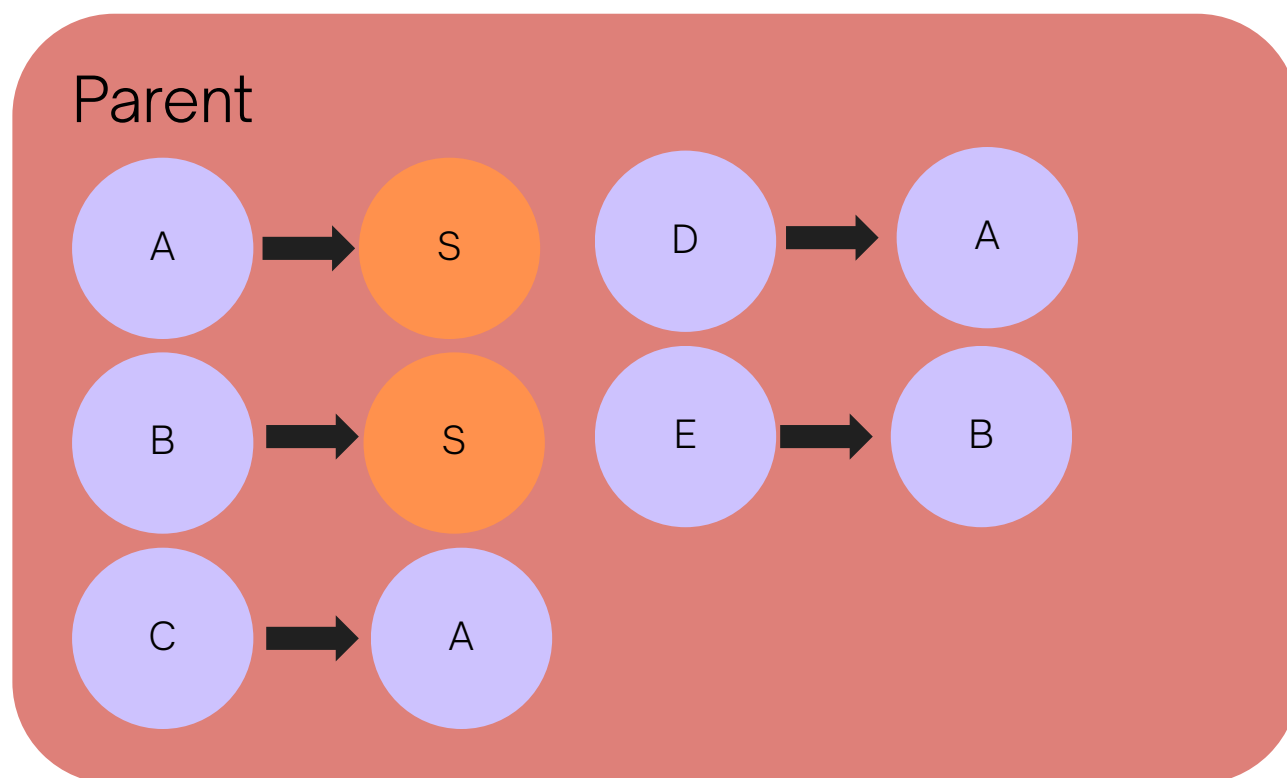
Stack



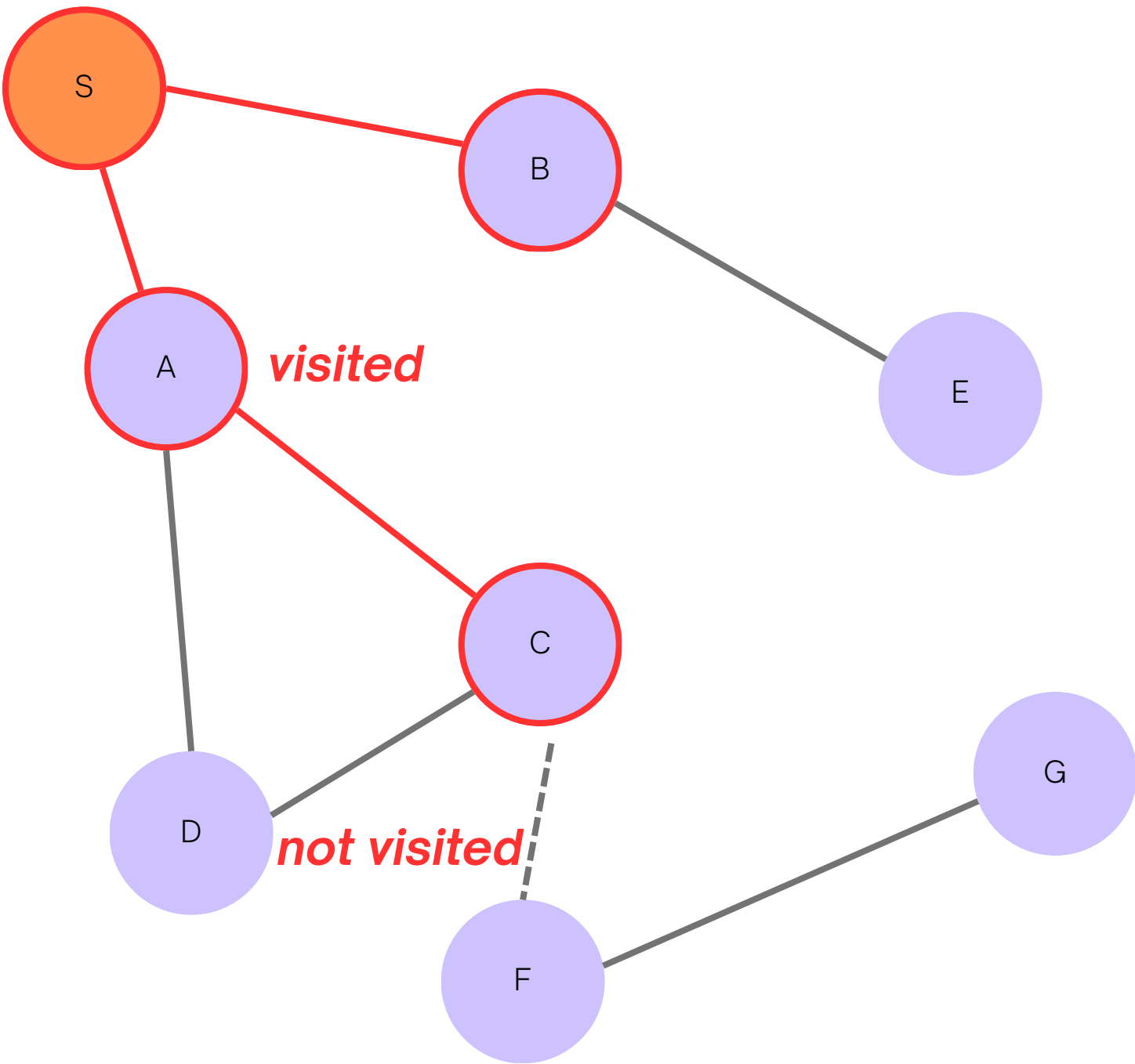
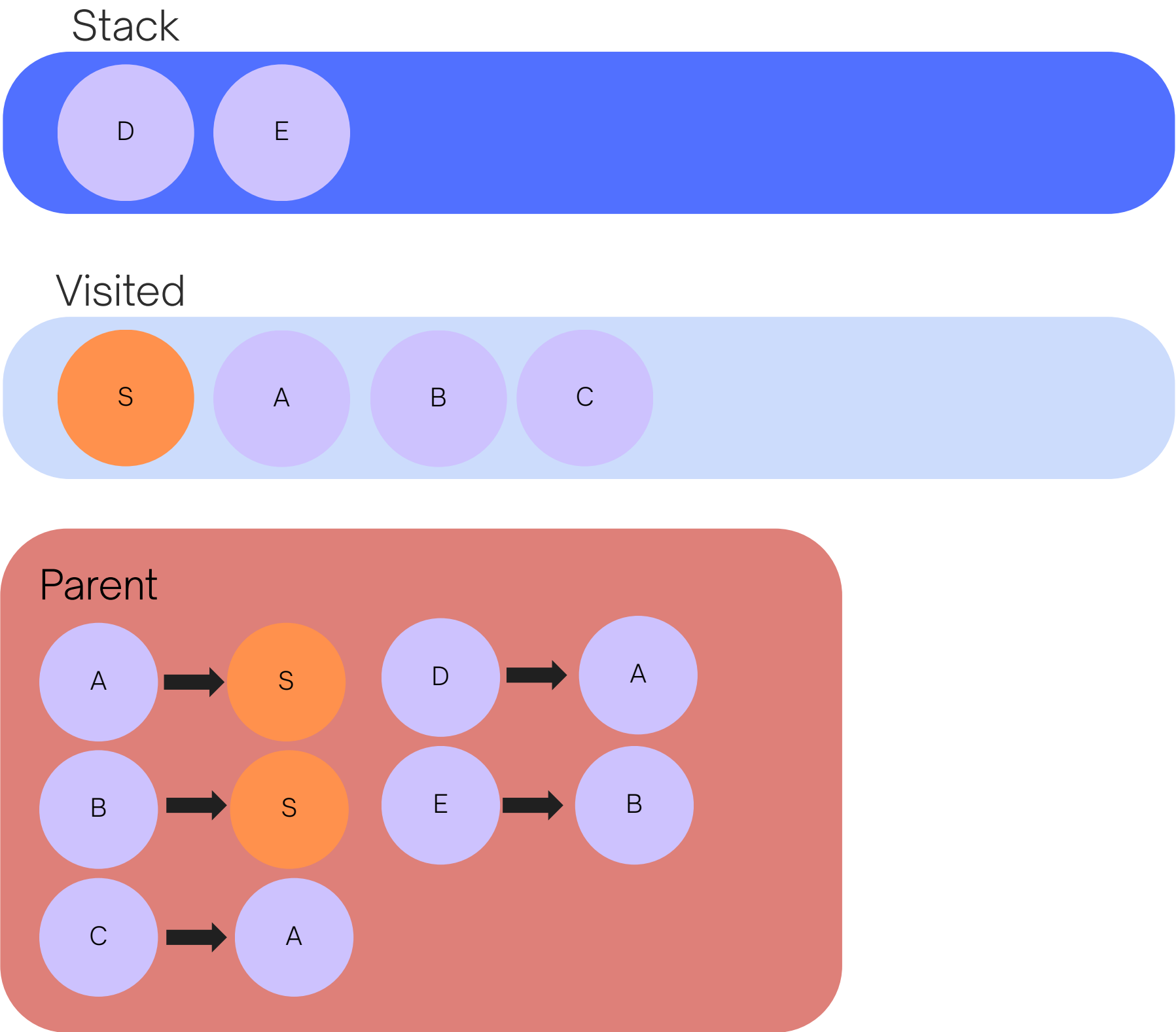
Visited



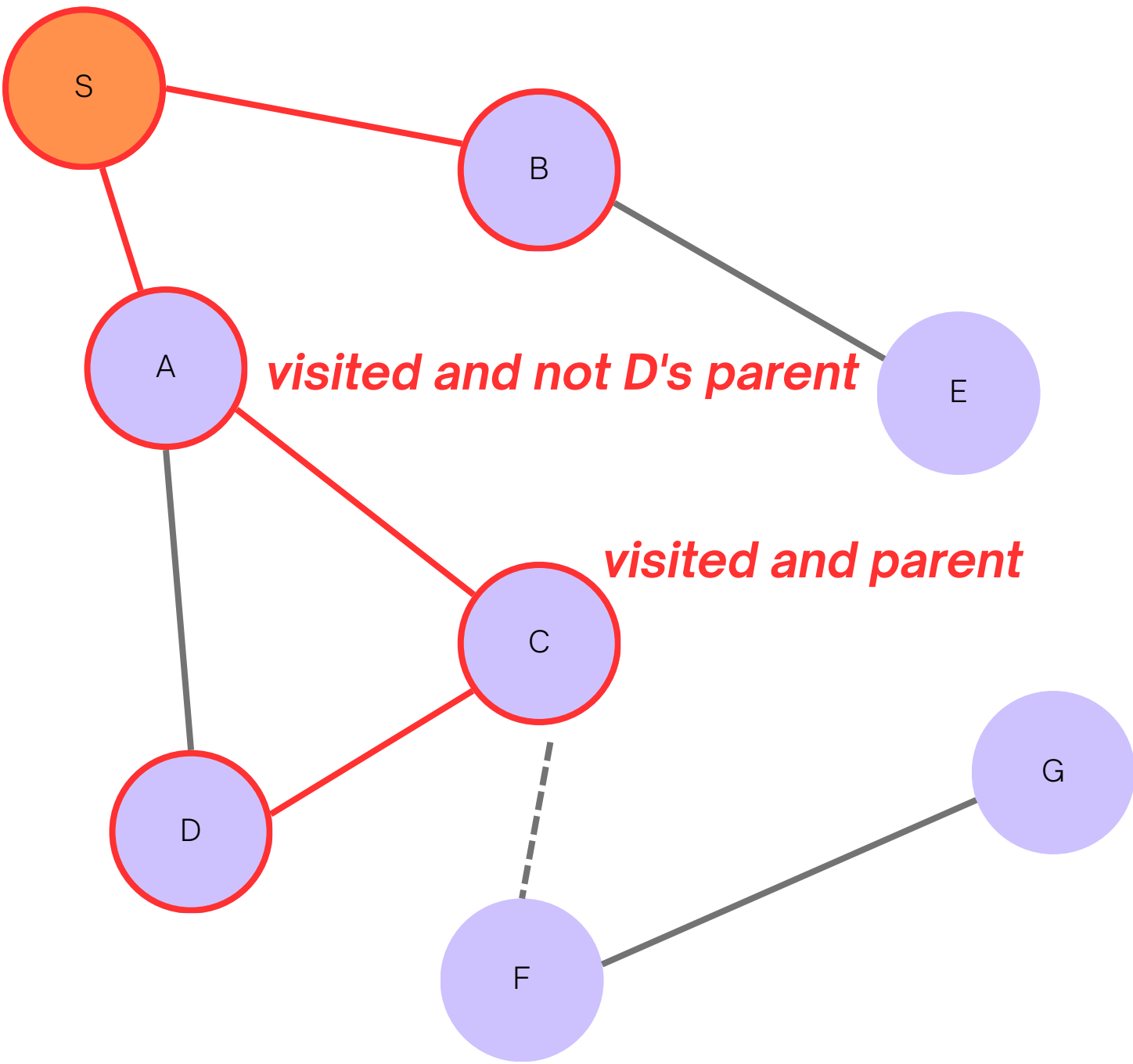
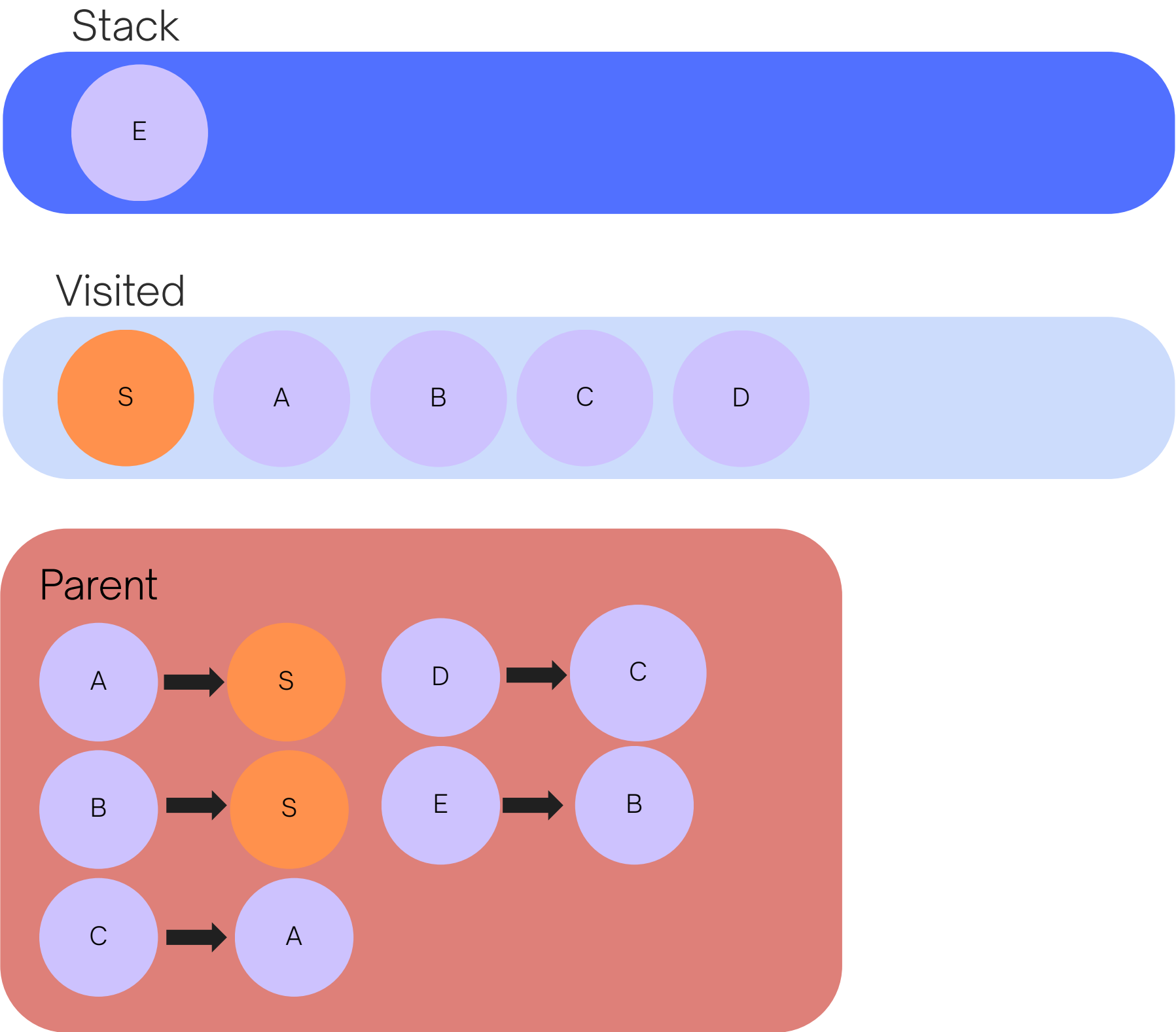
Parent



LOOP DETECTION



LOOP DETECTION

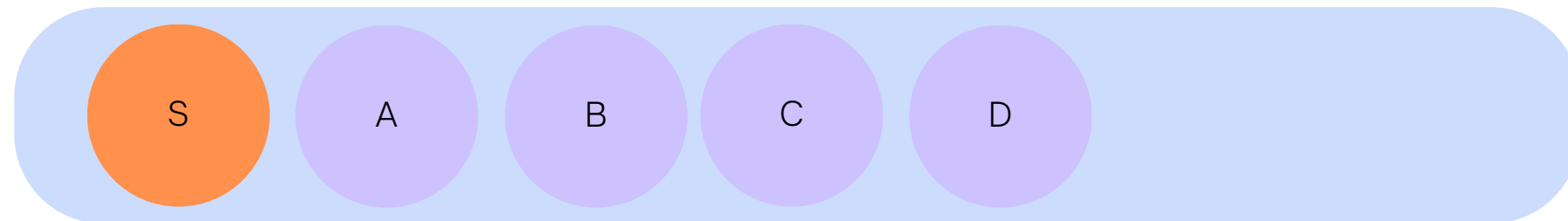


LOOP DETECTION

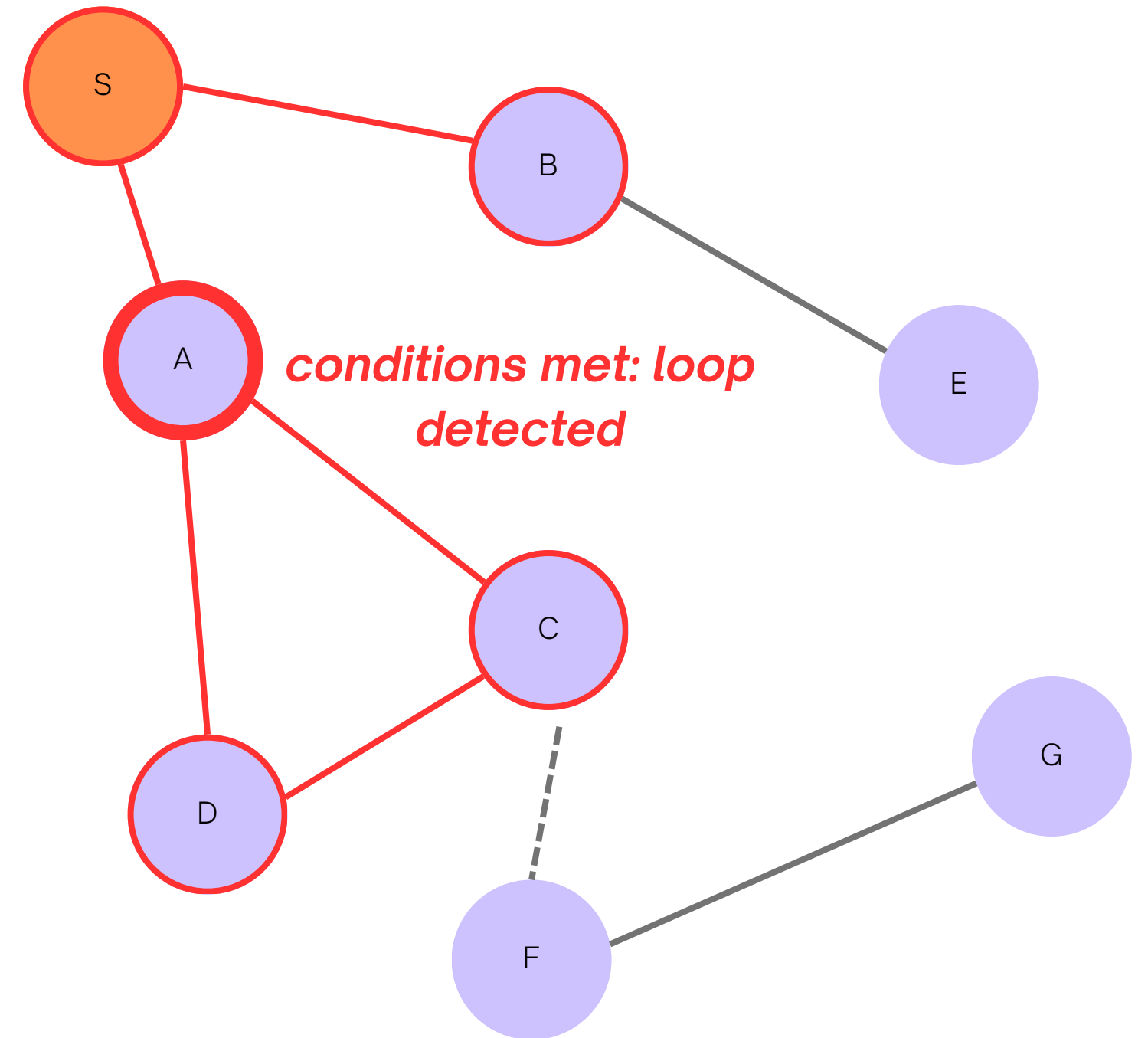
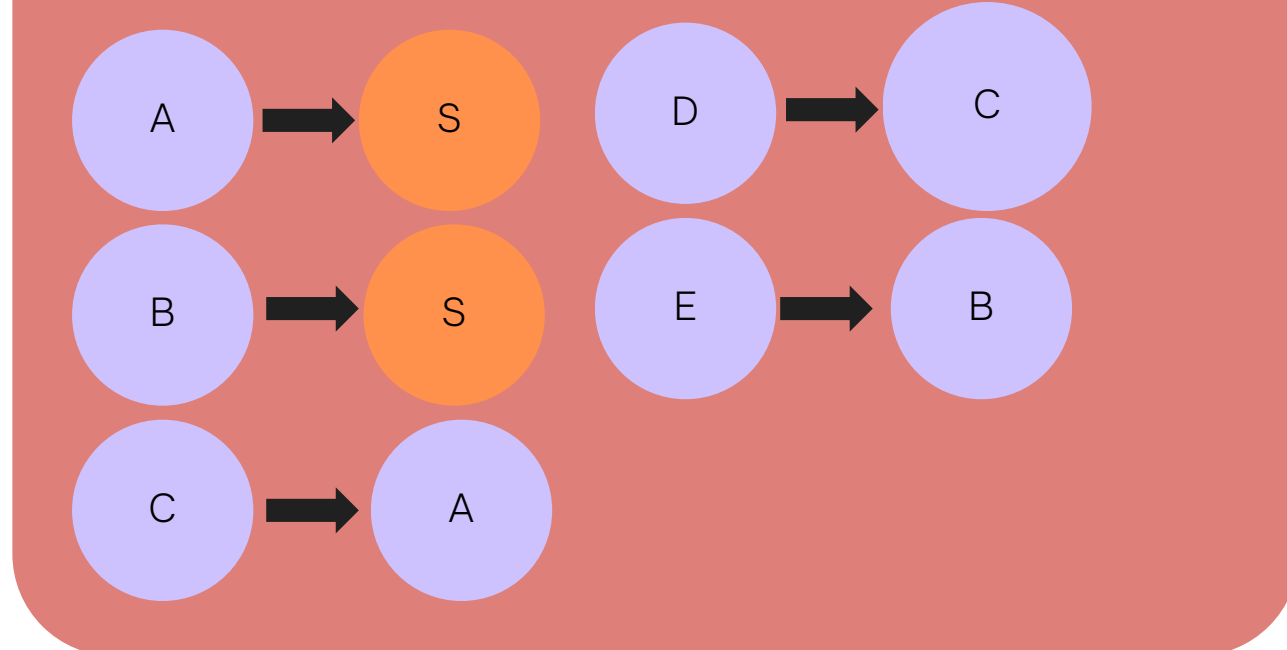
Stack



Visited

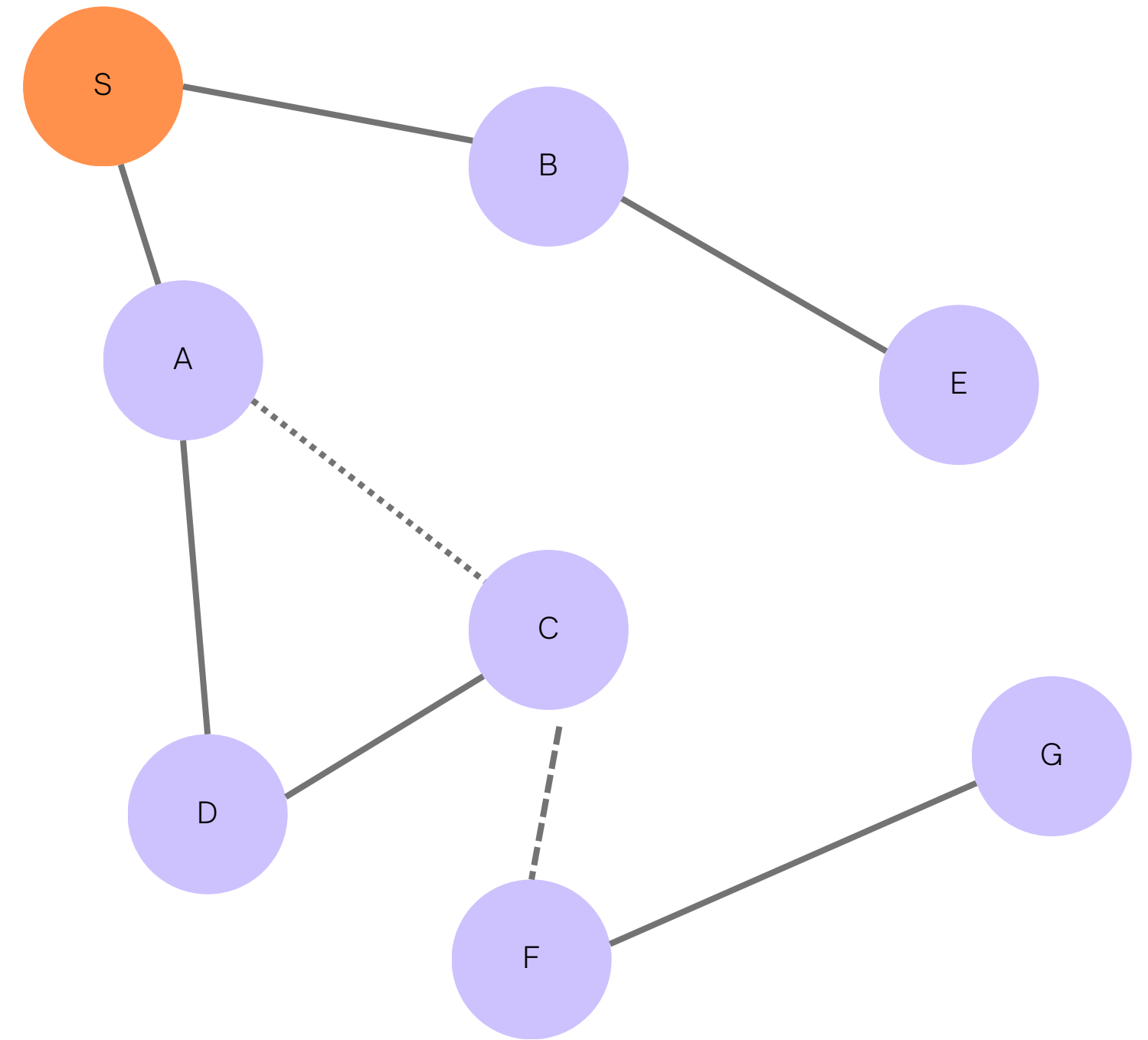


Parent



SHORTEST PATH

Dijkstra algorithm is used in this scenario to find the shortest path from one node to another. As each edge have different value and Dijkstra algorithm guarantee the shortest path to the destination



SHORTEST PATH

Example : From S to C

Dist

S

:

0

A

:

∞

B

:

∞

C

:

∞

D

:

∞

E

:

∞

Heap

Parent

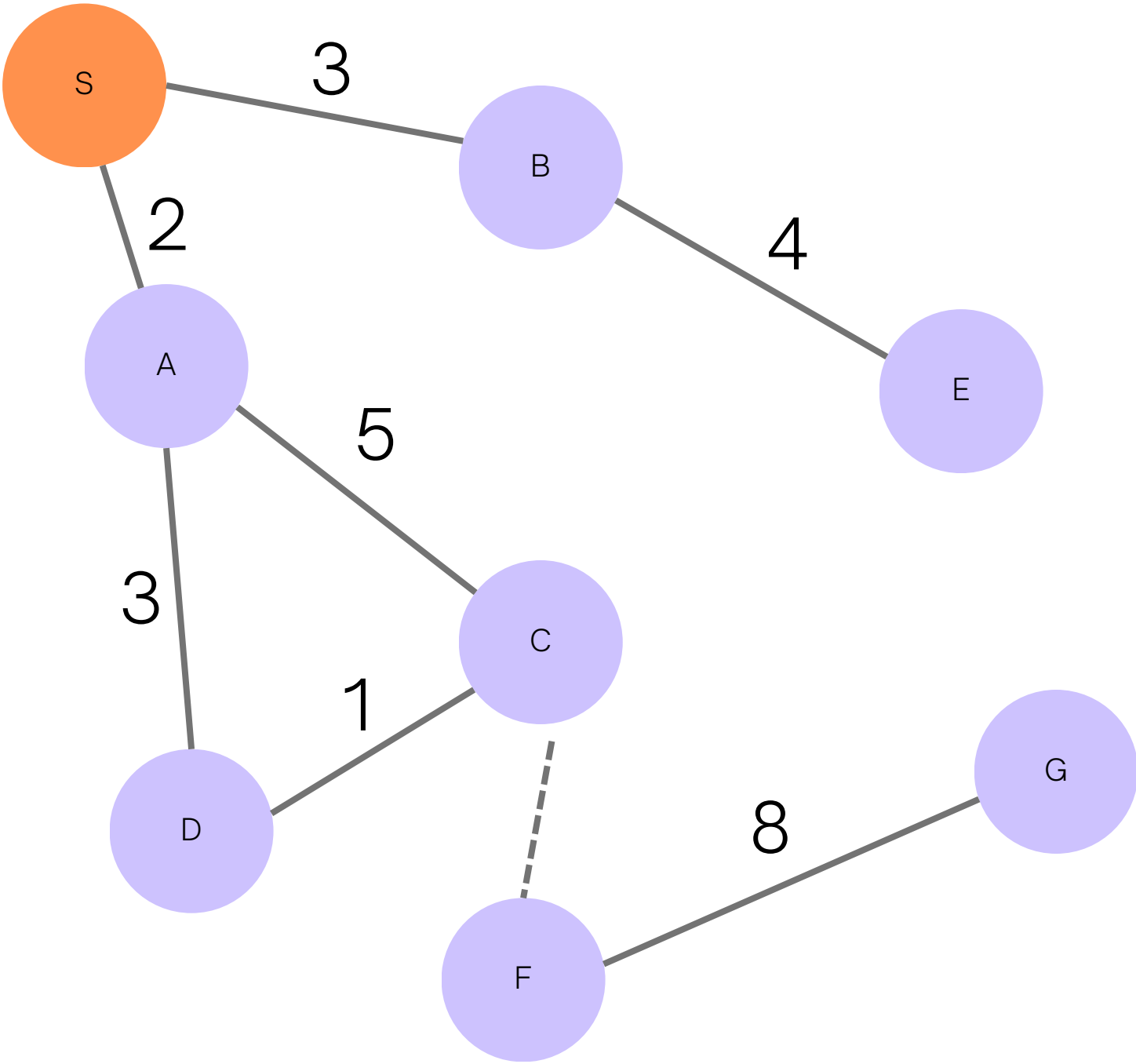
A

D

B

E

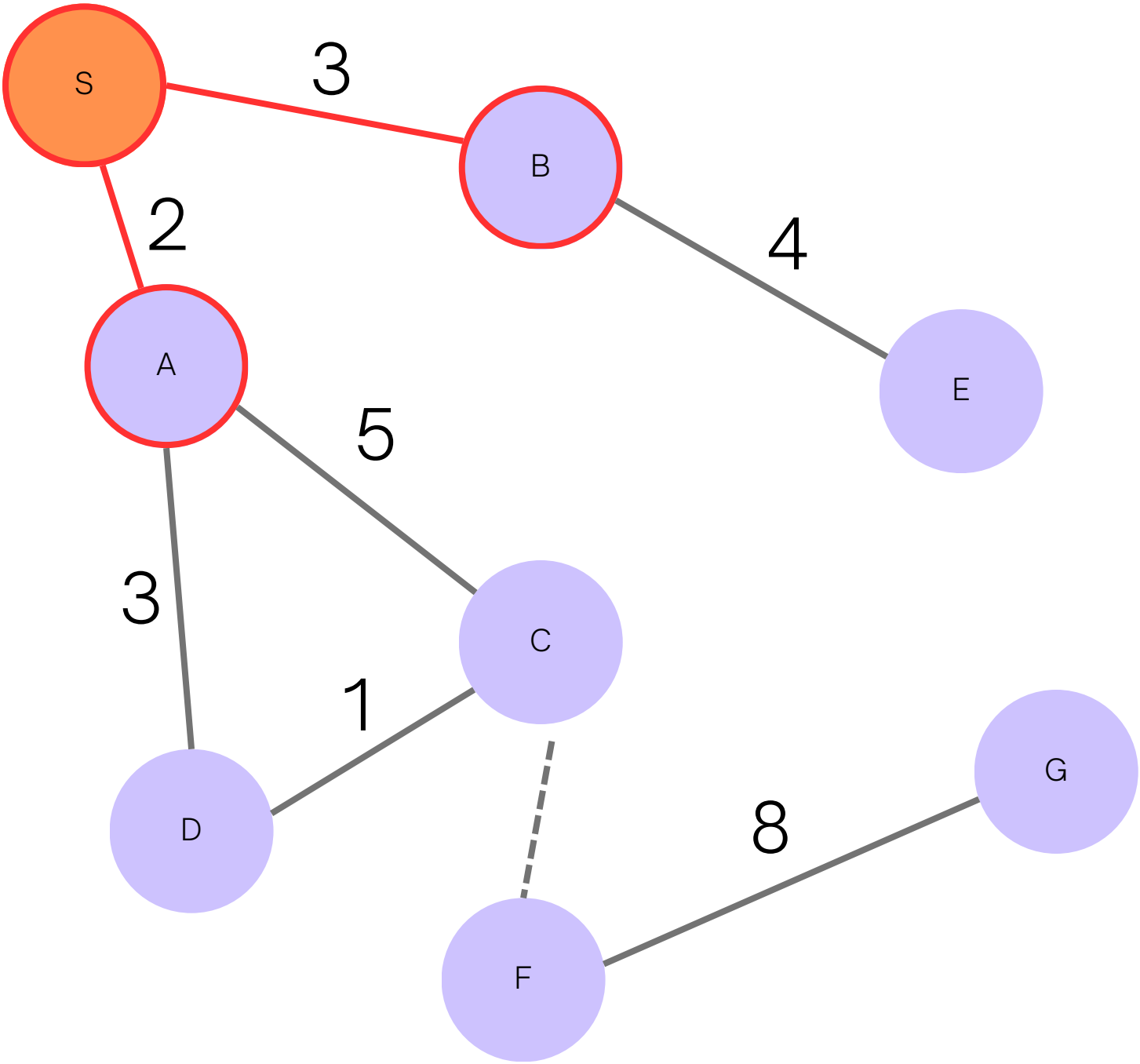
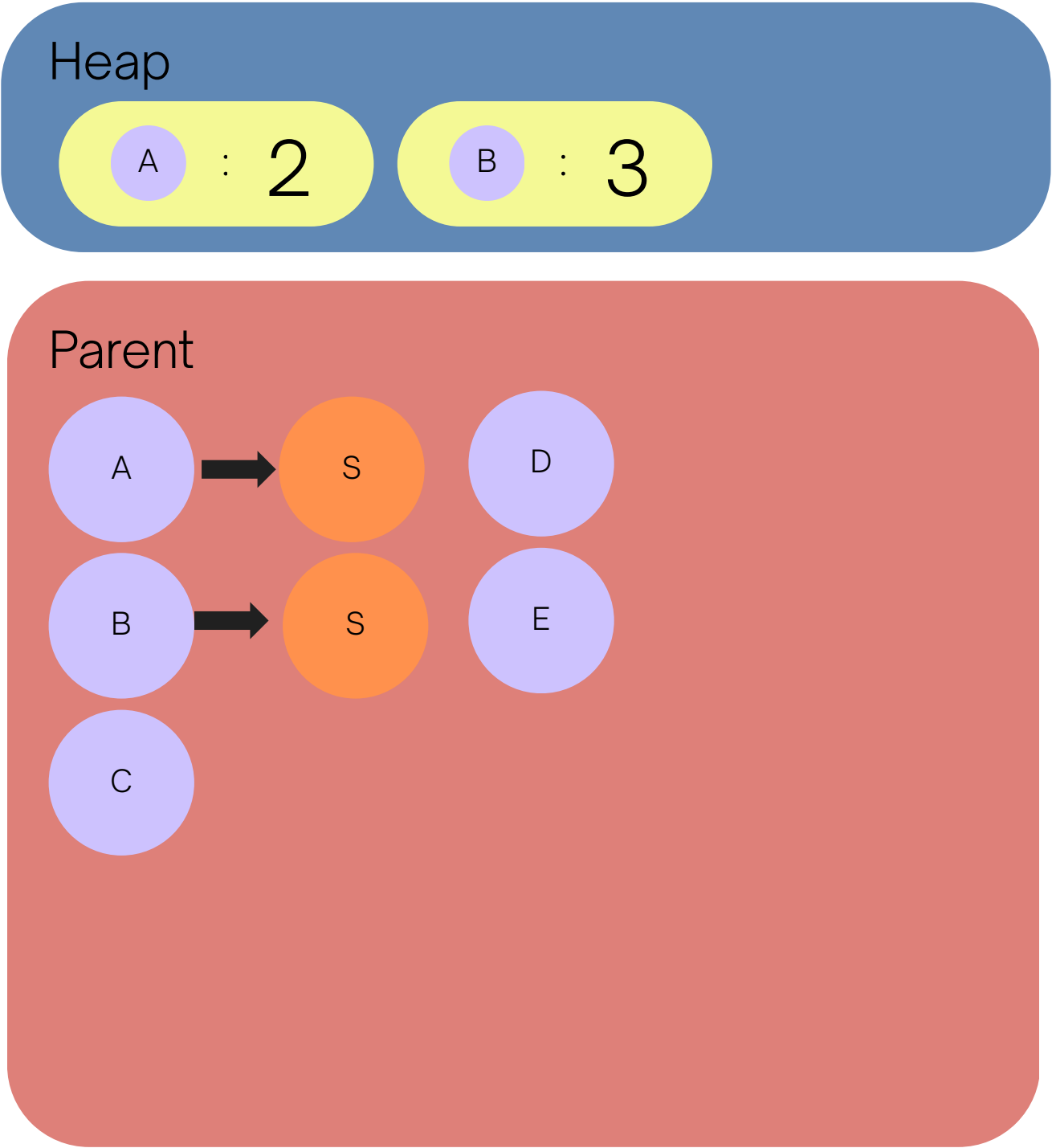
C



SHORTEST PATH

Example : From S to C

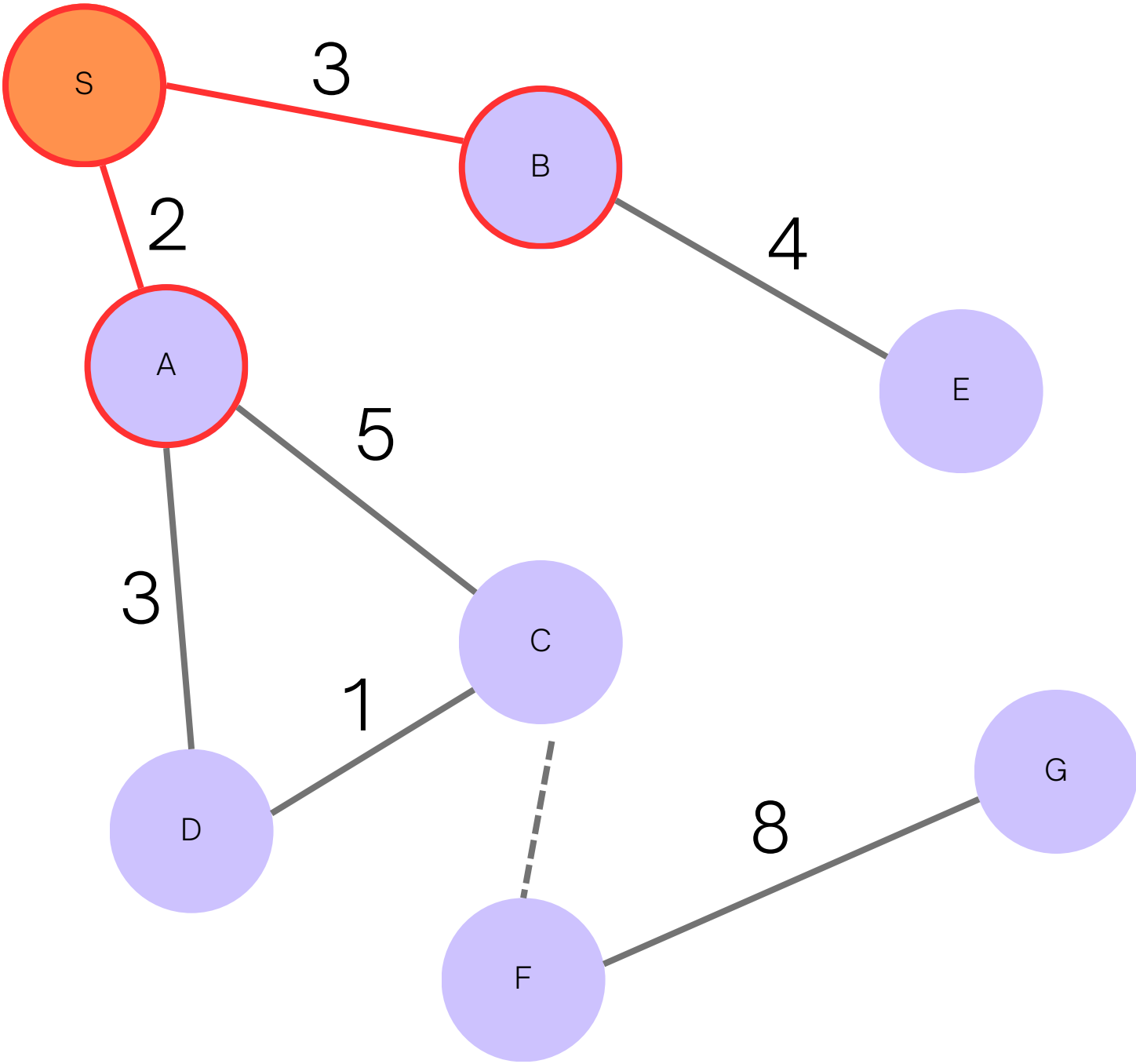
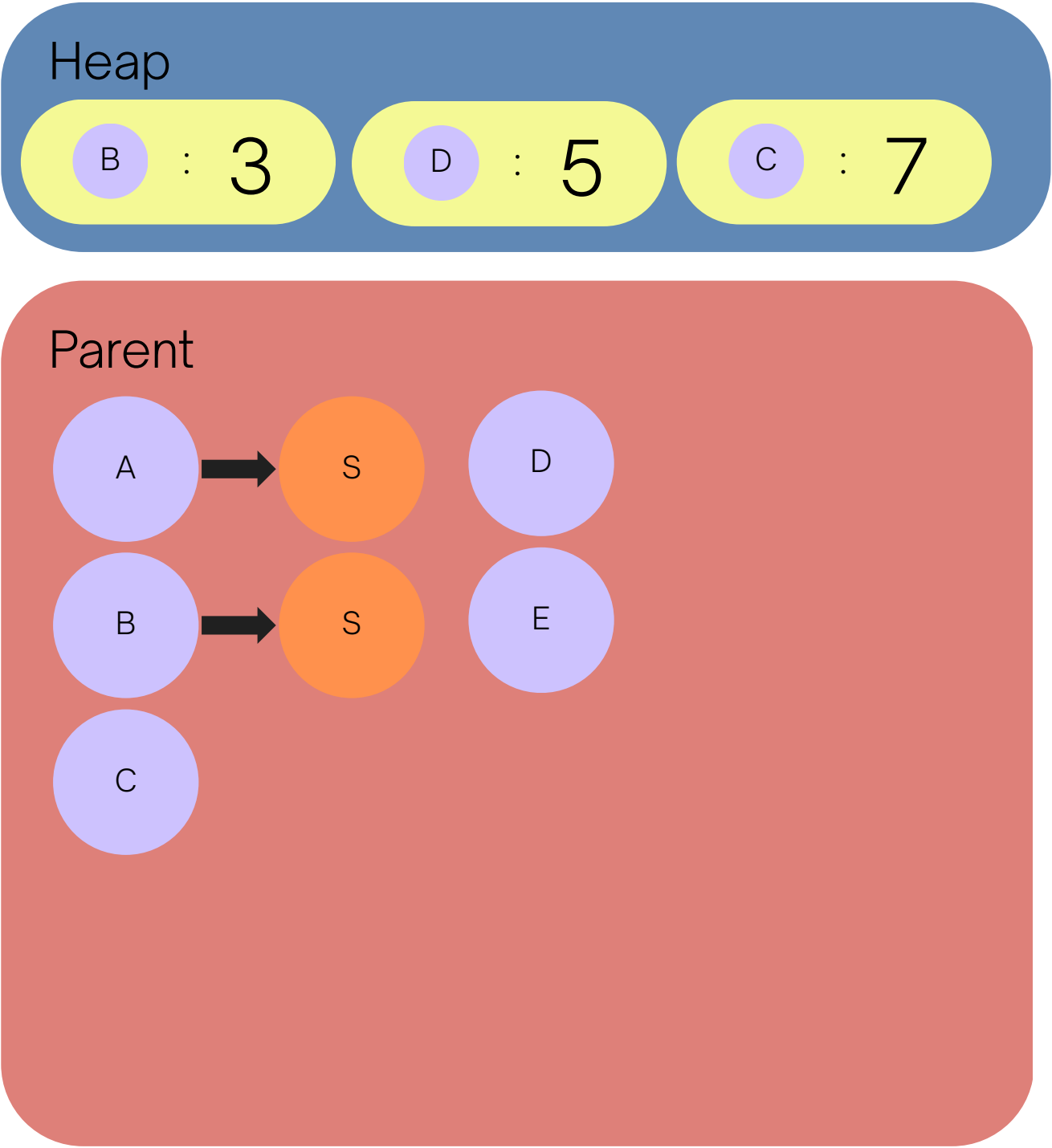
Dist		
S	:	0
A	:	2
B	:	3
C	:	∞
D	:	∞
E	:	∞



SHORTEST PATH

Example : From S to C

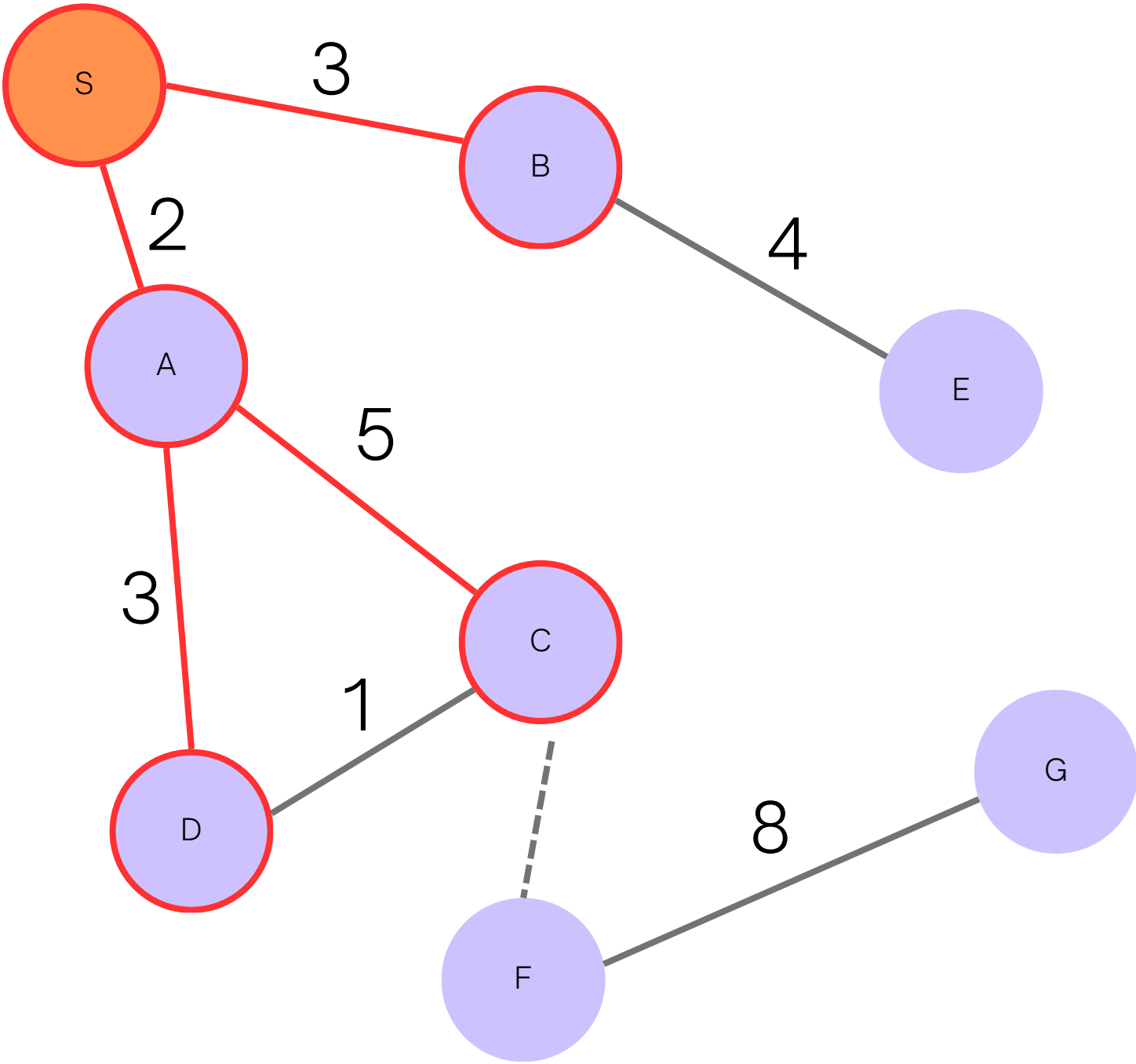
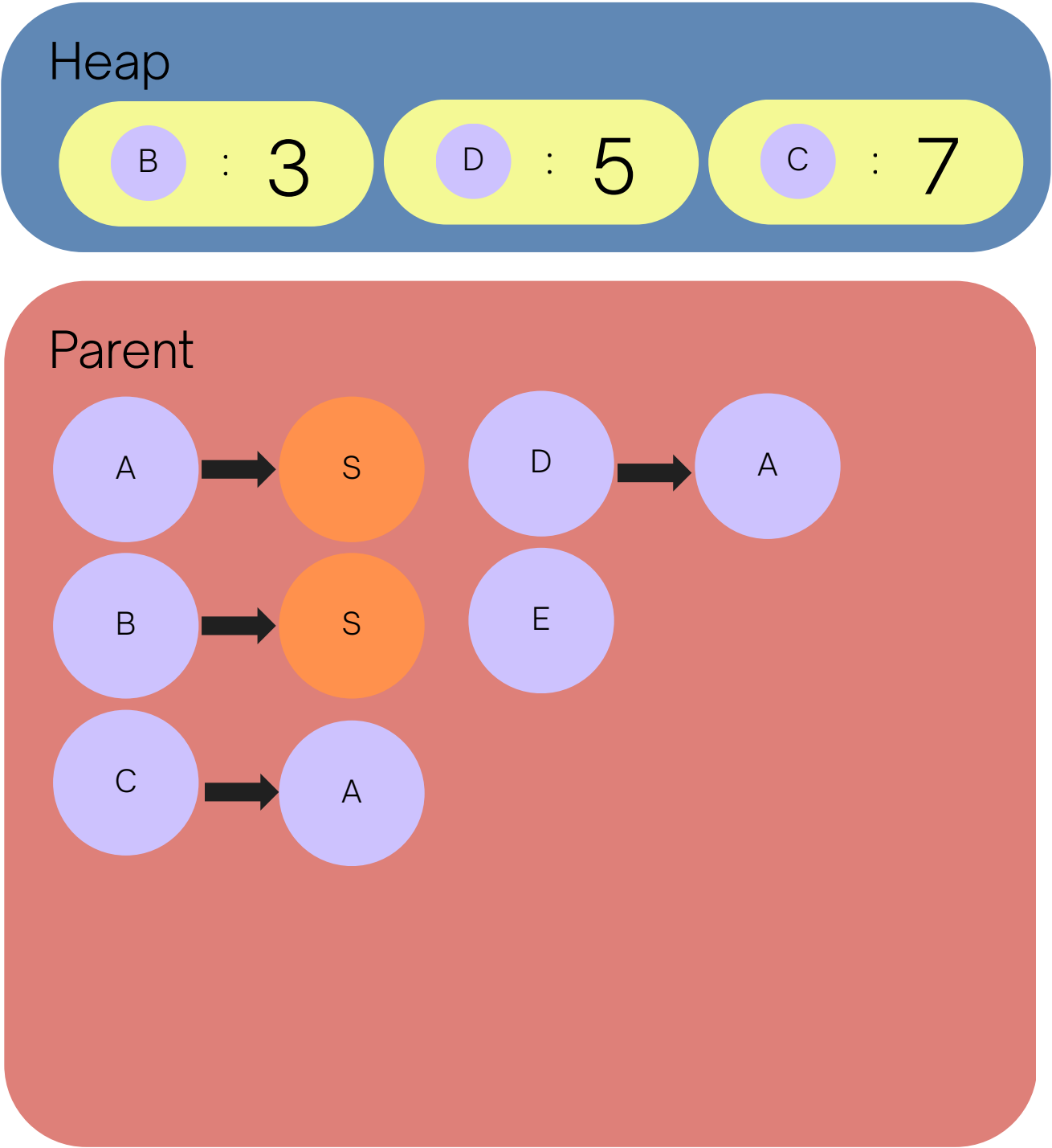
Dist		
S	:	0
A	:	2
B	:	3
C	:	7
D	:	5
E	:	∞



SHORTEST PATH

Example : From S to C

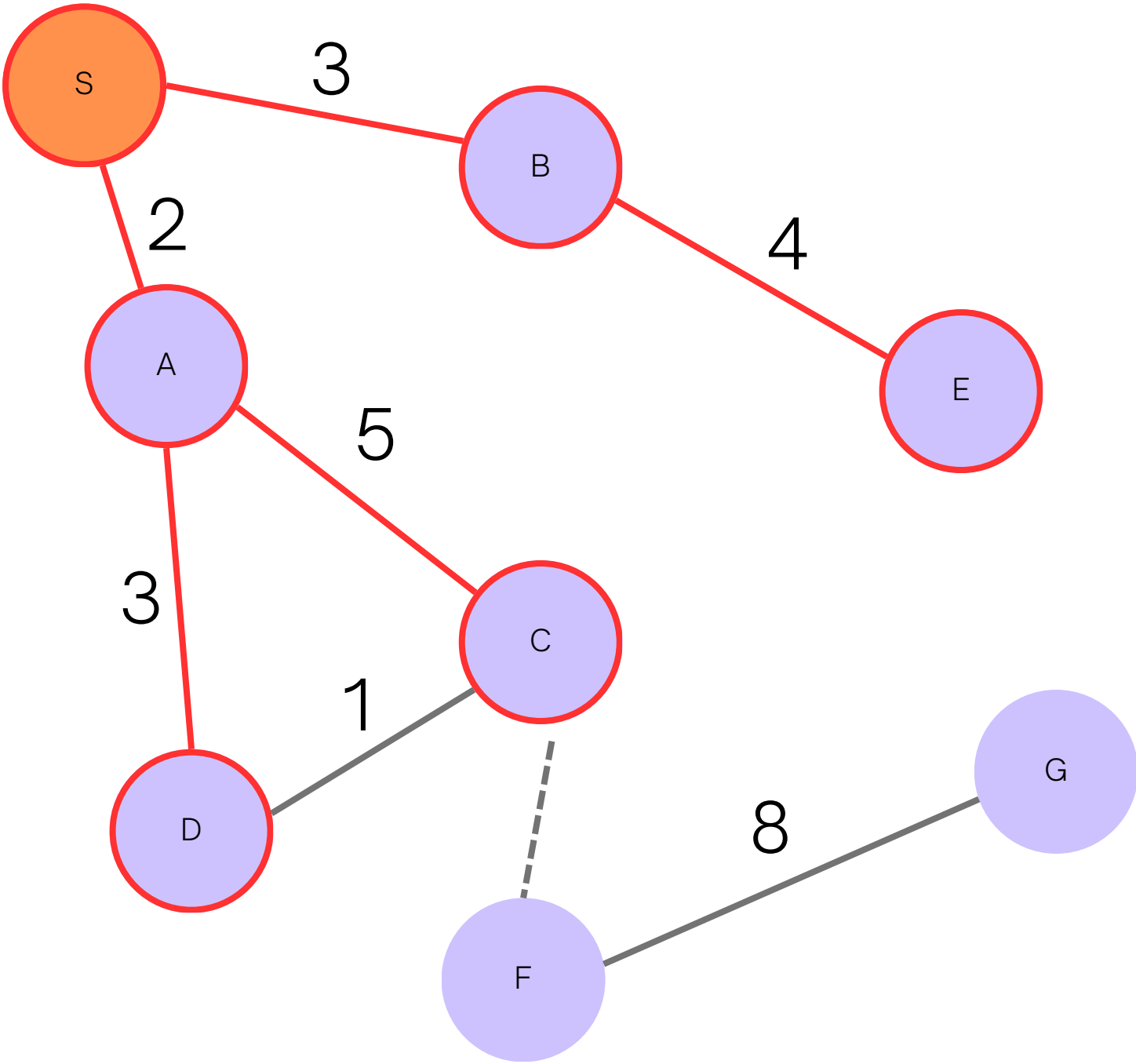
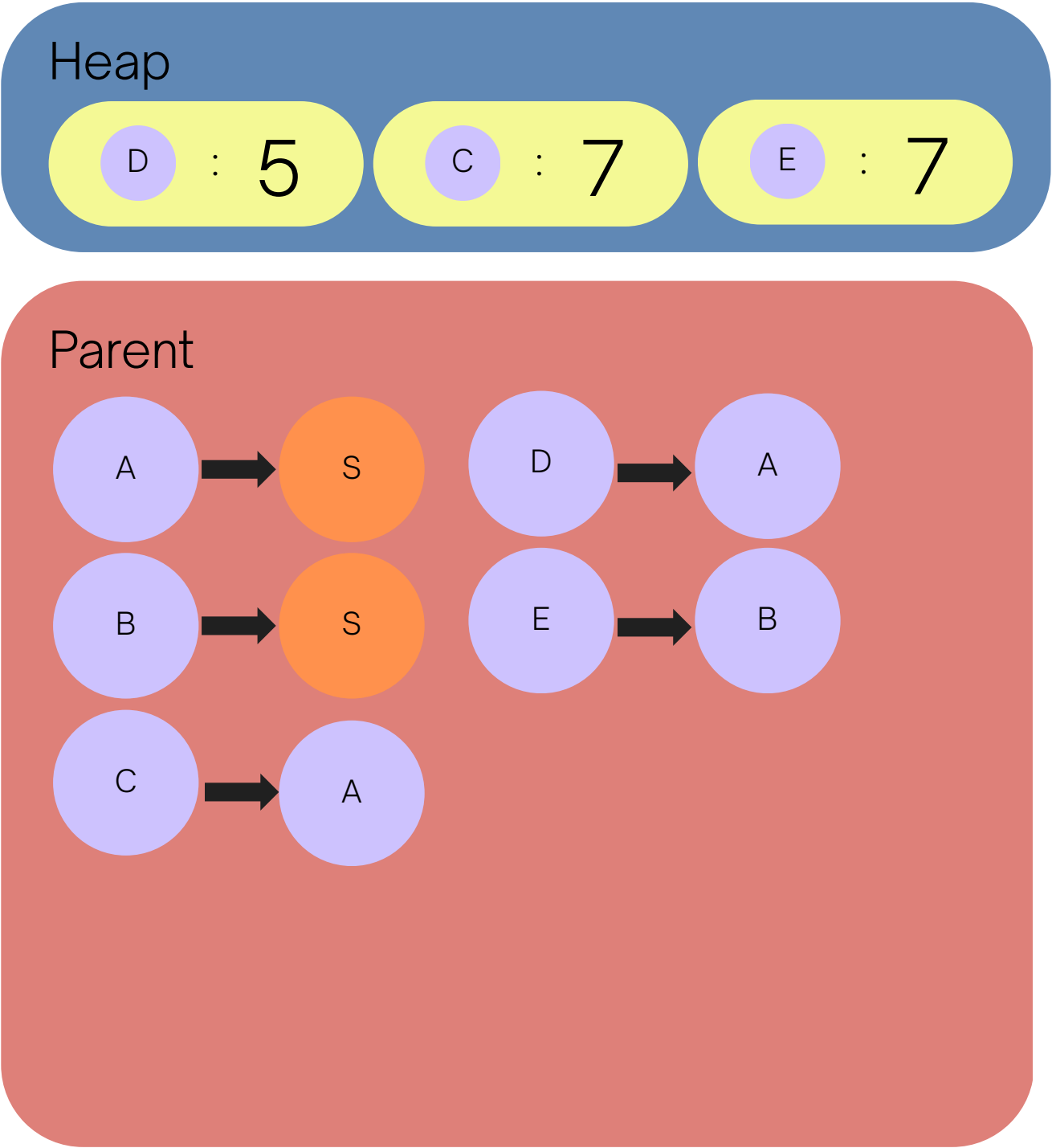
Dist		
S	:	0
A	:	2
B	:	3
C	:	7
D	:	5
E	:	∞



SHORTEST PATH

Example : From **S** to **C**

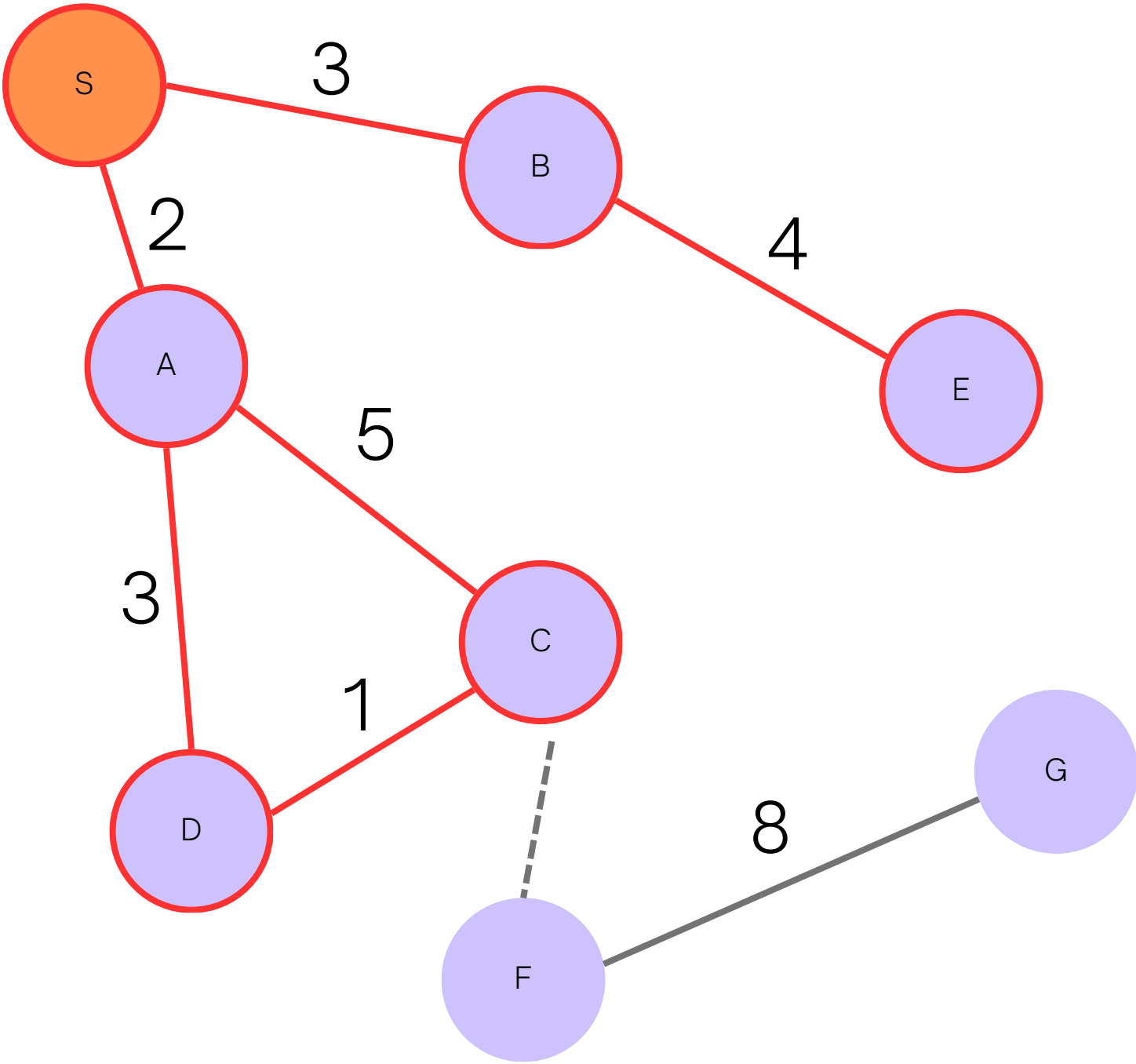
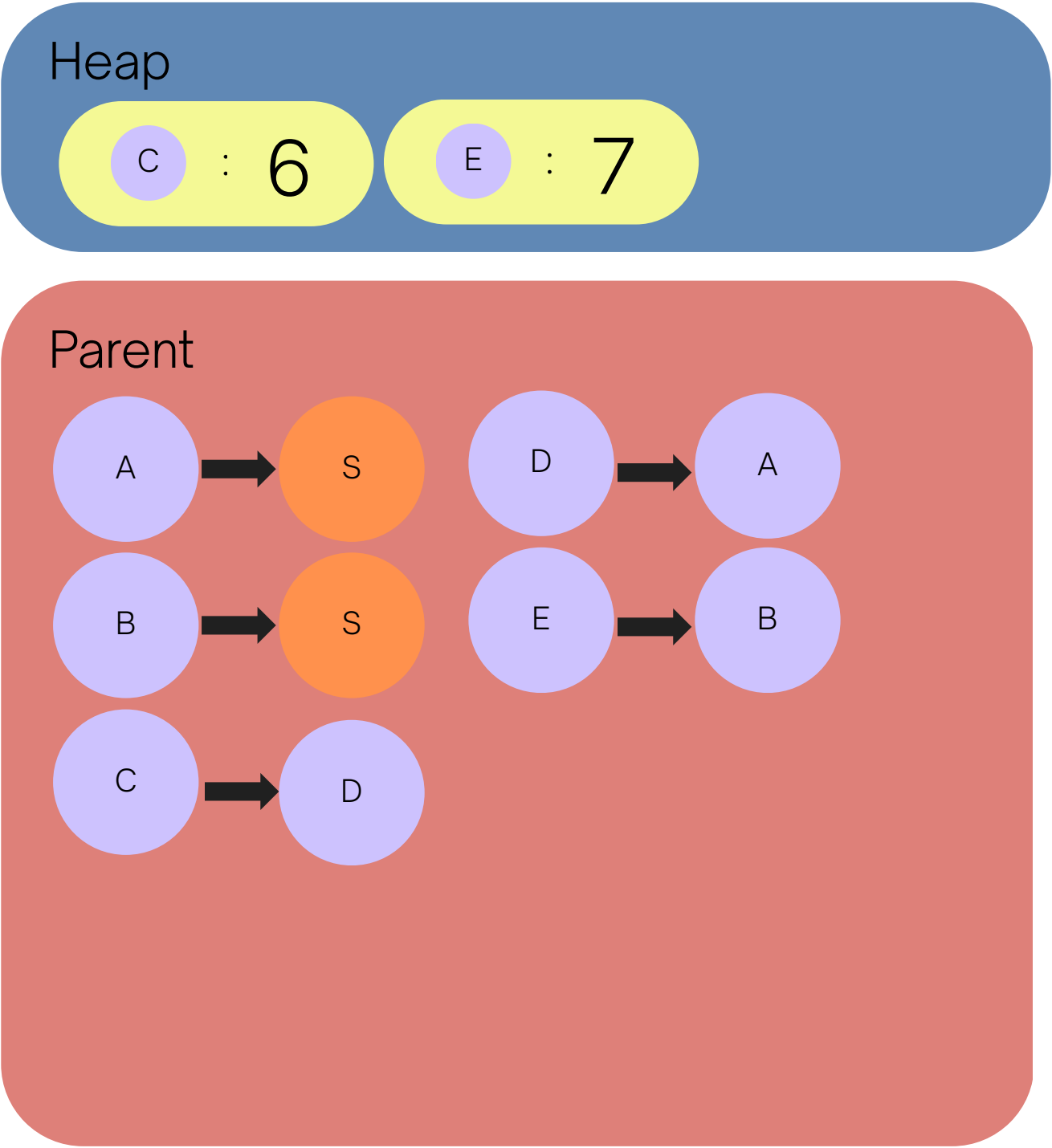
Dist		
S	:	0
A	:	2
B	:	3
C	:	7
D	:	5
E	:	7



SHORTEST PATH

Example : From S to C

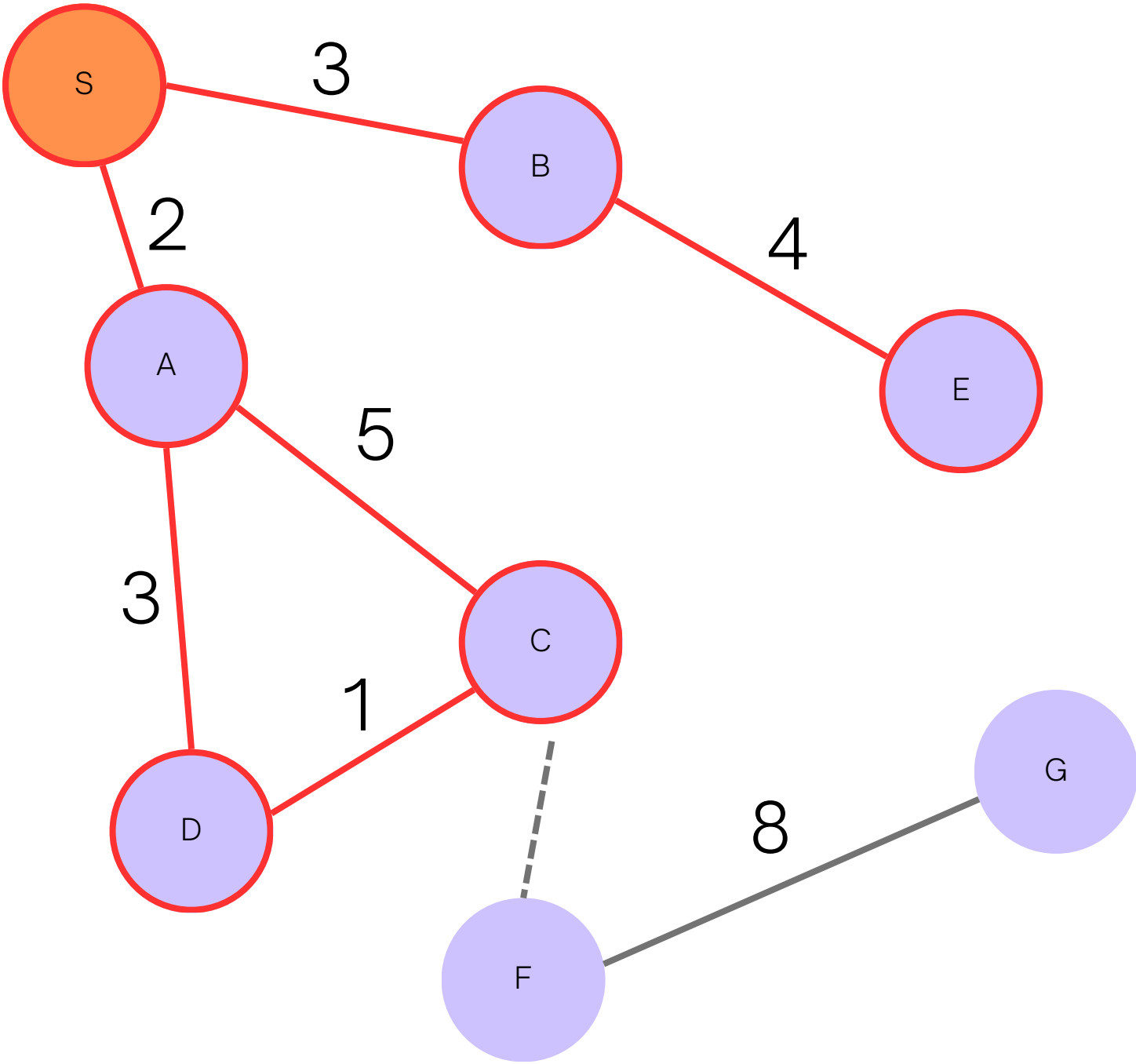
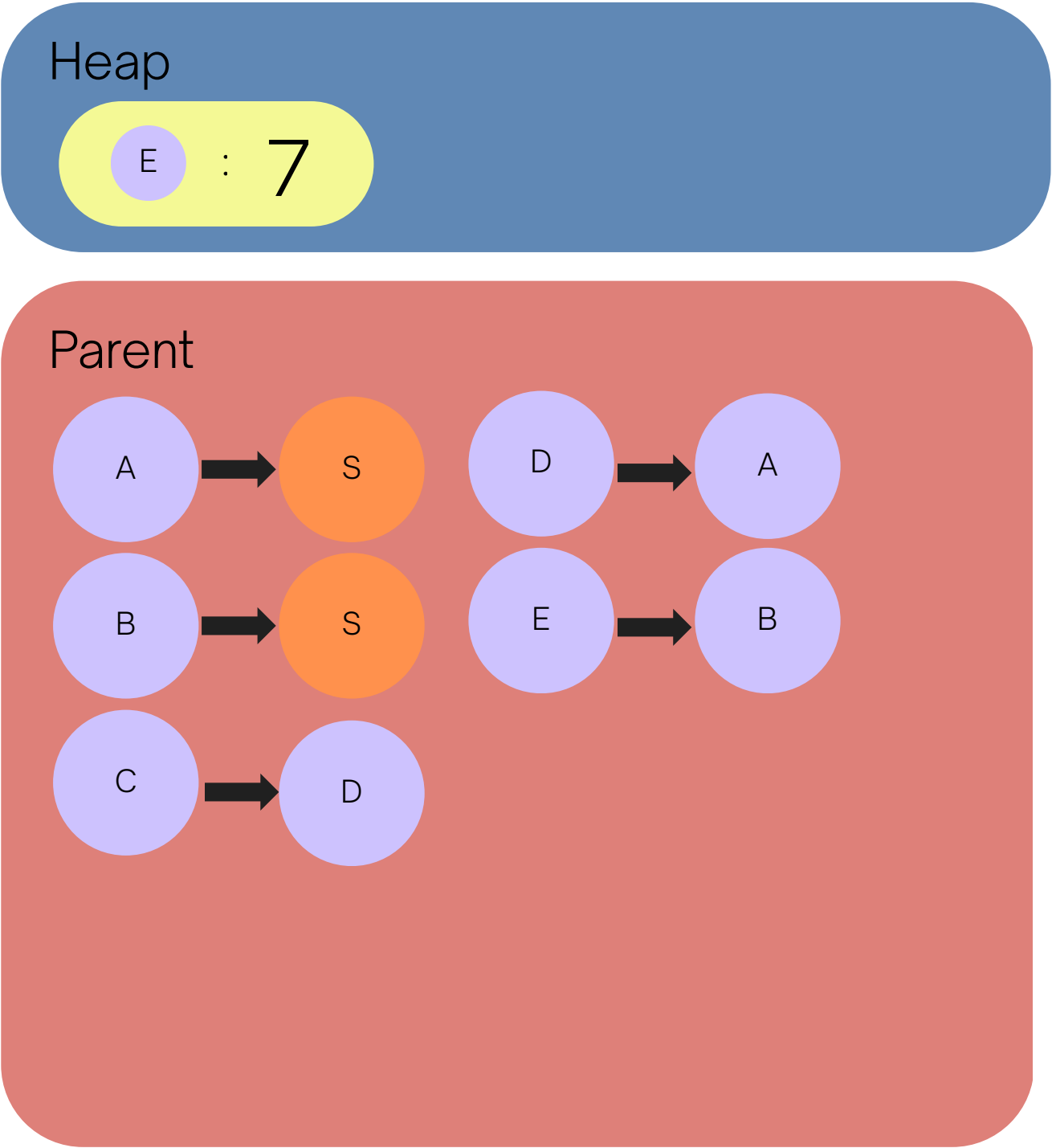
Dist		
S	:	0
A	:	2
B	:	3
C	:	6
D	:	5
E	:	7



SHORTEST PATH

Example : From S to C

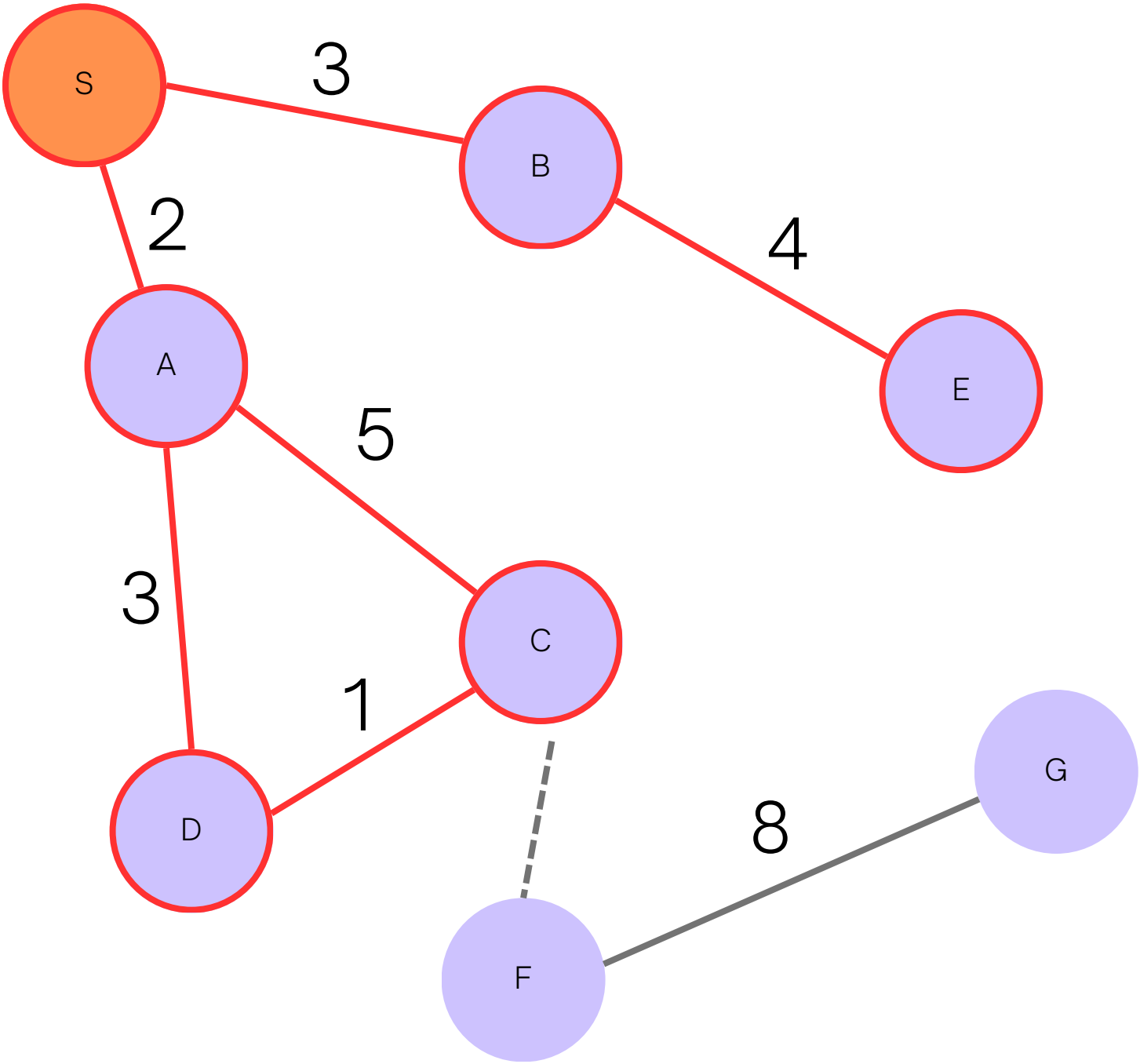
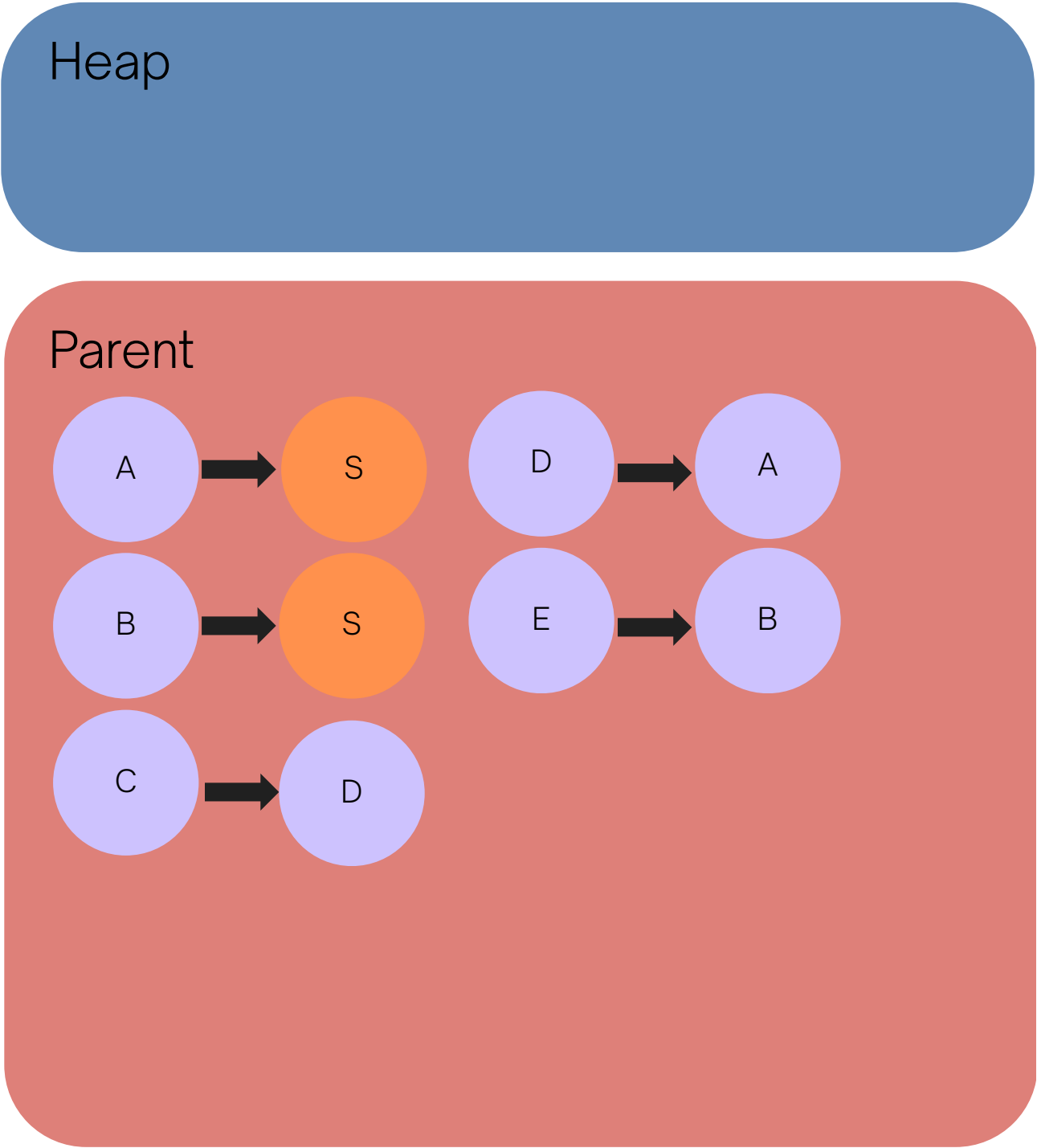
Dist		
S	:	0
A	:	2
B	:	3
C	:	6
D	:	5
E	:	7



SHORTEST PATH

Example : From S to C

Dist		
S	:	0
A	:	2
B	:	3
C	:	6
D	:	5
E	:	7



SHORTEST PATH

Example : From **S** to **C**

Dist

S

:

0

A

:

2

B

:

3

C

:

6

D

:

5

E

:

7

Heap

Parent

A

→

S

D

→

A

B

→

S

E

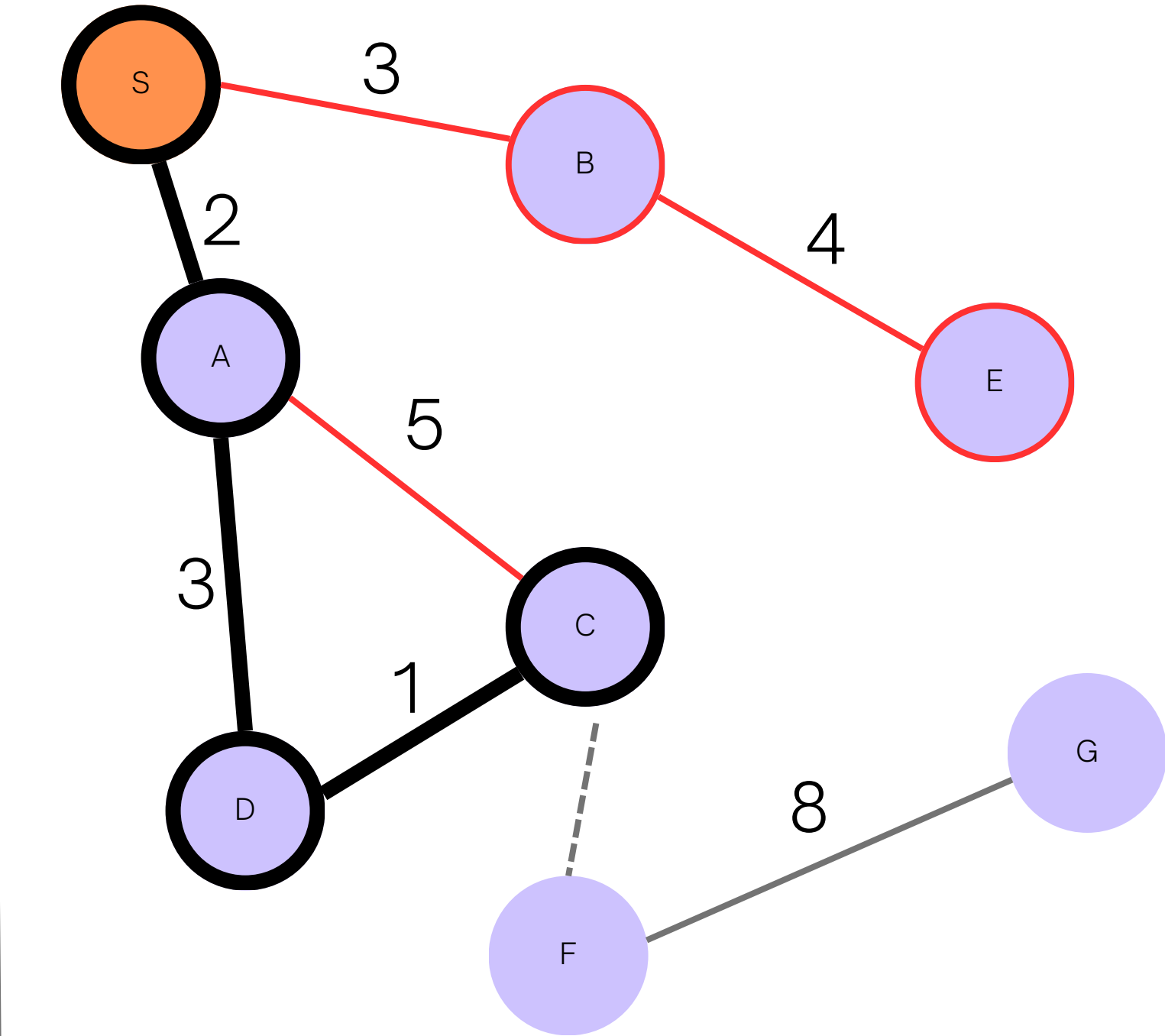
→

B

C

→

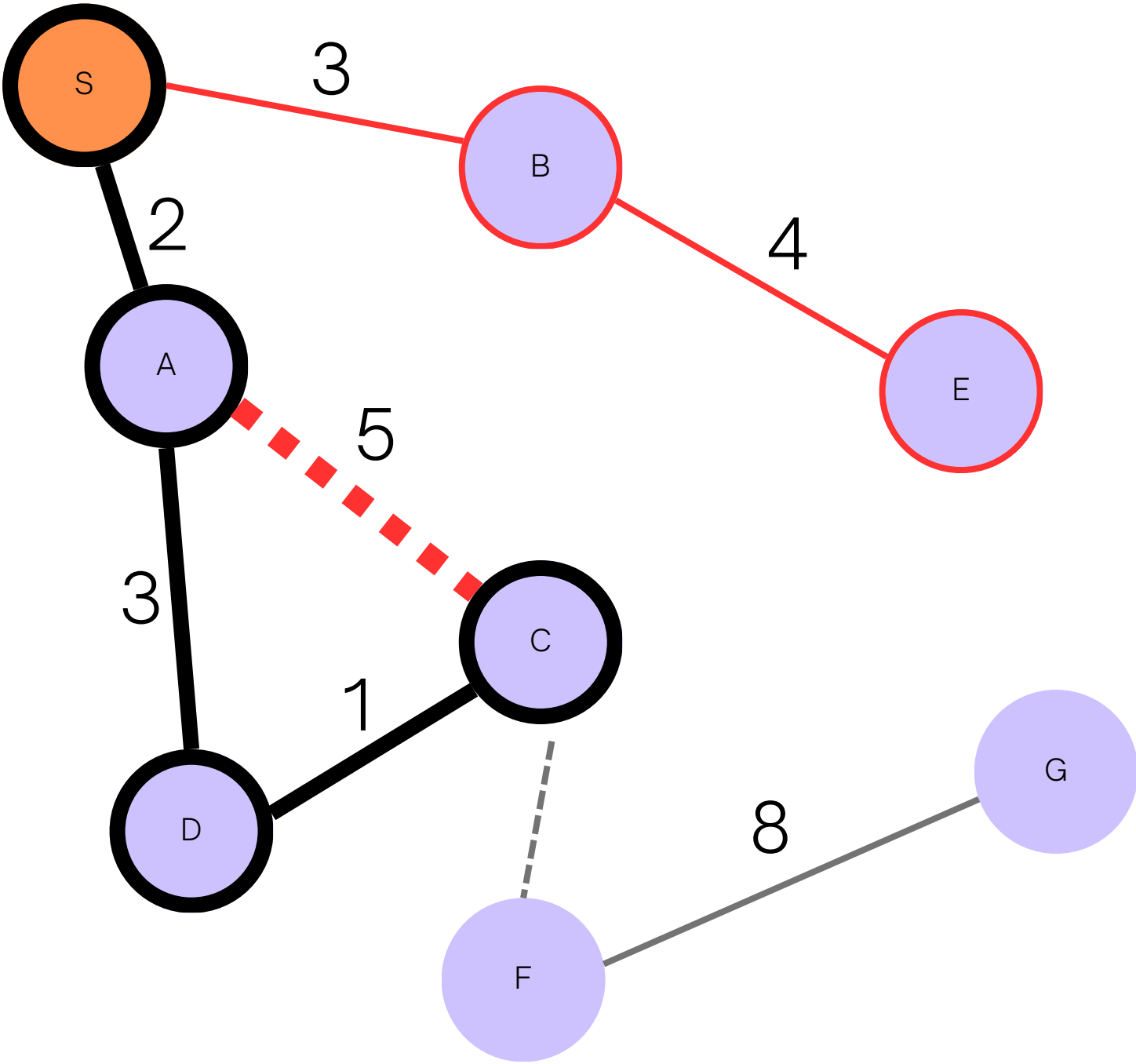
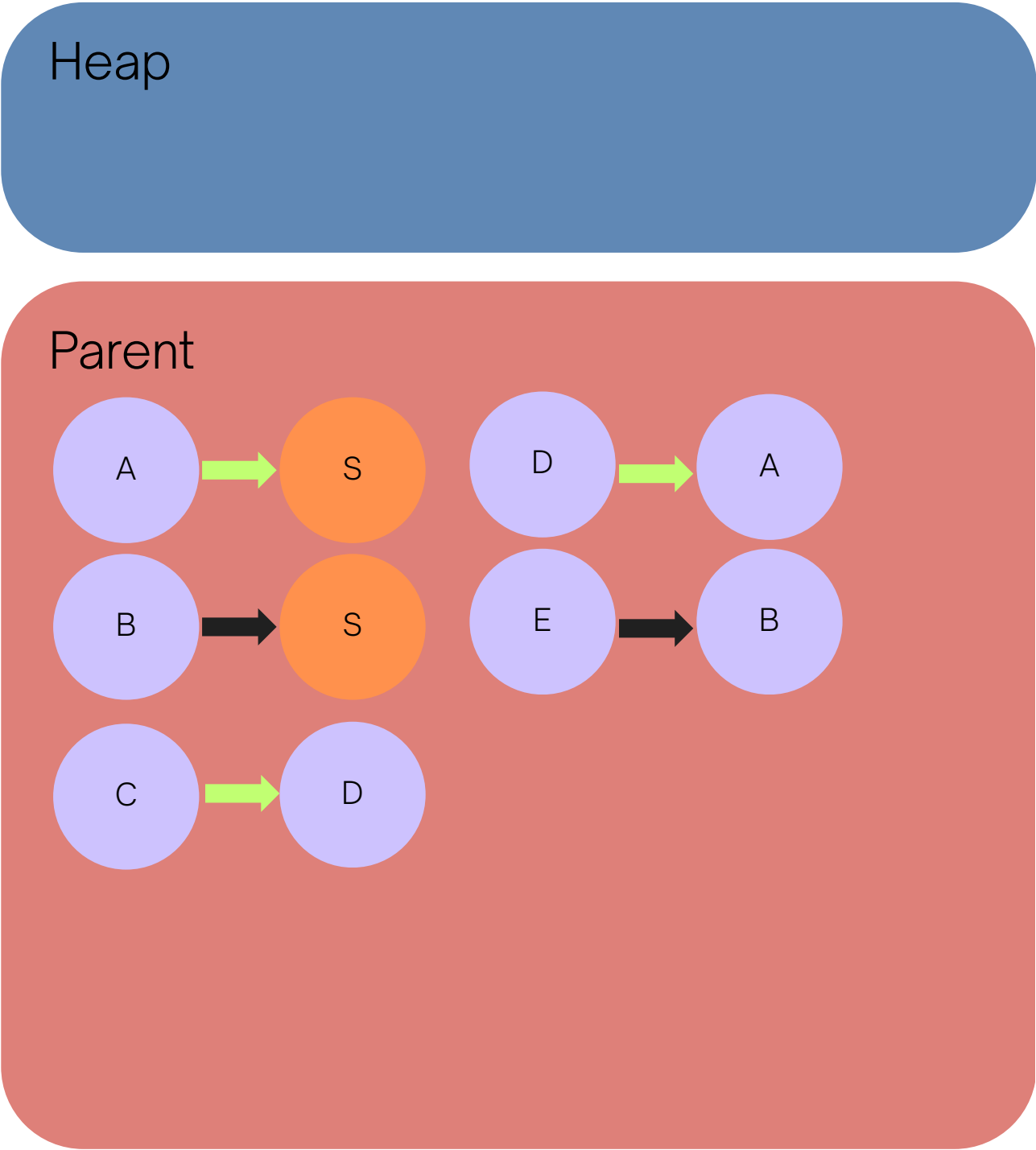
D



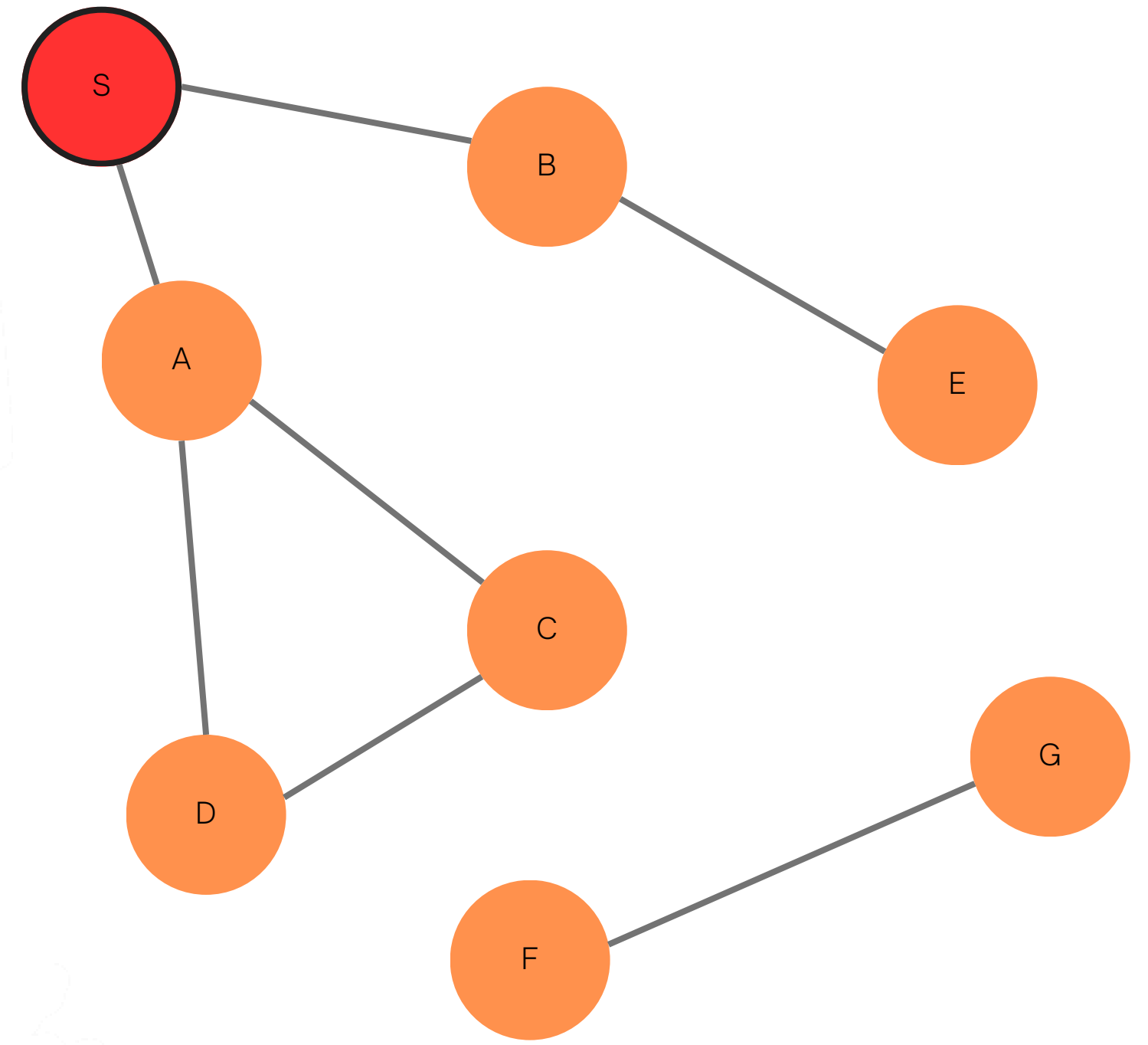
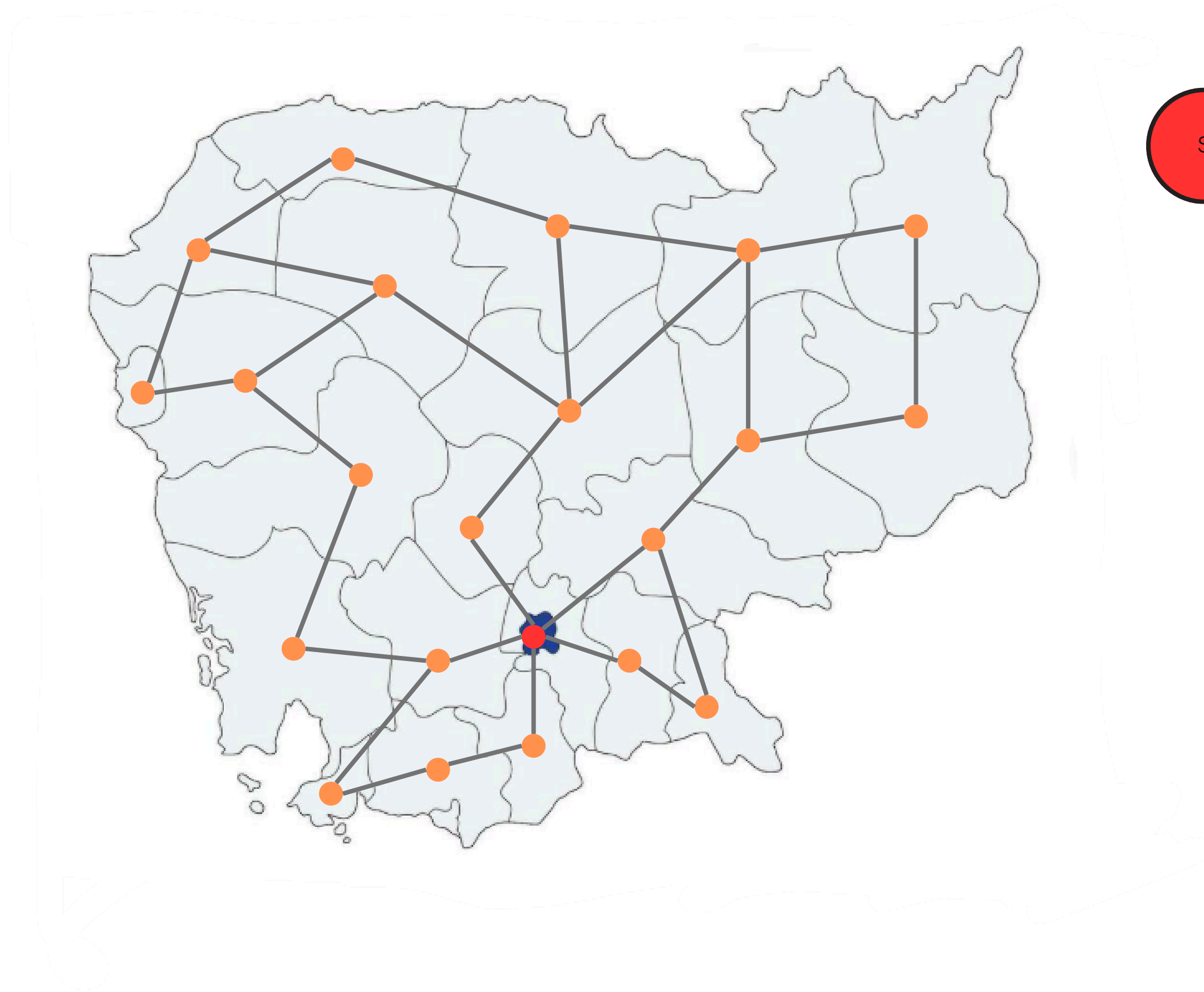
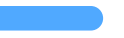
SHORTEST PATH

Example : From S to C

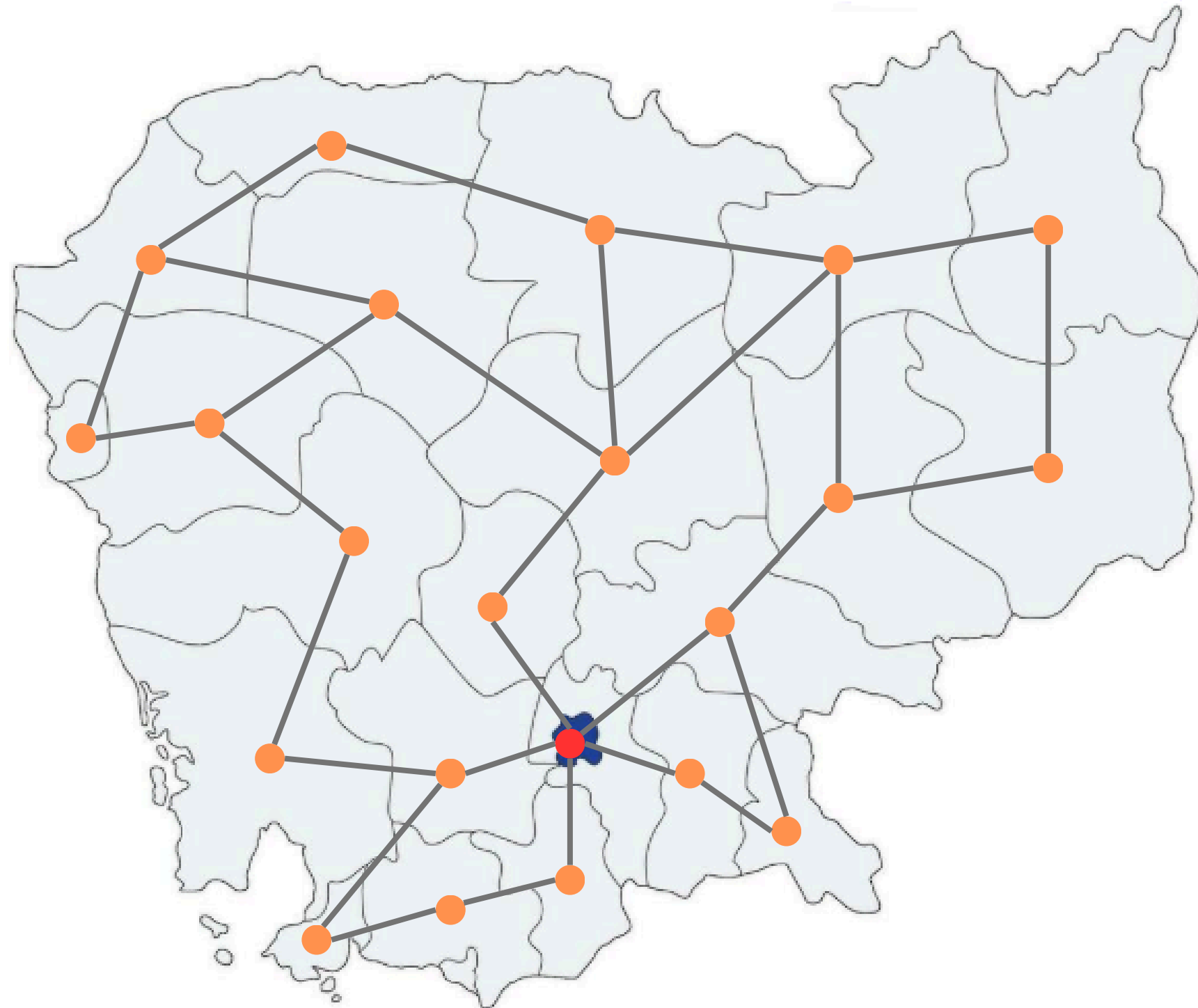
Dist		
S	:	0
A	:	2
B	:	3
C	:	6
D	:	5
E	:	7



EXAMPLE AND OUTPUT



EXAMPLE AND OUTPUT

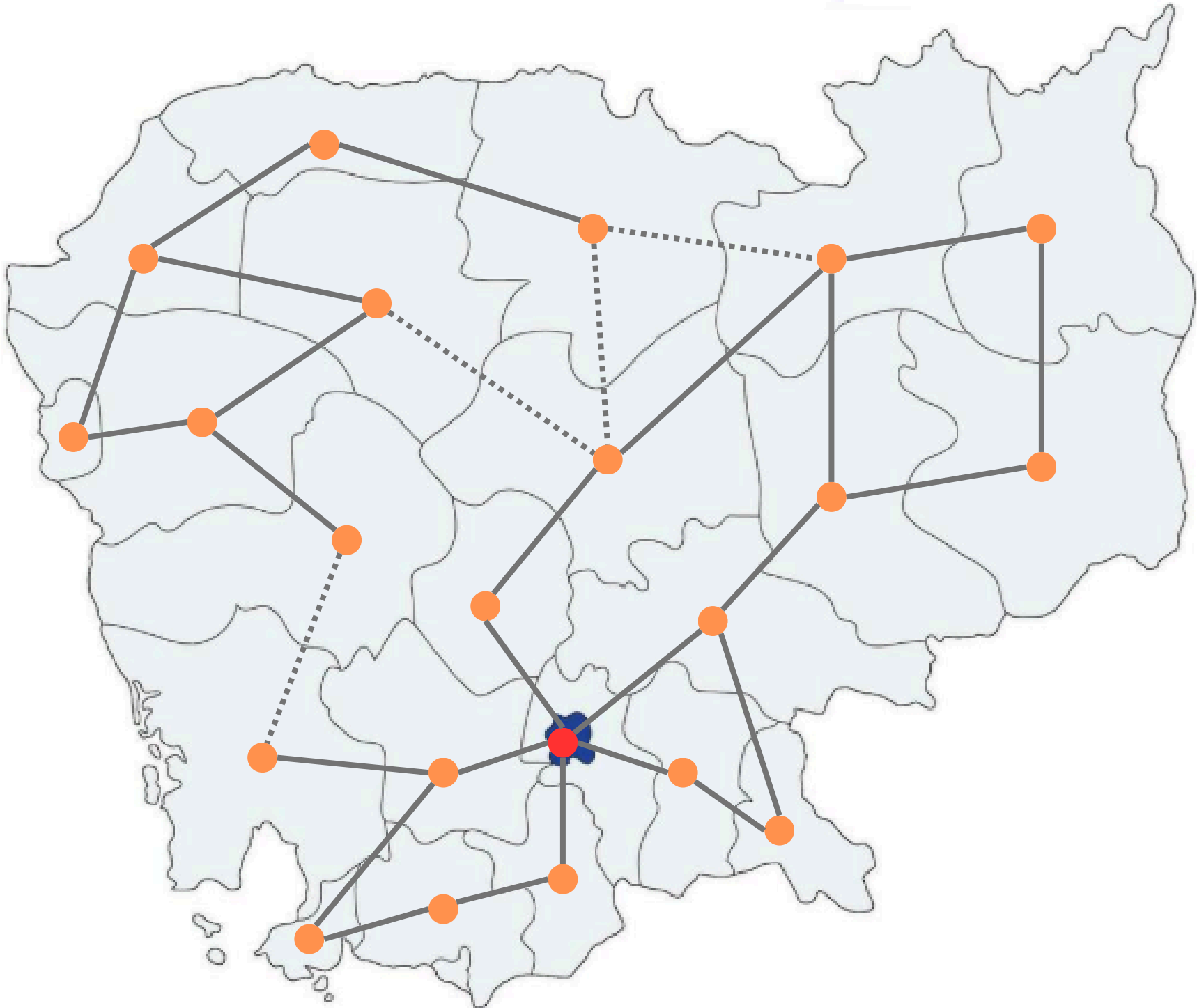
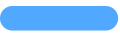


CONNECTIVITY

All node is connected to each other simulating real life

all 25 nodes in one vector

EXAMPLE AND OUTPUT



CONNECTIVITY

In case of disconnection

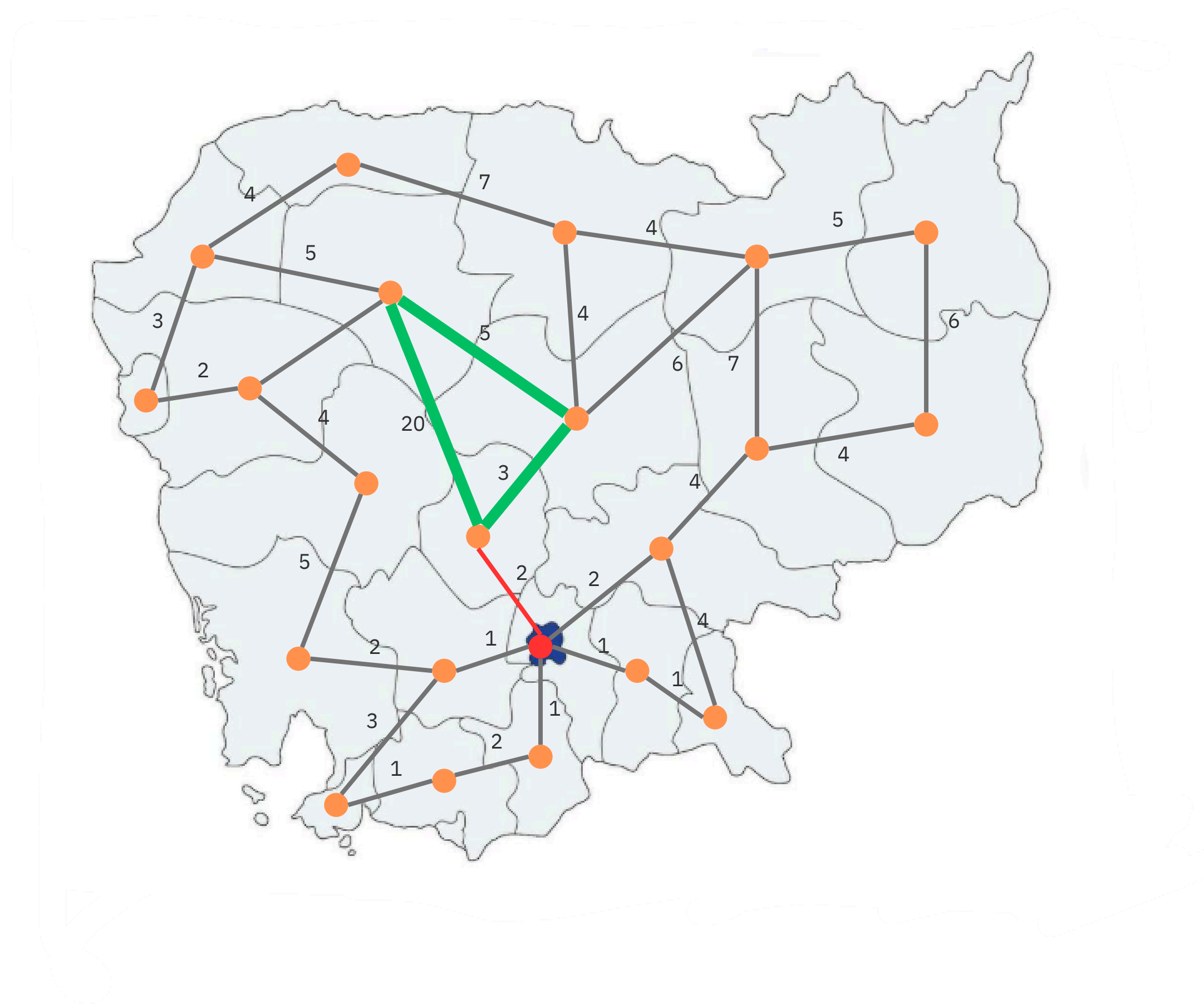
17 nodes in one vector



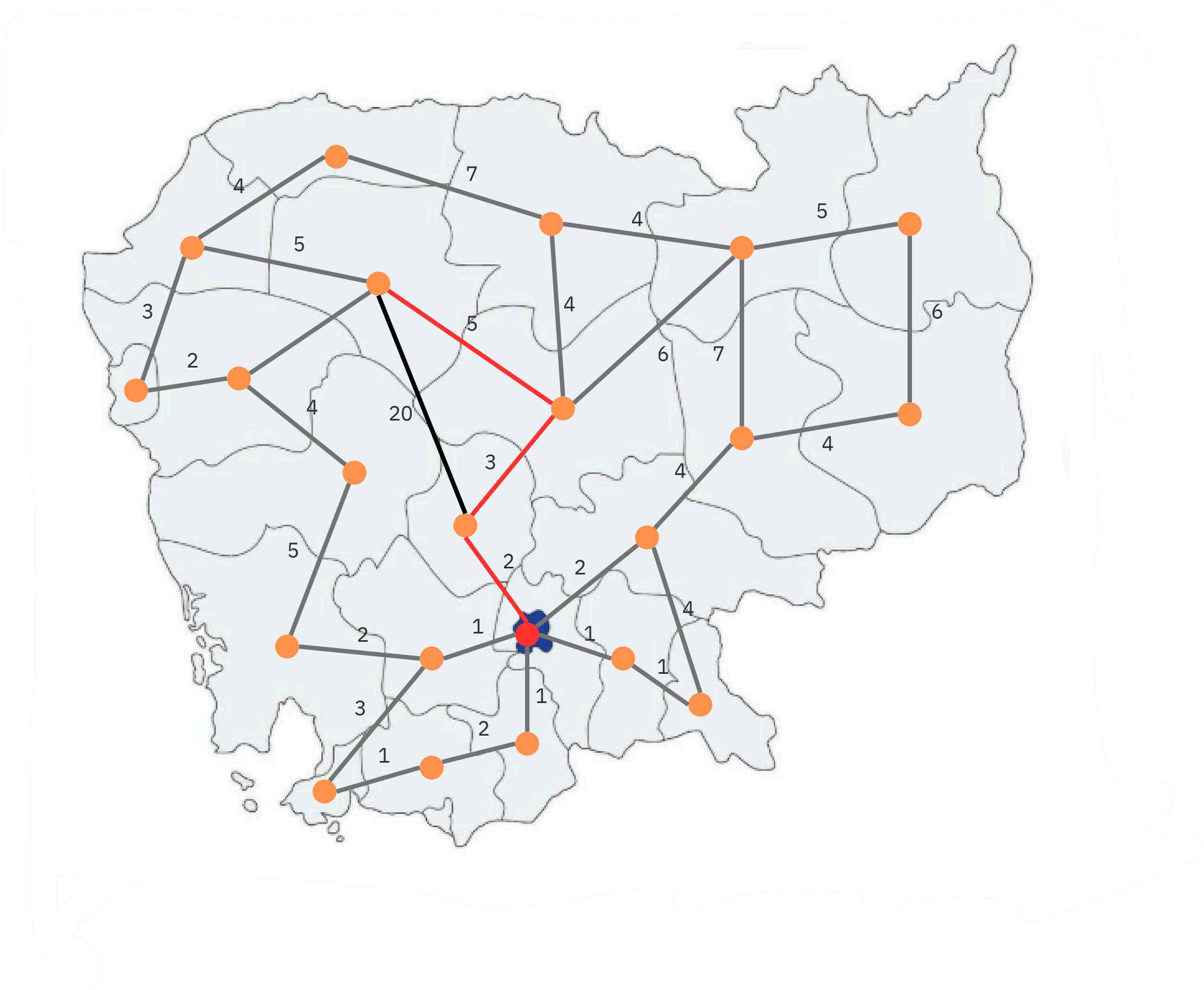
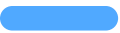
Island

SHORTEST PATH & LOOP

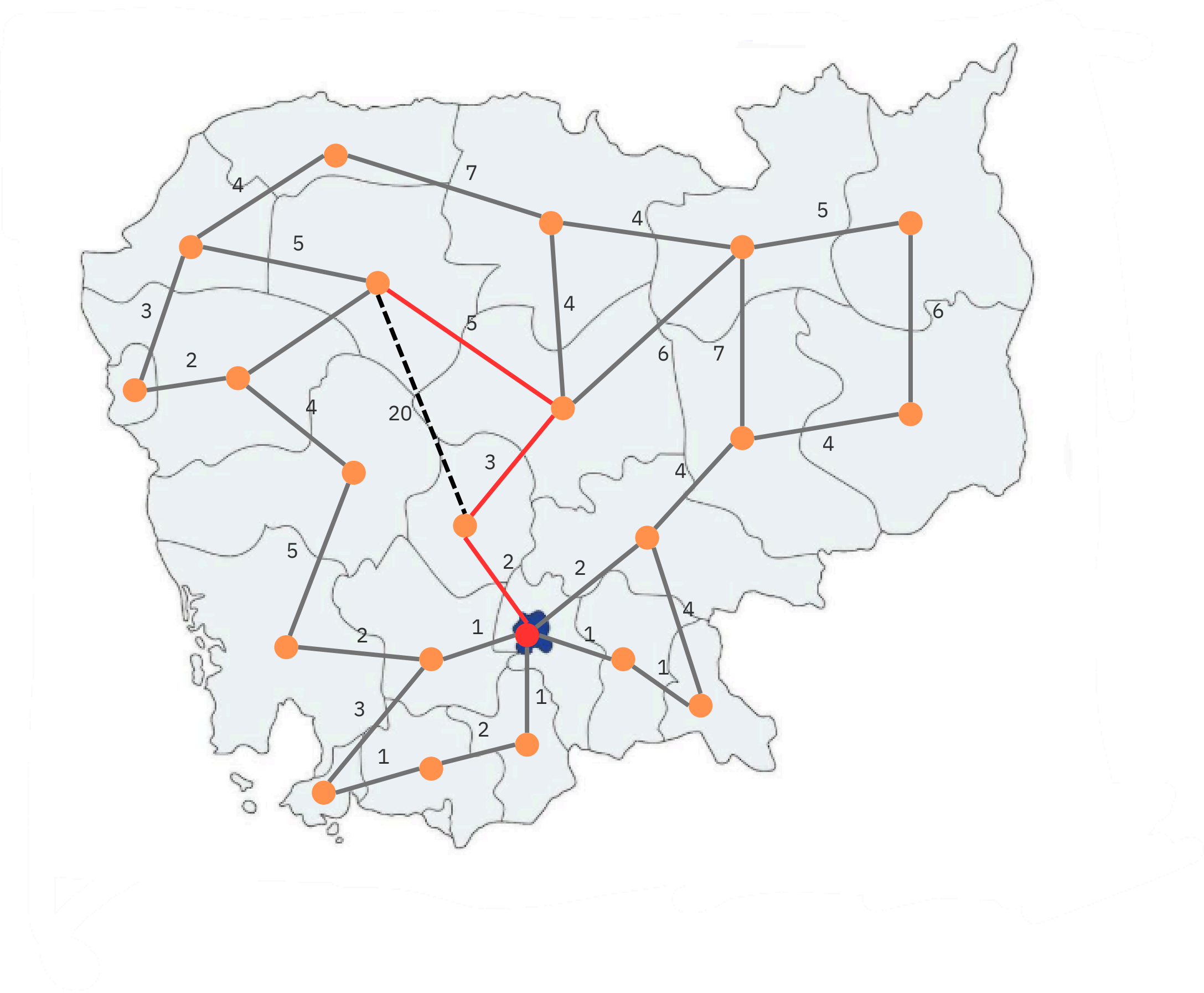
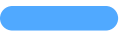
If an operator want find the shortest path to **Siem Reap** from the **Source**, the operator only need to enter in the province ID node and the algorithm find the path and also shows the value of the path.



EXAMPLE AND OUTPUT



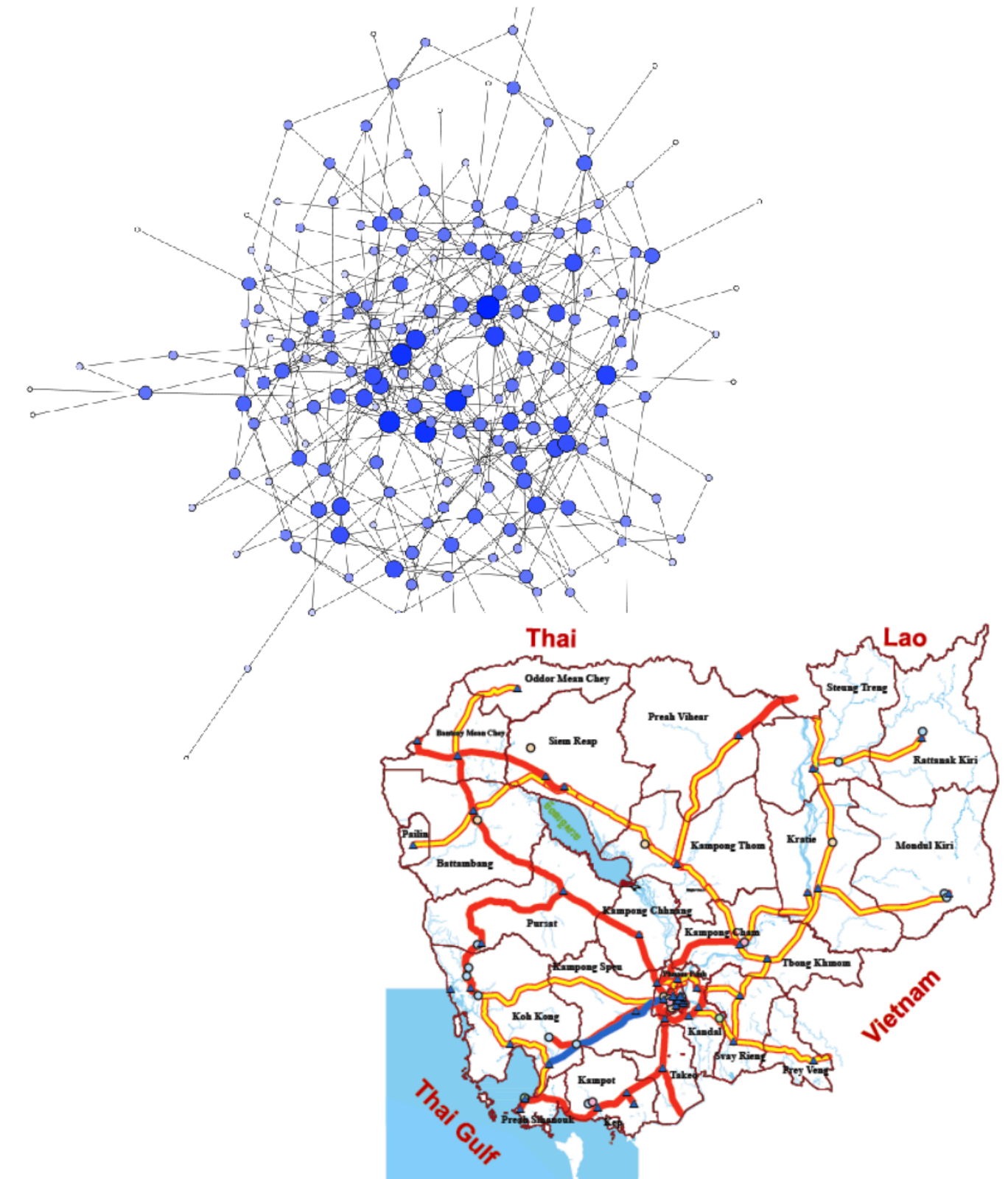
EXAMPLE AND OUTPUT



LIMITATIONS

and Assumptions

- All of this is just theoretical; it's only a graph, therefore, we do not have the correct properties for real-world usage, such as power, watts, flow, and resistance etc.
- This graph is very compressed to show an example of our system. On a larger scale would take up a lot of memory and load time.
- This graph is only an assumption. In a real-life scenario, we can't predict the electricity's path, real-life physics and laws do not apply here



CONCLUSION

AND LESSON LEARNT

To sum it all up, graphs are really effective for representing and analysing connectivity in an electrical network, but they can't fully model real-life behaviour.

We learnt about graph structures and their traversal algorithm, and how they solve connectivity problems while recognizing and addressing their real-world limitation



THANK YOU

DO NOT ASK PLEASEEE



We appreciate your attention! No further questions will be taken



+855-777-88373



www.ashxio.com



ashbookenhart@gmail.com



CADT, IDT building, room A209

